Red Hat Openshift 29 June to 3rd July 2026

Course designed and delivered by Jeganathan Swaminathan

Jeganathan Swaminathan

jegan@tektutor.org

https://www.tektutor.org

GitHub: https://github.com/tektutor

LinkedIn: www.linkedin.com/in/jeganathan-swaminathan-2a6a6a6

Day 1

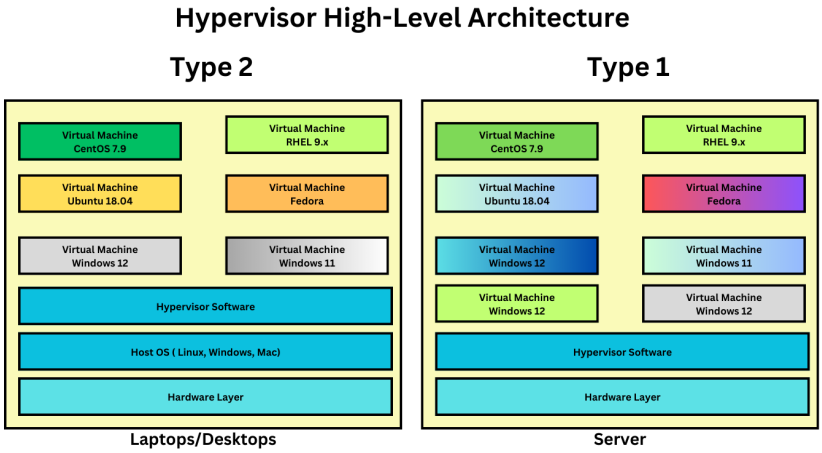
Info - Hypervisor Overview

```
- nothing but virtualization
- with virtualization/hypervisor software, we can run multiple OS side
by side on the same laptop/desktop/workstation/server
- there are 2 types of Hypervisors
  1. Type 1
     - a.k.a Baremetal Hypervisor
     - is used in Workstation & Server
     - performance wise it offers almost near native performance ( 3%
less performance as opposed to running a OS on direct H/W )
     - this can be installed directly on top of a H/W with no Operating
System
     - examples
       1. VMWare vSphere(vCenter) - Paid
       2. Linux KVM ( opensource & Free )
       3. Microsoft Hyper-V
  2. Type 2
     - a.k.a Hosted Hypervisor
     - this can be installed only top of a Host Operating System
( Windows, Mac OSX, Linux, etc )
     - is used in Workstation, Desktops & Laptops
     - examples
       - Oracle VirtualBox ( Free - Windows/Linux/Mac )
       - VMWare Wokstation ( Free - after Broadcom acquired VMWare -
Windows, Linux, Mac )
       - VMWare Fusion ( Mac OS-X - Need a commercial license )
       - Parallels ( Mac OS-X - Need a commercial license )
- virtualization helps organization save cost in many fold
  - with less physical servers, many virtual machines(Guest OS) can be
supported
  - saves cost in terms of procuring less physical servers
  - saves cost in terms of power consumption
  - saves cost in terms of Air Condition
  - saves cost in terms of Sound proofing
  - saves cost in terms of Real estate rental/leasing
- this type of virtualization is called heavy-weight
  - because each VM requires dedicated Hardware resources
  - each VM runs a fully functional OS with its own dedicated OS Kernel
- each VM, represents one fully function Operating System
```

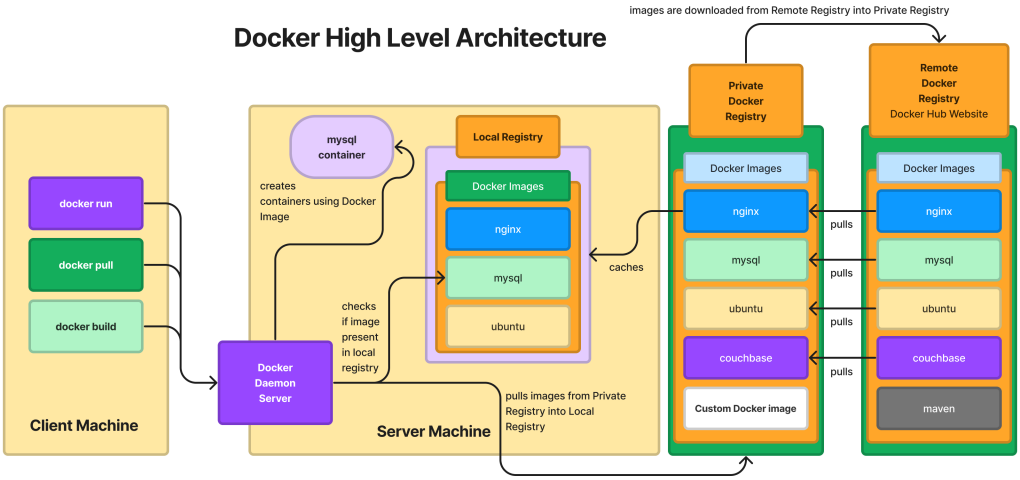
Info - Containerization

```
- application virtualization technology
- it is light weight, because each container represents a single
application not an Operating System
- all the containers that runs on top a OS shares the Hardware resources
available on the underlying OS
- containers will never be able to replace an Operating System or
Virtualization
- in fact, in real-world scenarios, virtualization and containerization
are used in combination
- it is a linux technology
- linux kernel features that enables containerization
  1. Namespace
     - helps isolation of containers from each other
  2. Control Groups (CGroups )
     - it helps us apply resource quota restrictions like
       - we can restrict how much RAM, disk and cpu can be utilized by a
container
- container runtime
  - is a low-level software that manages container images and containers
  - it is not user-friendly, hence end-user like us never use container
runtime directly
  - examples
    - runC
    - cRun
    - CRI-O
- container engine
  - is a high-level software that manages container images and
containers
  - it is very user-friendly, but they depend on container runtimes
internally to manage images and containers
  - examples
    - docker
    - podman
```

Info - Hypervisor High-Level Architecture



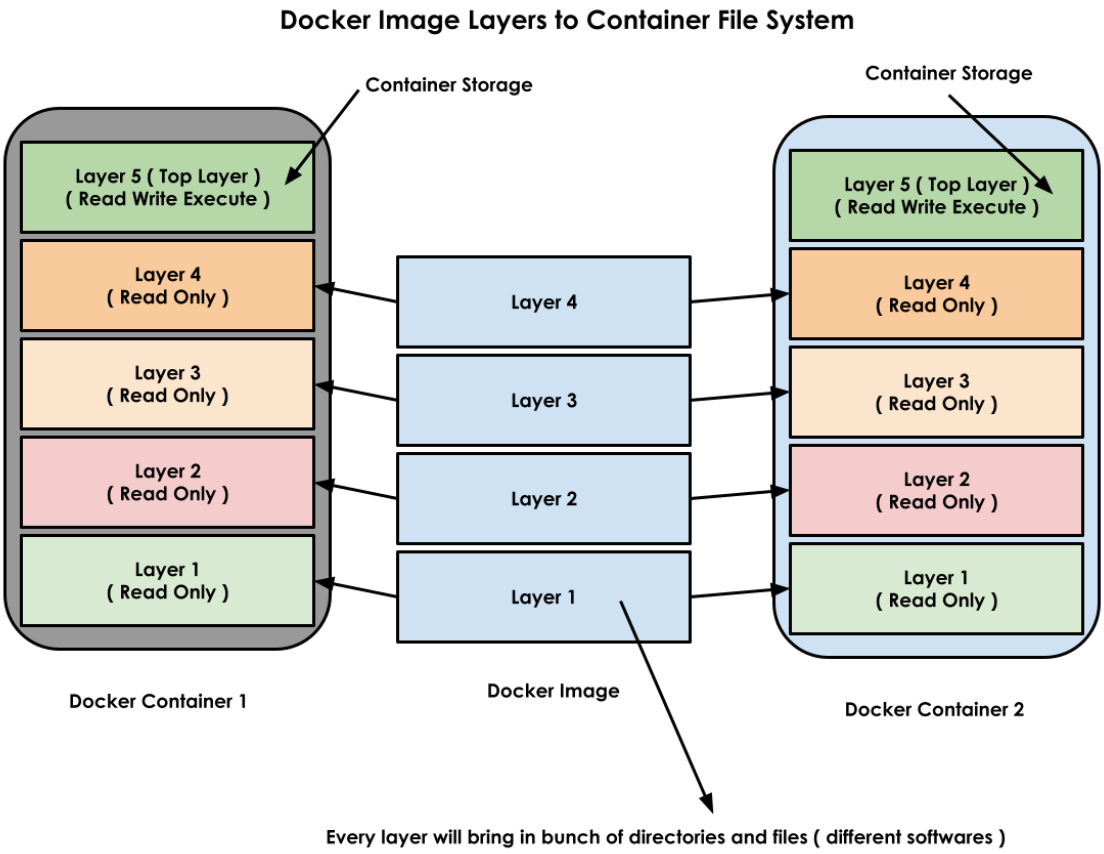
Info - Docker High-Level Architecture



Info - Docker image

```
- is a blueprint of a docker container
- you can imagine docker image similar to Window12.iso (OS Image) or
Ubuntu24.04.iso(OS Image)
- with the help of a docker image, we can create any number of docker
```

```
containers
- docker images => typically has one application and its dependencies
( libraries, dependent tools, etc., )
- docker images are conservatively built, which means it only contains
bare minimum tools required to run a specific appliction
- every docker images gets a unique name and unique id ( 256-bit HASH )
```



Info - Docker Container

```
- is the running instance of a Docker Image ( Container Image )
- each container gets a unique name and ID (SHA Hash)
- each container gets its own dedicated software defined network stack
( 7 OSI Layers )
- each container get its own file system ( files & folders )
- each container uses about 7 namespaces
- each container gets one or more IP addresses ( generally Private IP )
```

Info - Docker Registry

```
- is a collection of multiple docker images
- there are 3 types
```

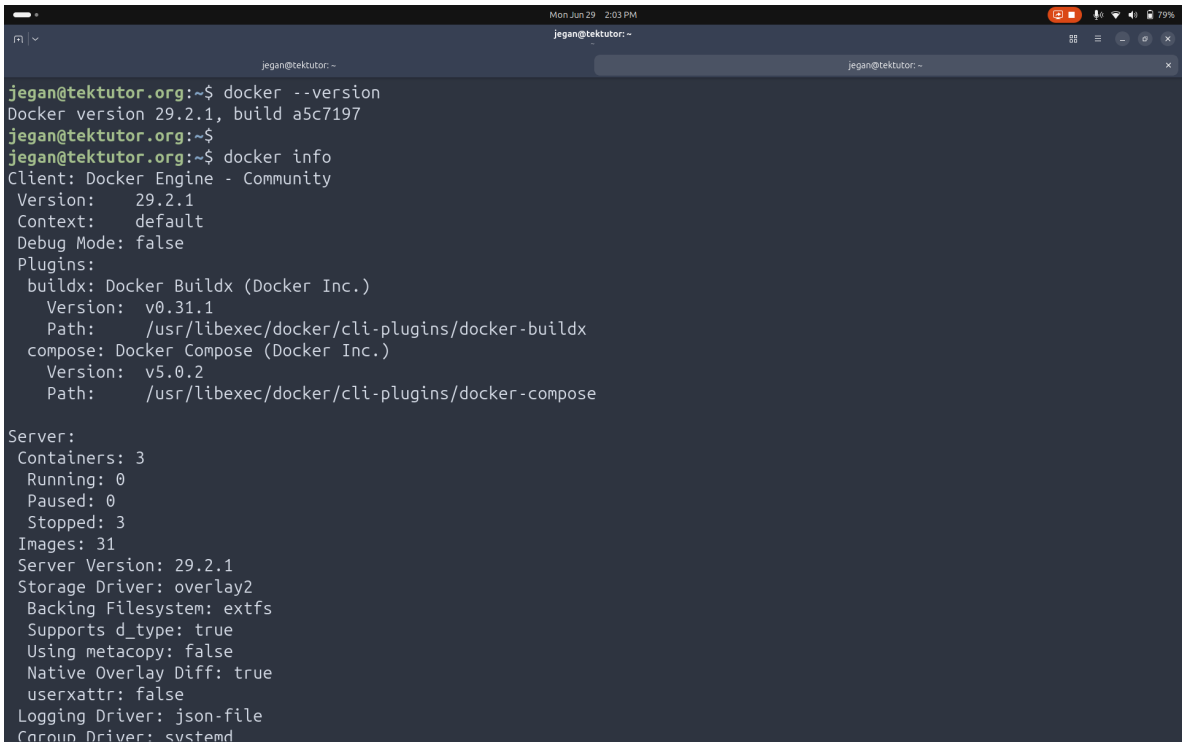
```
  1. Local Docker Registry
     - is a folder created and maintained by Docker server on the same
machine where it runs
     - for example, in linux distros, local docker registry is
maintained under directory /var/lib/docker
  2. Remote docker registry
     - is a web site ( hub.docker.com ) maintained by Docker Inc and
opensource community
  3. Private Docker registry
     - can be setup using JFrog Artifactory or Sonatype Nexus
```

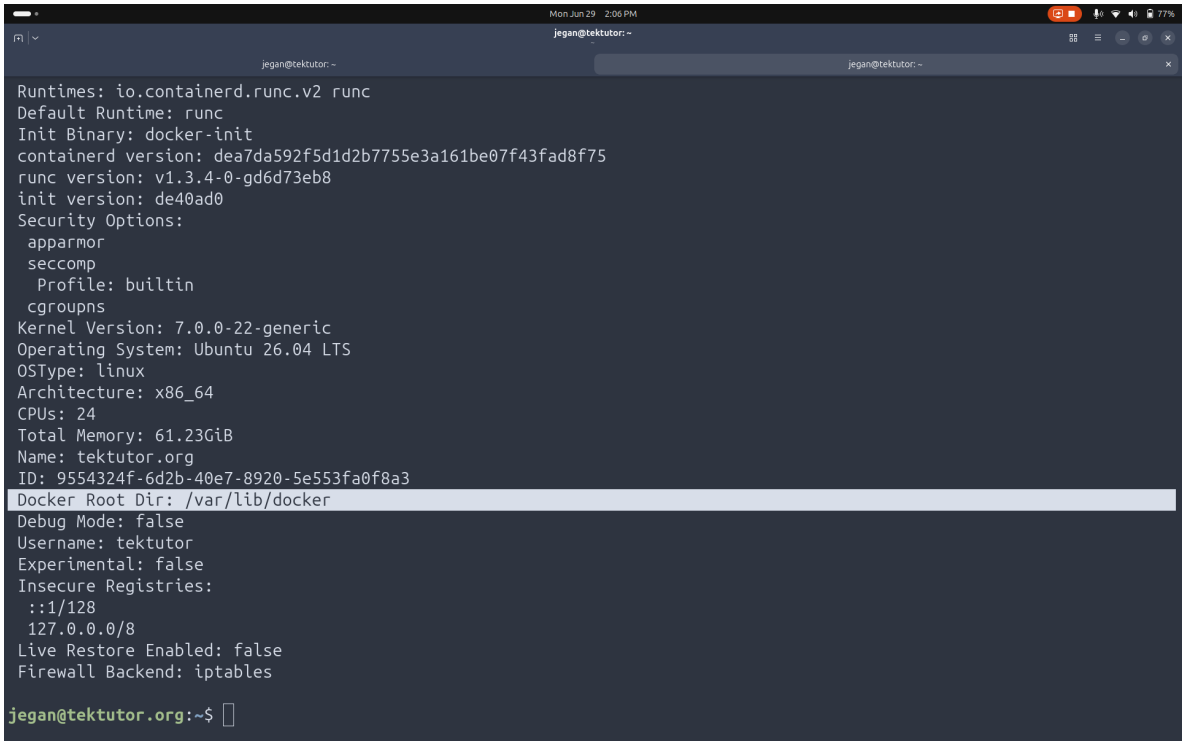
Demo - Installing Docker community edition (Docker CE)

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/
keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin -y
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker $USER
newgrp docker
```

Lab - Checking docker details

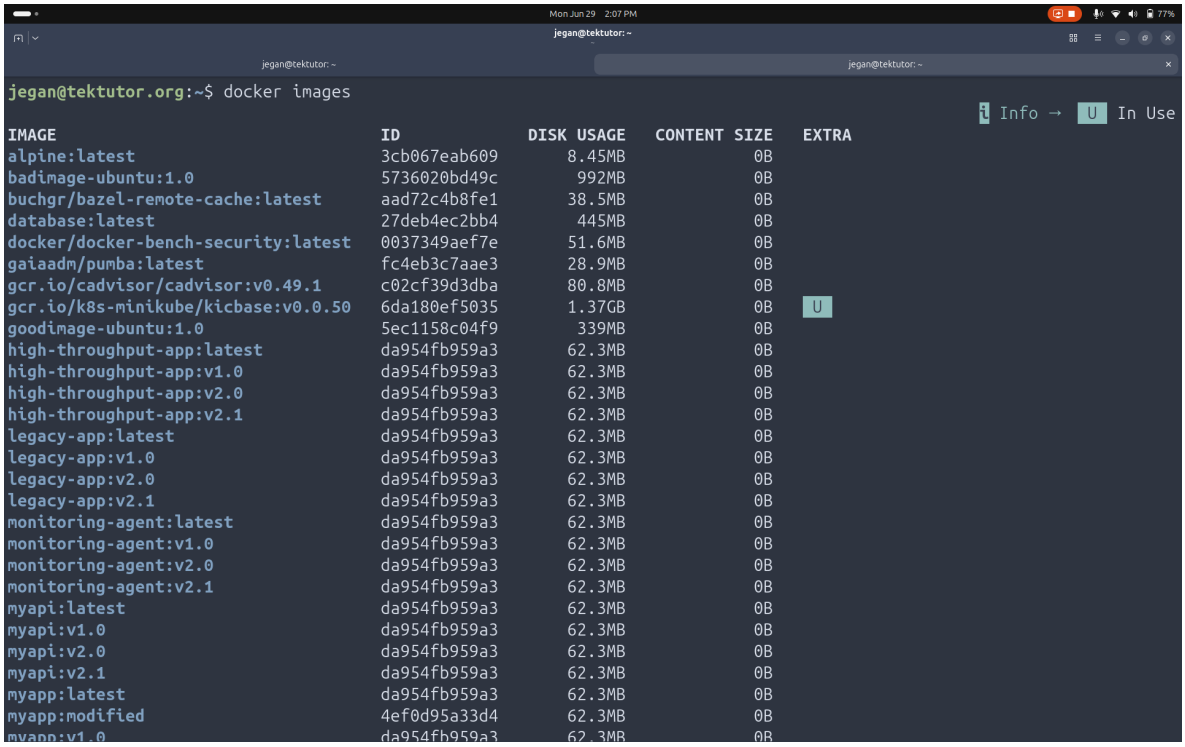
```
docker --version
docker info
```





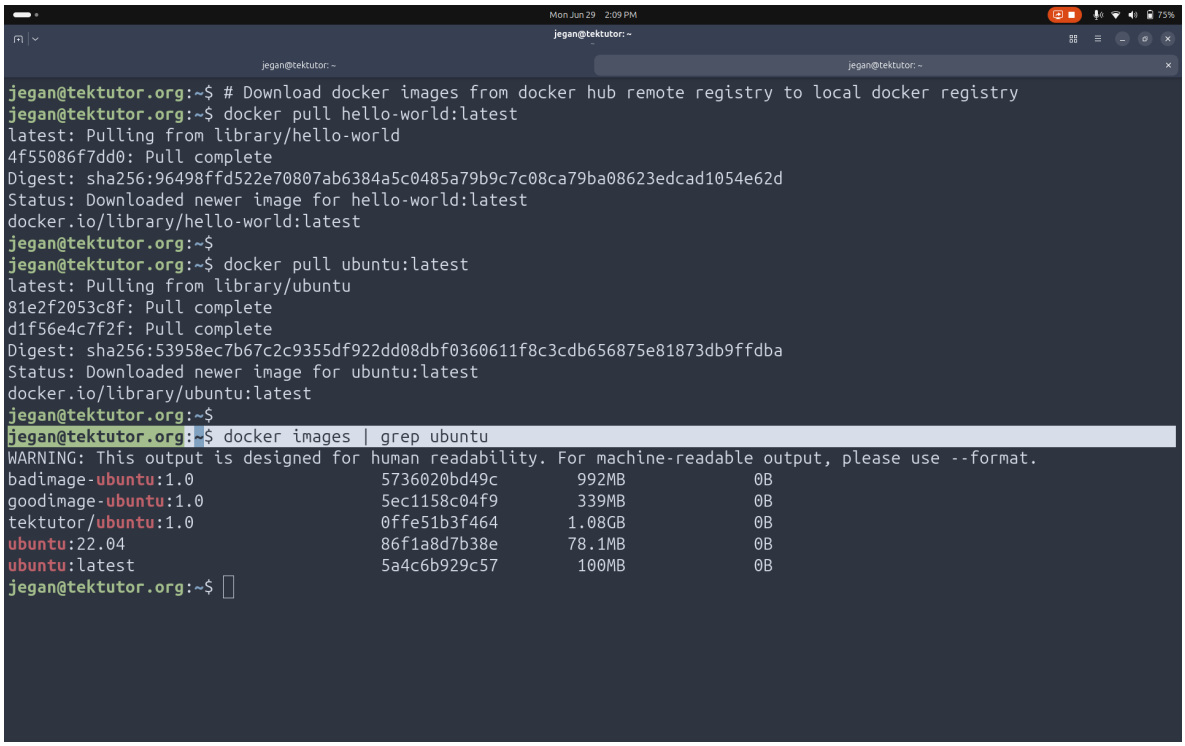
Lab - Listing docker images from your local docker registry

```
docker images
```



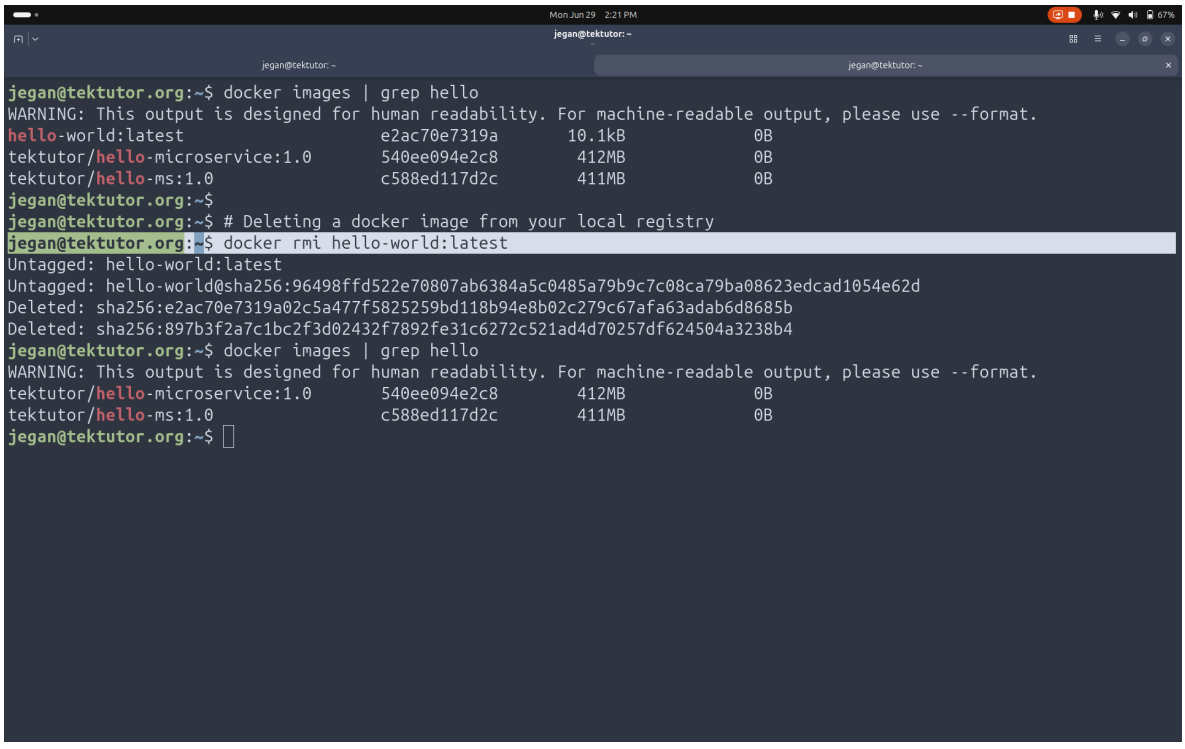
Lab - Downloading docker images from docker hub to local registry

```
docker pull hello-world:latest
docker pull ubuntu:latest

docker images
```



Lab - Deleting a docker image from your local registry

```
docker images | grep hello
docker rmi hello-world:latest
docker images | grep hello
```



Lab - Creating a container in the interactive(foreground) mode

From terminal 1

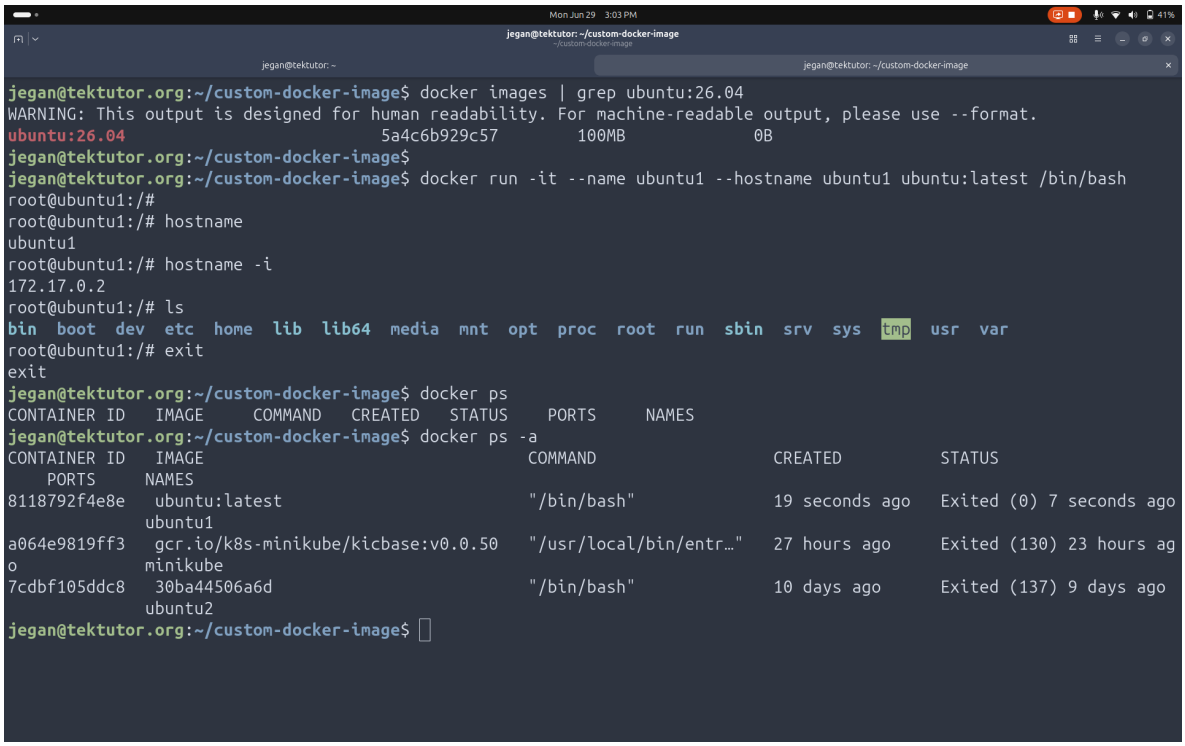
```
docker run -it --name ubuntu1-jegan --hostname ubuntu1-jegan ubuntu:latest /bin/
bash
hostname
hostname -i
ls
exit
```

List all running containers from second terminal

```
docker ps | grep jegan
```

List all running containers you created

```
docker ps -a | grep jegan
```



Lab - Create a custom docker image

In your home directory, create a folder something like custom-docker-image

```
cd ~/
mkdir -p custom-docker-image
cd custom-docker-image
touch Dockerfile
```

Add the below content to your Dockerfile, you can run gedit to open windows notepad like editor

```
FROM ubuntu:26.04

RUN apt update && apt install -y net-tools iputils-ping
RUN apt update && apt install -y default-jdk maven
```

Build your custom docker image

```
cd ~/custom-docker-image
docker build -t ubuntu-jegan:1.0 .
docker images | grep jegan
```

Create a container using your custom docker image

```
docker run -dit --name c1-jegan --hostname c1-jegan ubuntu-jegan:1.0 /bin/bash
docker exec -it c1-jegan /bin/bash
hostname
hostname -i
ping 8.8.8.8
ifconfig
ls
exit
```

List and see the container should still run because we created this container in background

```
docker ps
```

Lab - Stopping a running container

```
docker ps | grep c1-jegan
docker stop c1-jegan
```

```
docker ps | grep c1-jegan
docker ps -a | grep c1-jegan
```

Lab - Start an exited container

```
docker ps -a | grep c1-jegan
docker start c1-jegan
docker ps | grep c1-jegan
```

Lab - Restart a running container

```
docker ps a | grep c1-jegan
docker restart c1-jegan
docker ps | grep c1-jegan
```

Lab - Deleting a running container

In order to delete a running container gracefully, you must stop it first

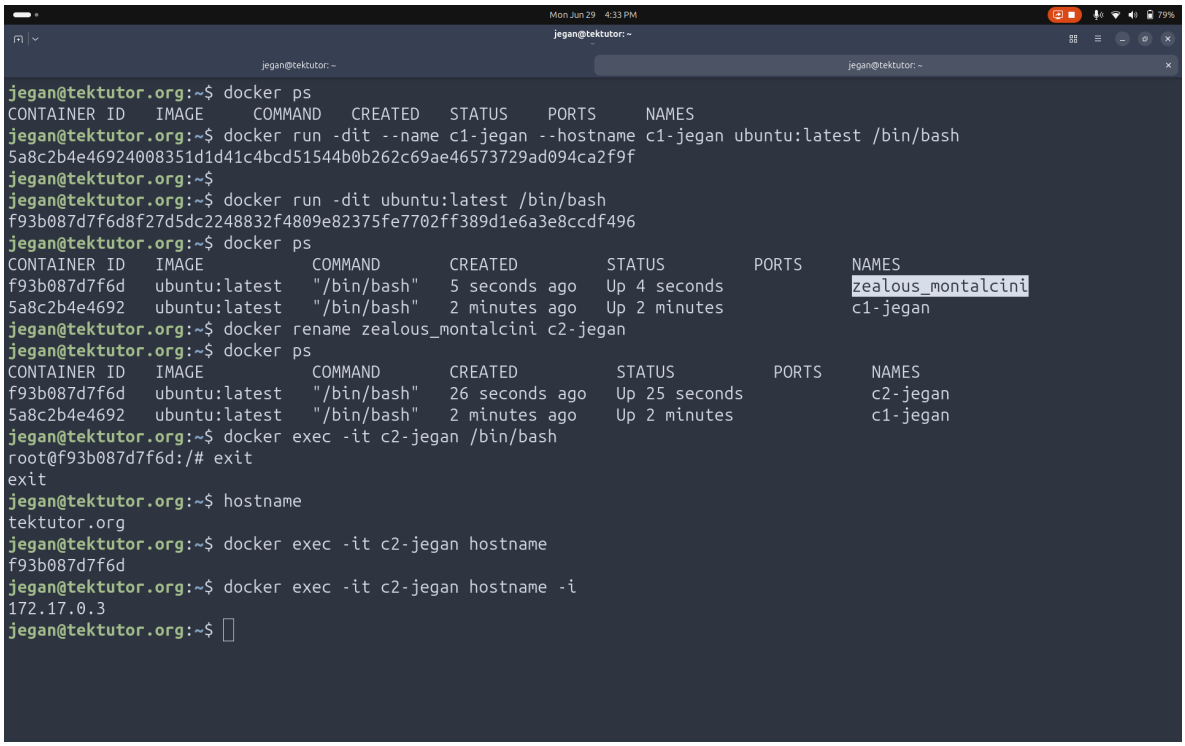
```
docker stop c1-jegan
docker rm c1-jegan
docker ps -a
```

In order to delete a running container forcibly

```
docker rm -f c1-jegan
docker ps -a
```

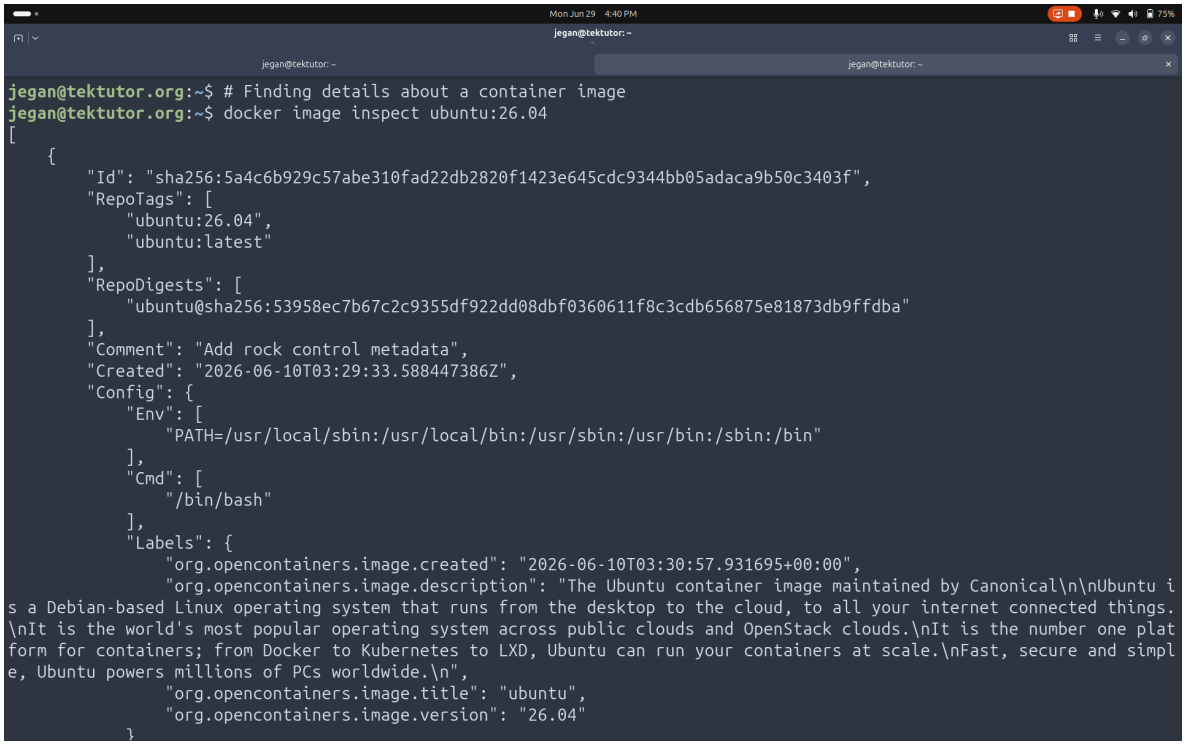
Lab - Rename a container

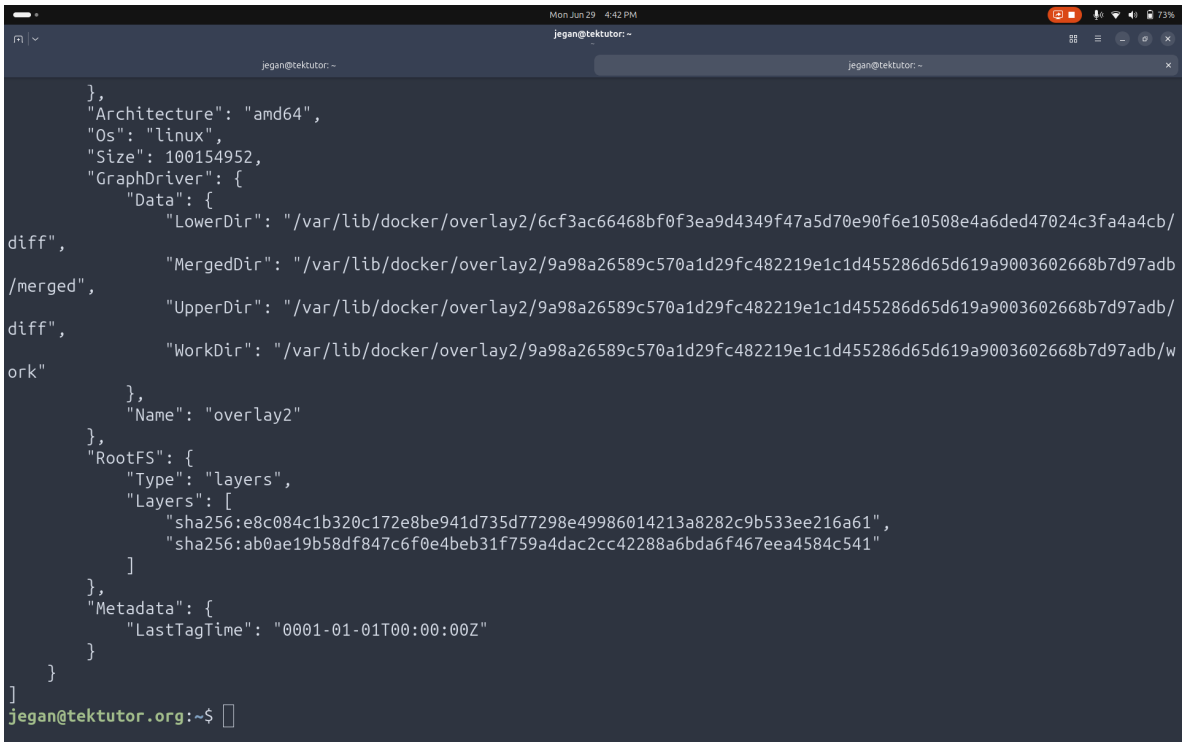
```
docker run -dit ubuntu:latest /bin/bash
docker ps a
docker rename zealous_montalcini c2-jegan
docker ps
```



Lab - Finding details about a container image

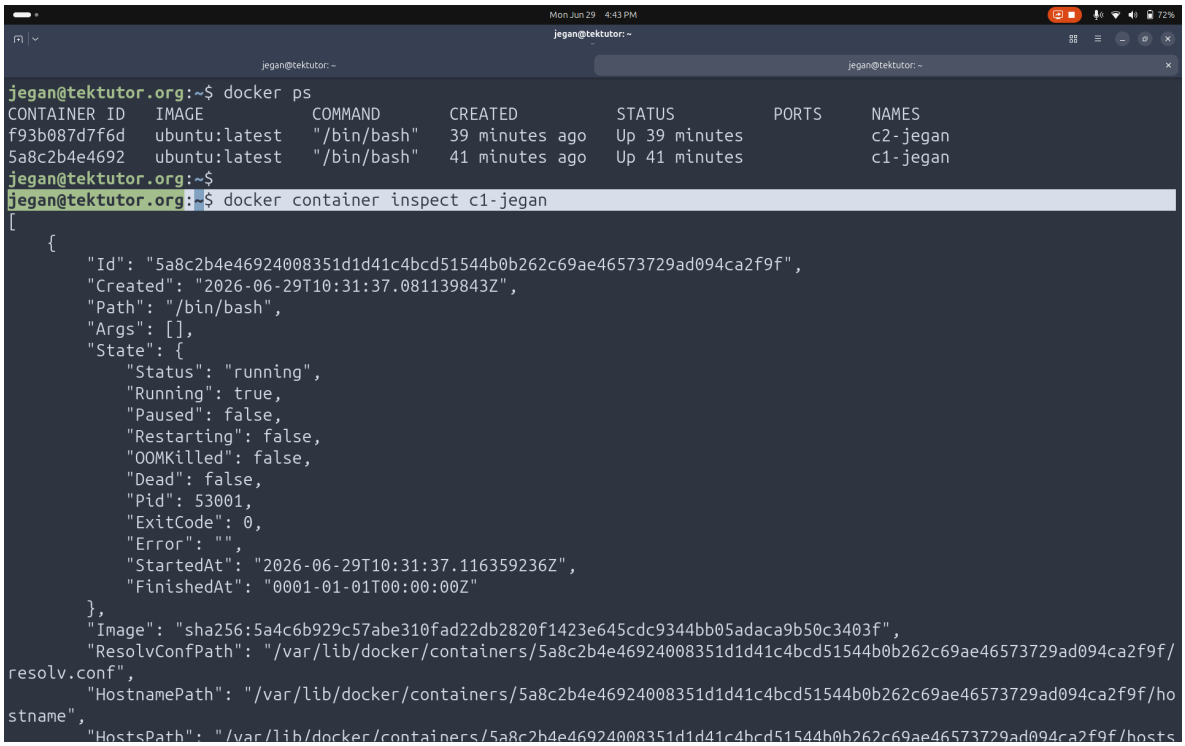
```
docker image inspect ubuntu:26.04
```

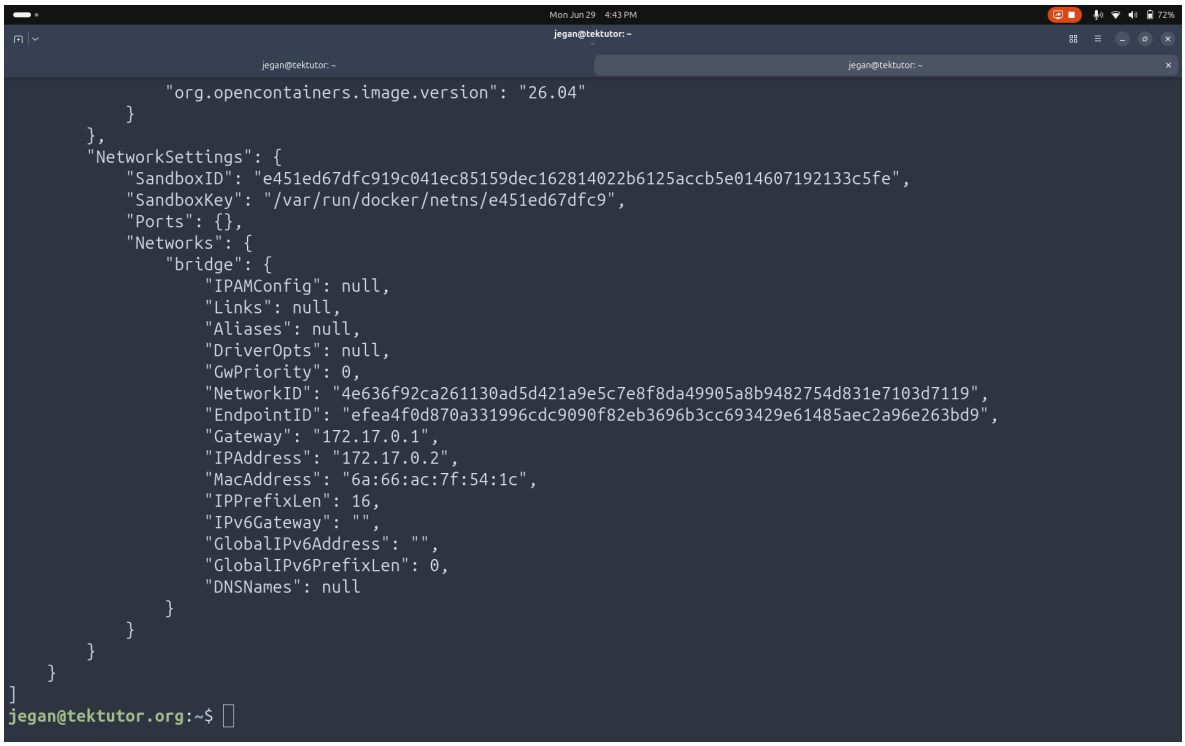




Lab - Finding details about a container

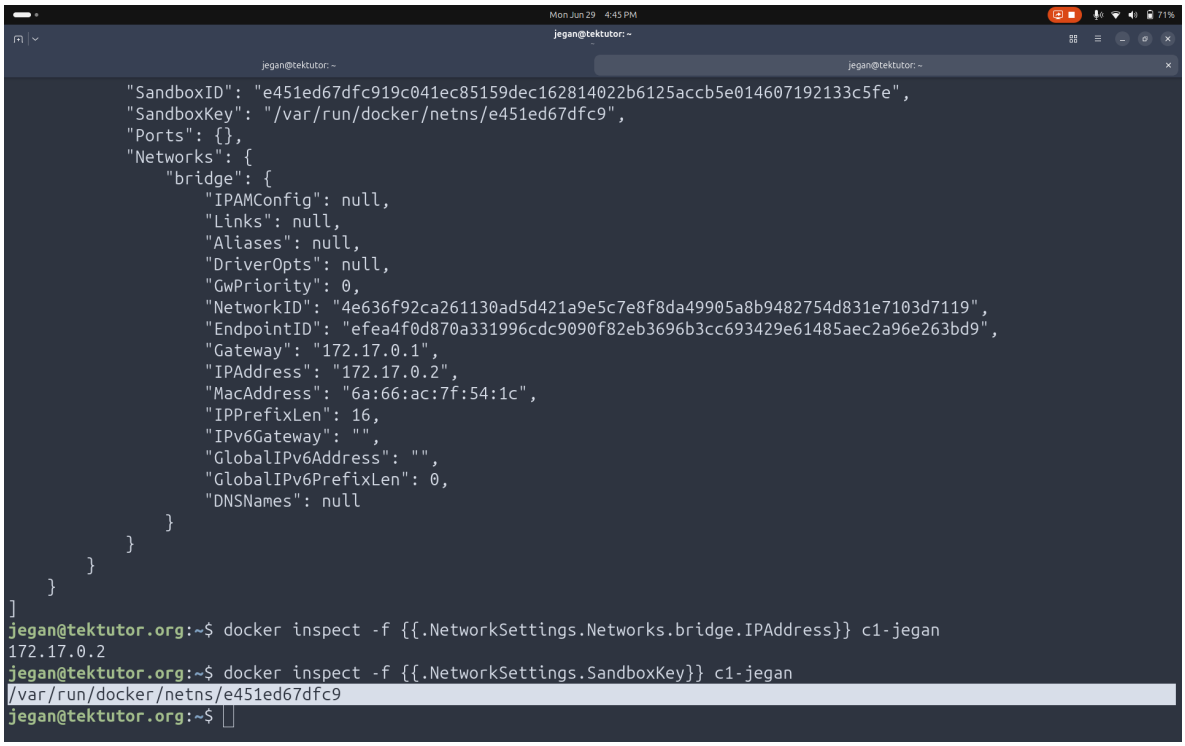
```
docker container inspect c1-jegan
docker inspect c1-jegan
```



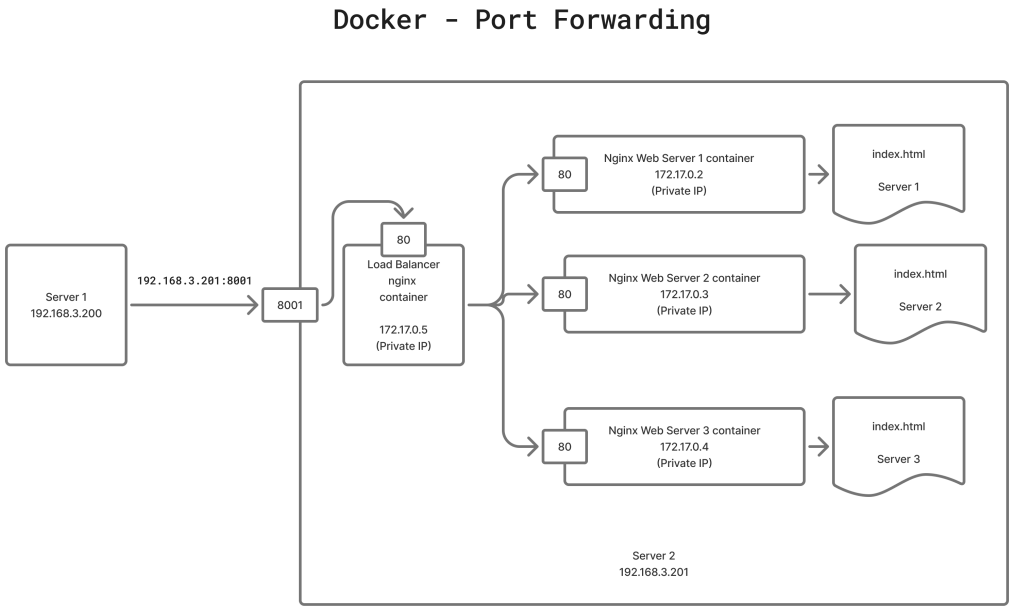


Lab - Finding IP address of a container

```
docker inspect c1-jegan | grep IPA
docker inspect -f {{.NetworkSettings.IPAddress}} c1-jegan
docker exec -it c1-jegan hostname -i
```



Lab - Setup a Load balancer using containers



Let's create 3 web server containers using ngnix:latest image from Docker Hub website

```
docker run -d --name nginx1-jegan --hostname nginx1-jegan nginx:latest
docker run -d --name nginx2-jegan --hostname nginx2-jegan nginx:latest
docker run -d --name nginx3-jegan --hostname nginx3-jegan nginx:latest
```

List all running containers

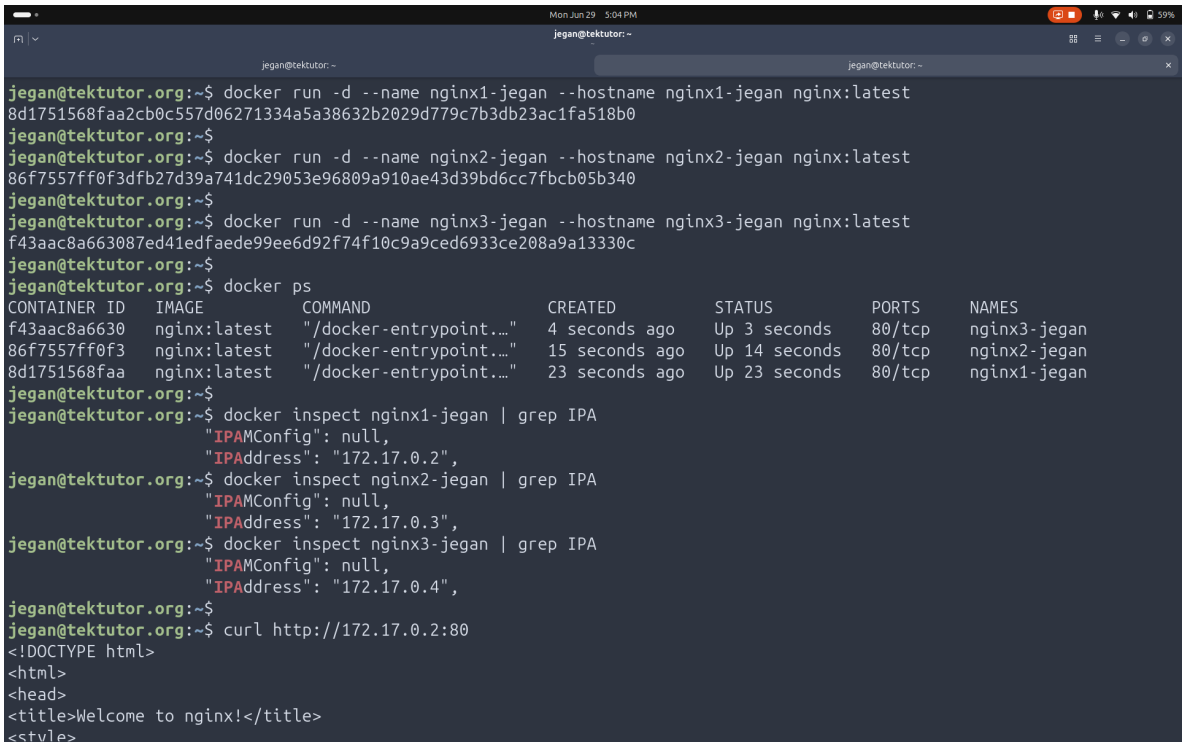
```
docker ps
```

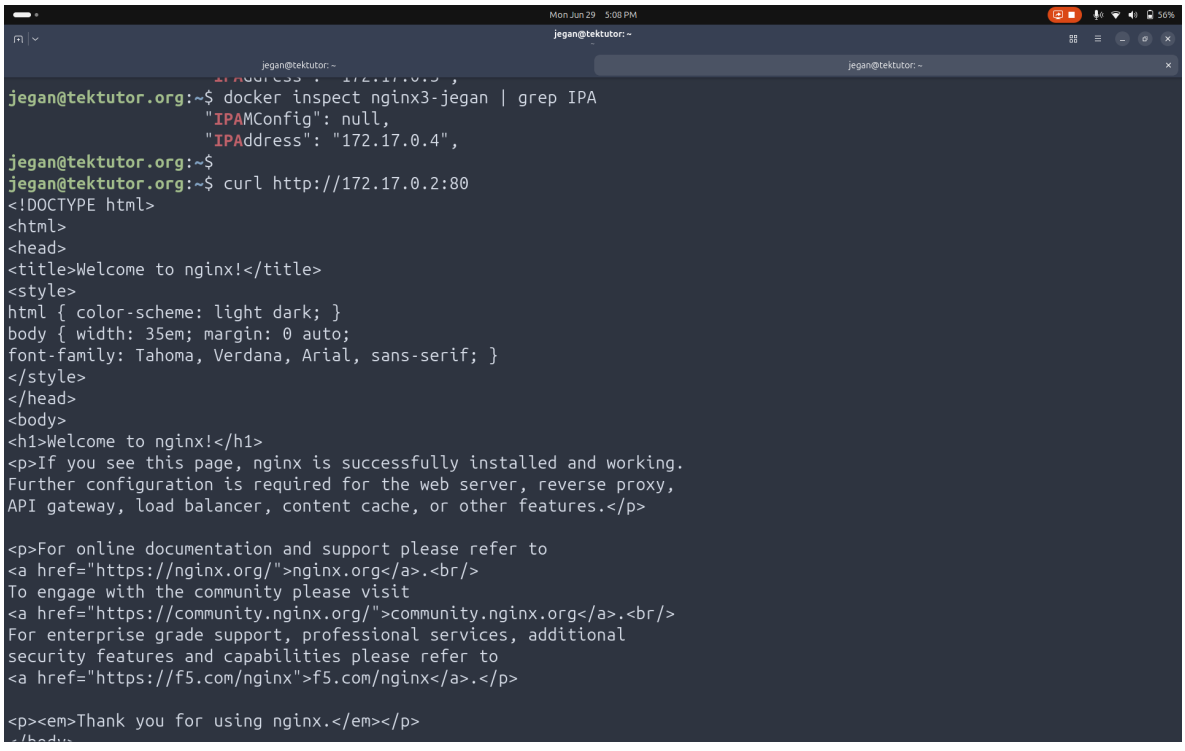
Find the IP addresses of the 3 webserver containers

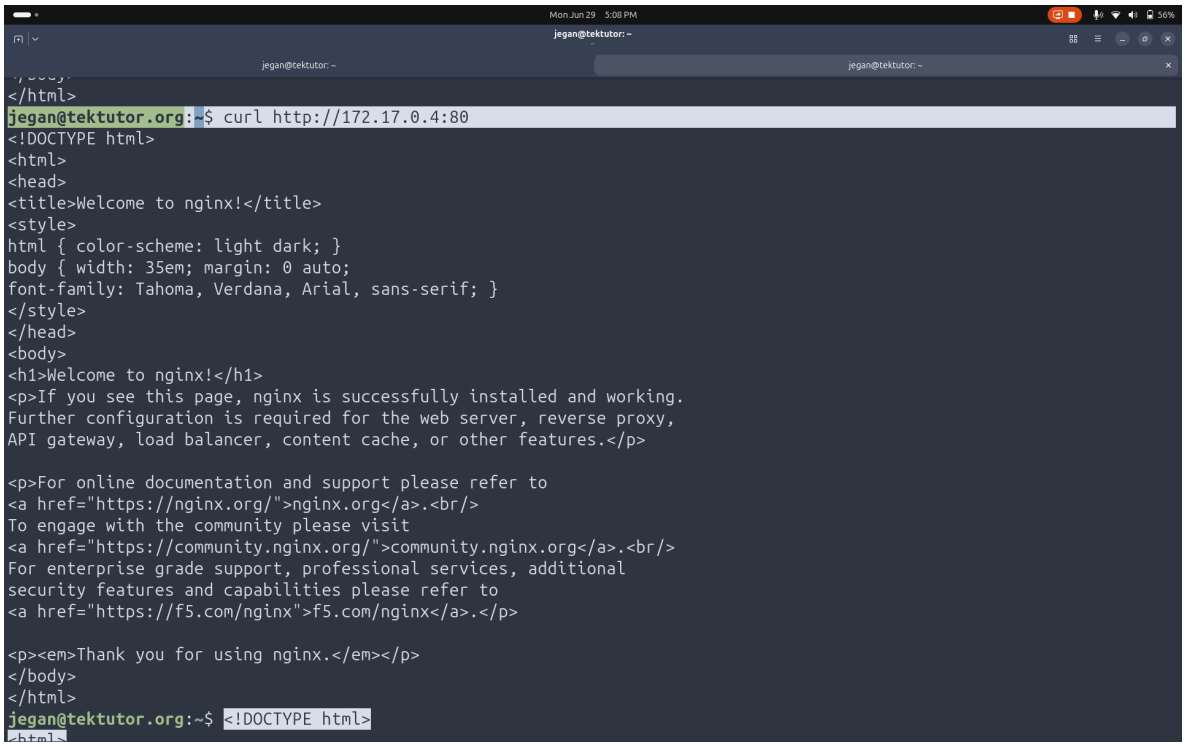
```
docker inspect nginx1-jegan | grep IPA
docker inspect nginx2-jegan | grep IPA
docker inspect nginx3-jegan | grep IPA
```

See if you are able to access the web pages from those web server containers

```
curl http://172.17.0.2:80
curl http://172.17.0.3:80
curl http://172.17.0.4:80
```





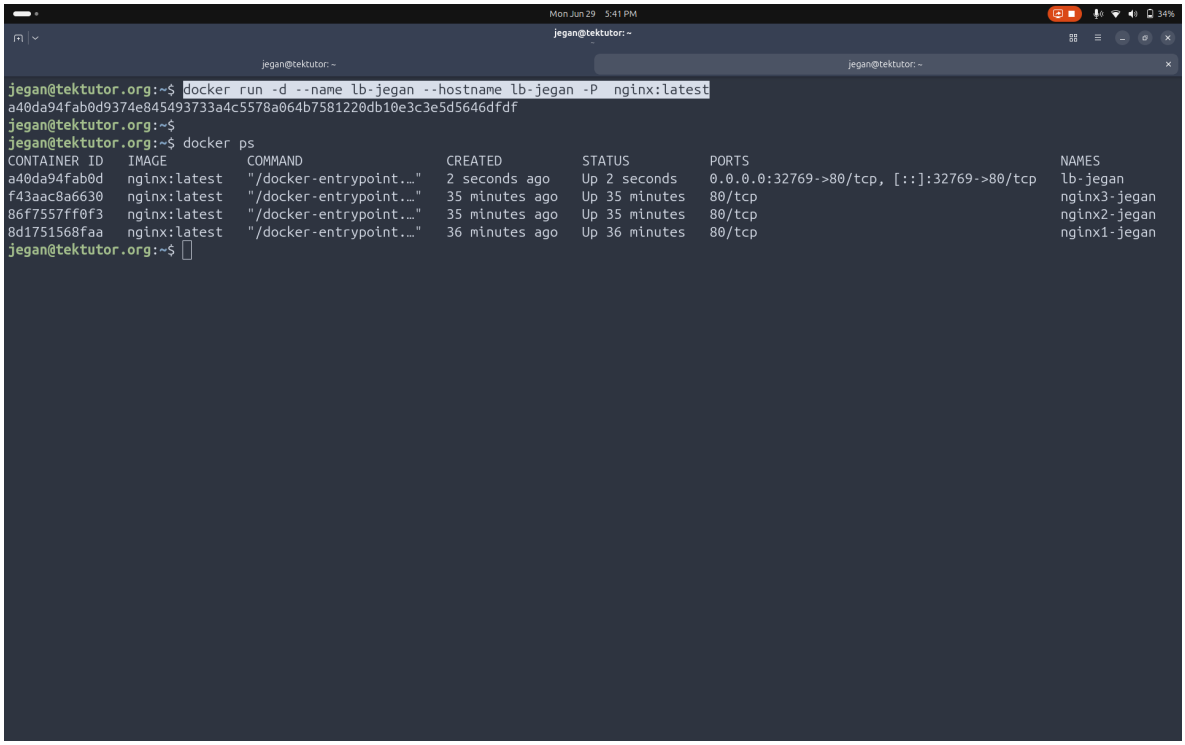


Now, let's create the load balancer container with port-forward(to make it accessible outside the system where the lb container is running)

```
docker run -d --name lb-jegan --hostname lb-jegan -p 8080:80 nginx:latest
```

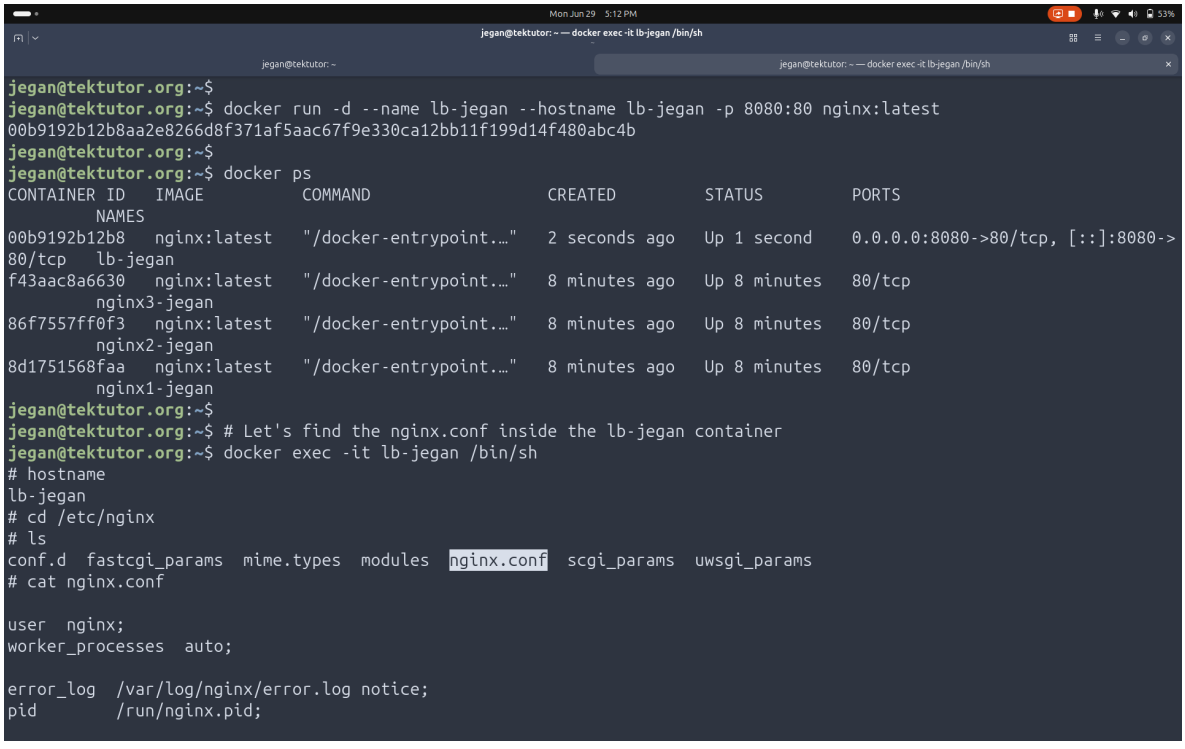
In case you wish docker to identify an available port on your lab machine and forward to that port

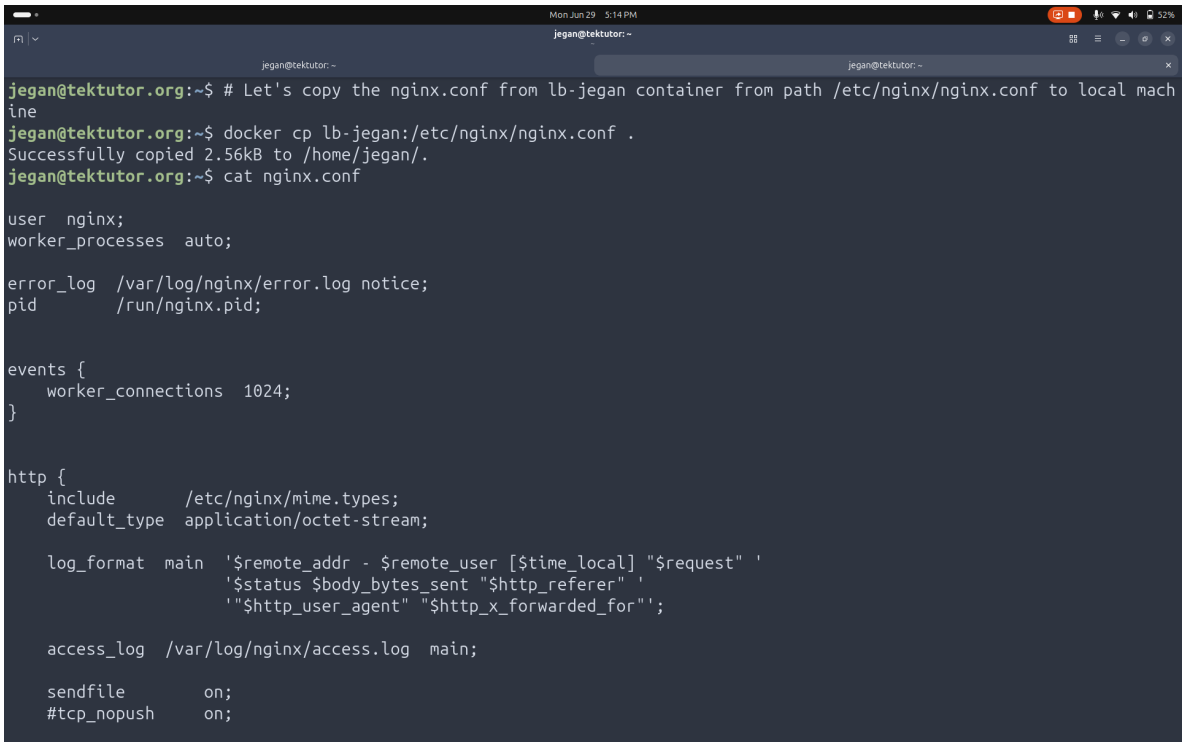
```
docker rm -f lb-jegan
docker run -d --name lb-jegan --hostname lb-jegan -P  nginx:latest
```

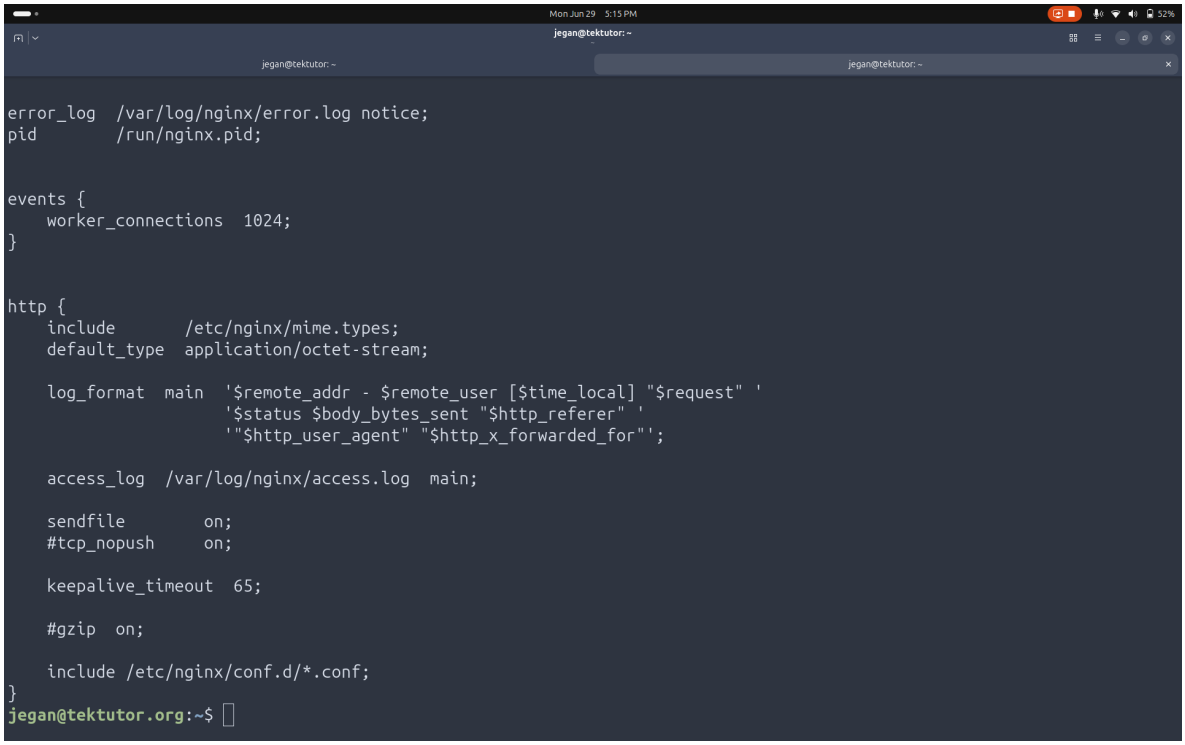


We need to copy the nginx.conf file from the lb-jegan container to configure it work like a load balancer

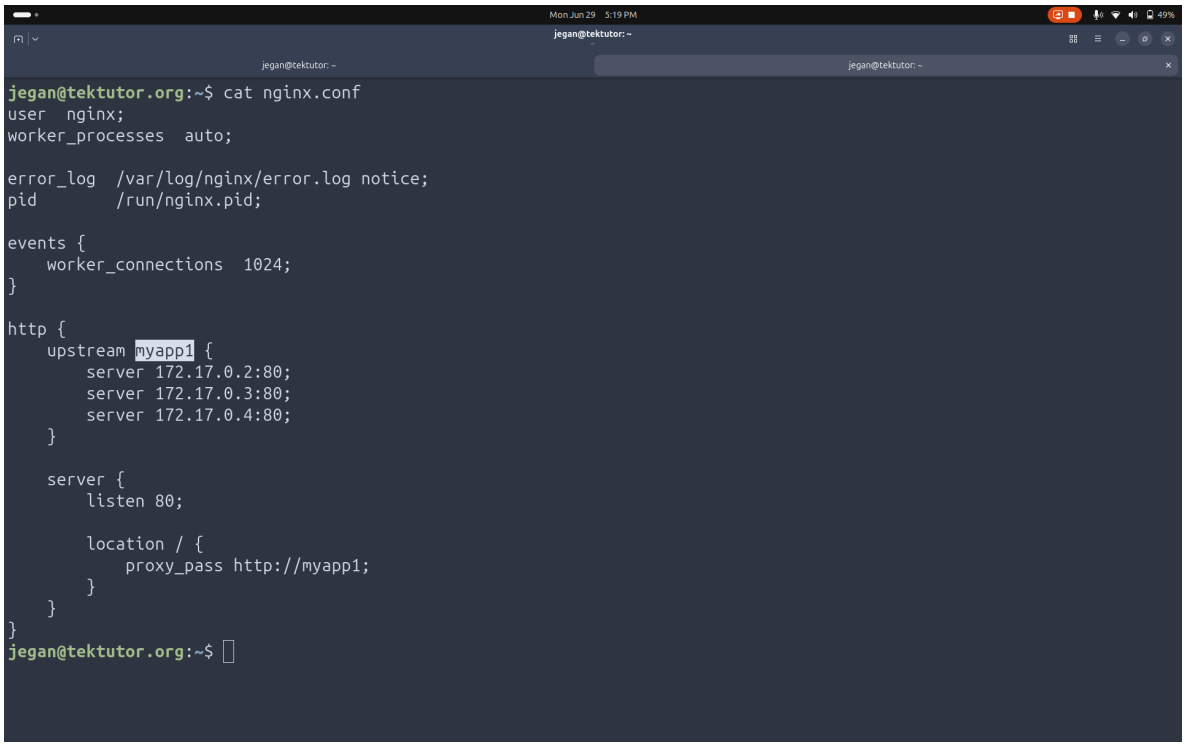
```
docker cp lb-jegan:/etc/nginx/nginx.conf .
cat nginx.conf
```







You need find the IP addresses of your nginx web server containers and update the nginx.conf file as shown below



You need to update the nginx.conf file with your web server IPs

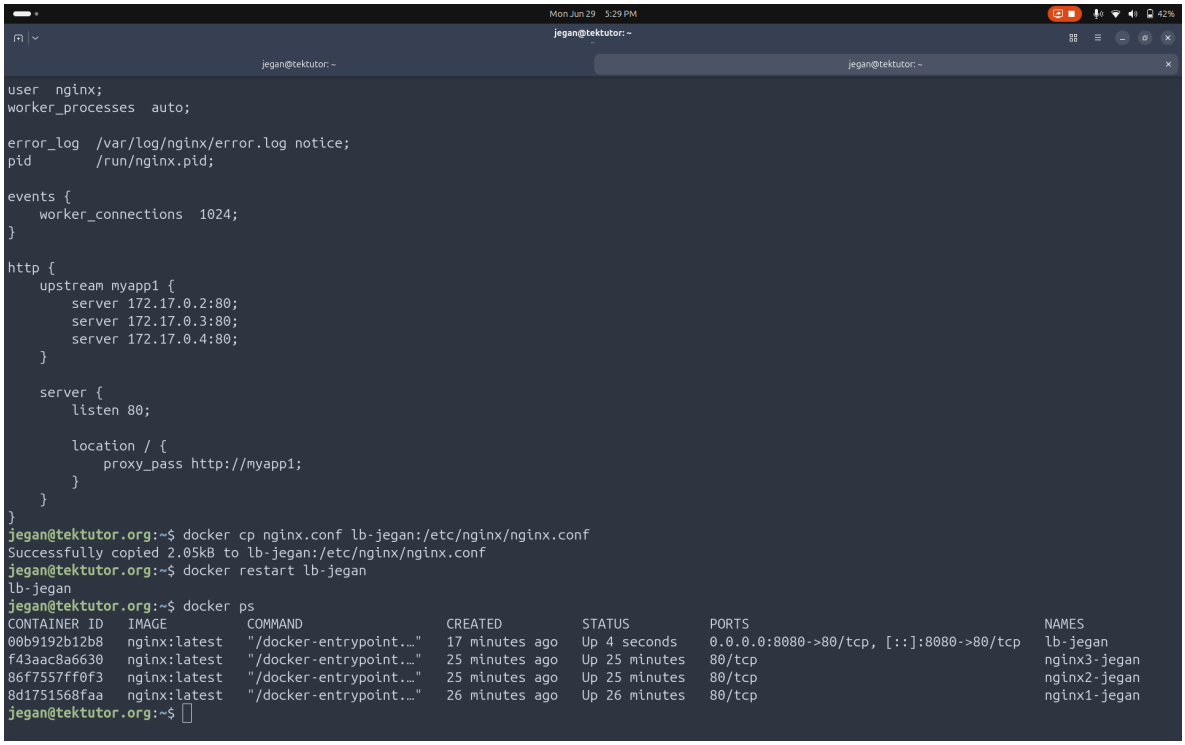
```
user  nginx;
worker_processes  auto;

error_log  /var/log/nginx/error.log notice;
pid        /run/nginx.pid;

events {
    worker_connections  1024;
}

http {
    upstream myapp1 {
        server 172.17.0.2:80;
        server 172.17.0.3:80;
        server 172.17.0.4:80;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://myapp1;
        }
    }
}
```

We need to copy this updated nginx.conf file from our local machine to the lb-jegan container

```
docker cp nginx.conf lb-jegan:/etc/nginx/nginx.conf
```

We need to restart the lb-jegan container to apply the config changes

```
docker restart lb-jegan
```



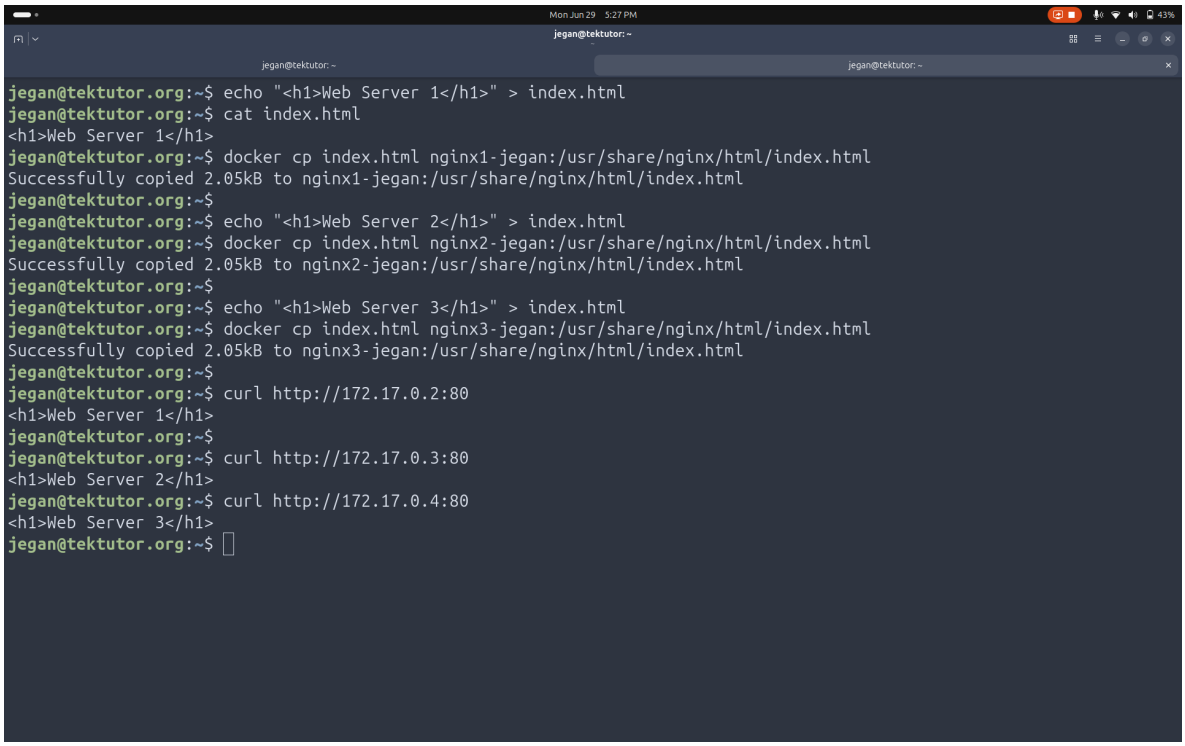
Make sure the lb-jegan container is running after our config changes

```
docker ps
```

Let's customize the web page in nginx1-jegan, nginx2-jegan and nginx3-jegan web server containers

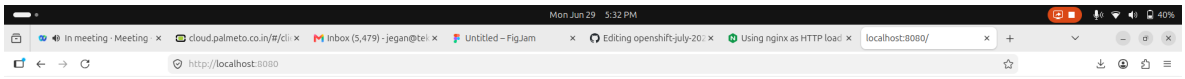
```
echo "<h1>Web Server 1</h1>" > index.html
docker cp index.html nginx1-jegan:/usr/share/nginx/html/index.html

echo "<h1>Web Server 2</h1>" > index.html
docker cp index.html nginx2-jegan:/usr/share/nginx/html/index.html

echo "<h1>Web Server 3</h1>" > index.html
docker cp index.html nginx3-jegan:/usr/share/nginx/html/index.html
```

```
curl http://172.17.0.2:80
curl http://172.17.0.3:80
curl http://172.17.0.4:80
```

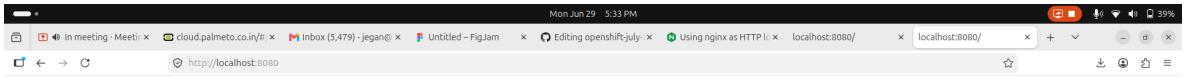


Let's verify if the lb-jegan container is working as configured(as a load balancer) from firefox web browser on your lab machine

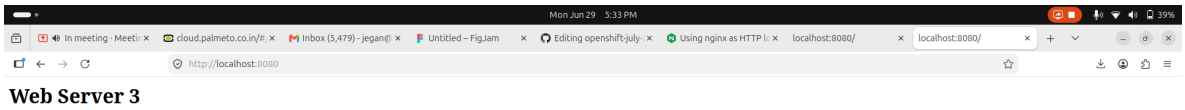
```
http://localhost:8080
http://localhost:8080
http://localhost:8080
```



Web Server 1

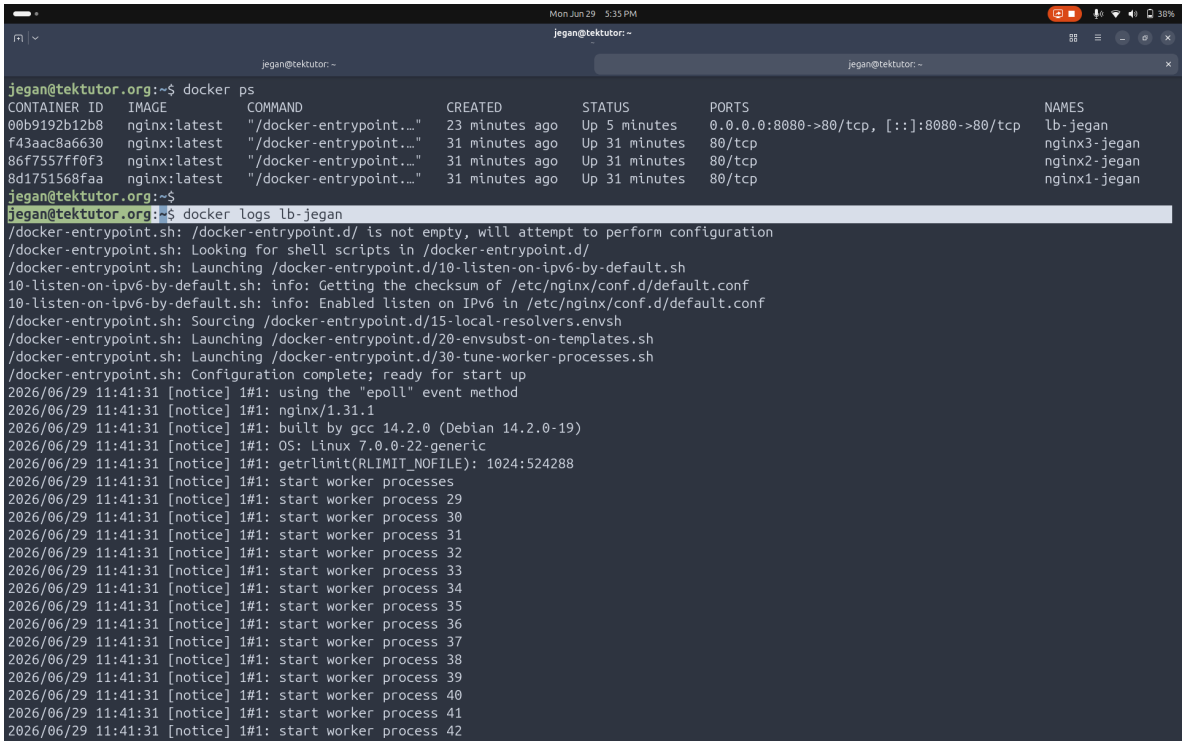


Web Server 2



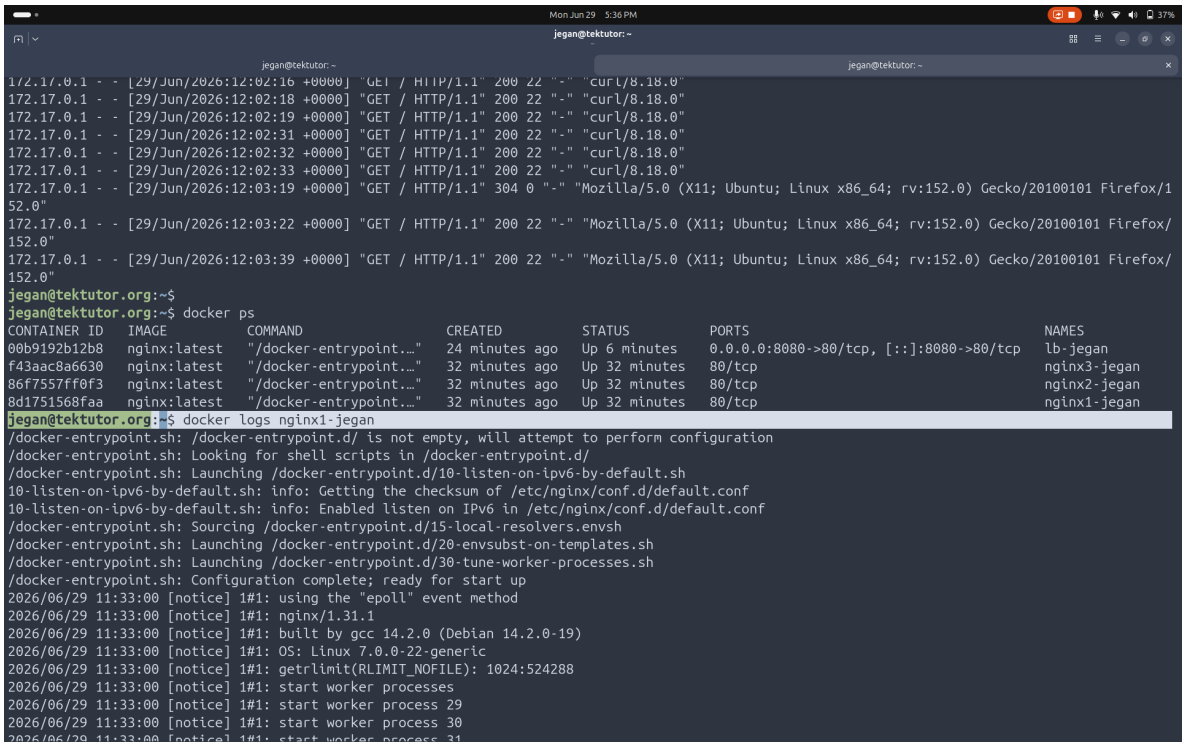
Checking the load balancer logs

```
docker logs lb-jegan
```



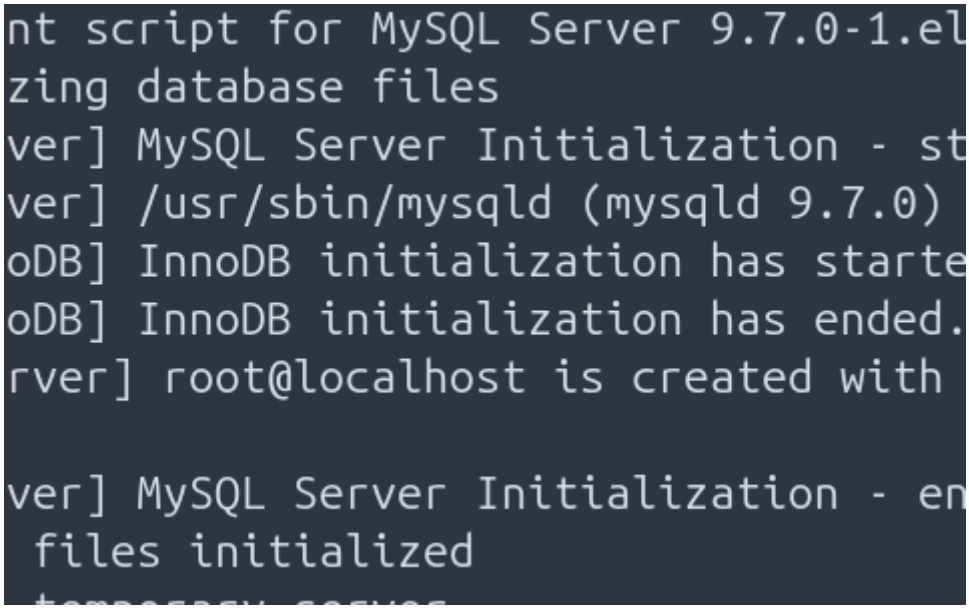
Checking the webserver logs

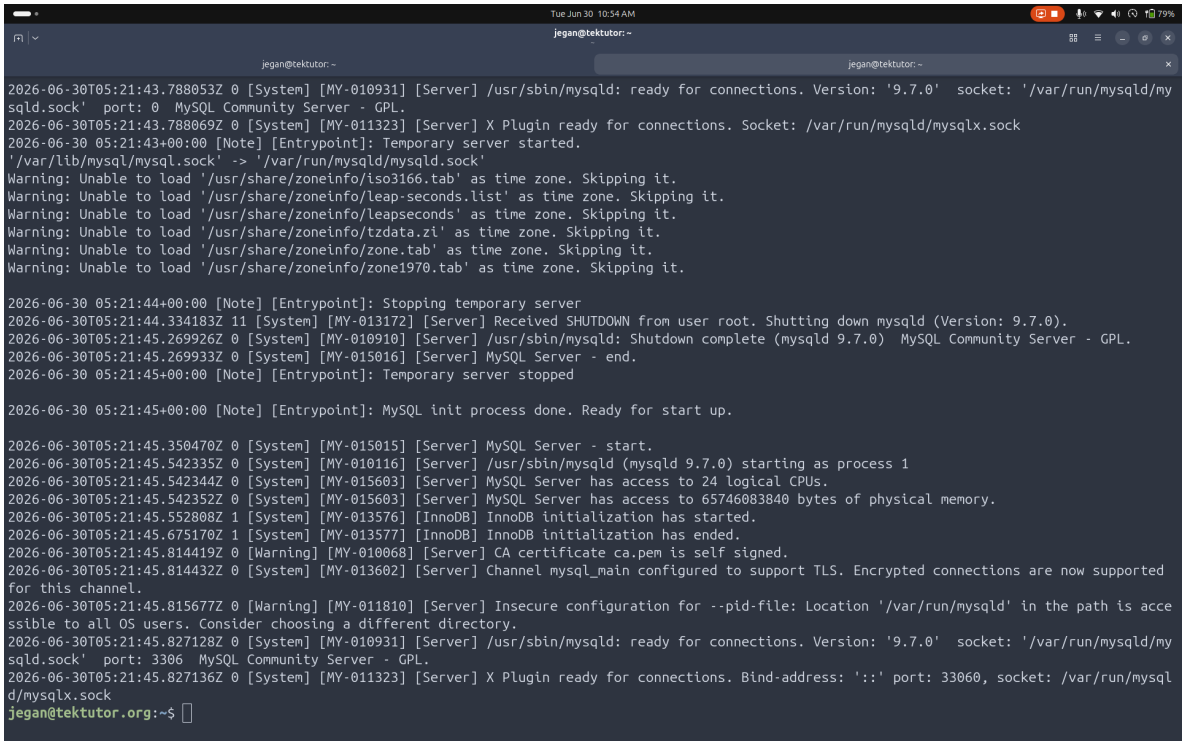
```
docker logs nginx1-jegan
docker logs nginx2-jegan
docker logs nginx3-jegan
```



Lab - Create a mysql db container

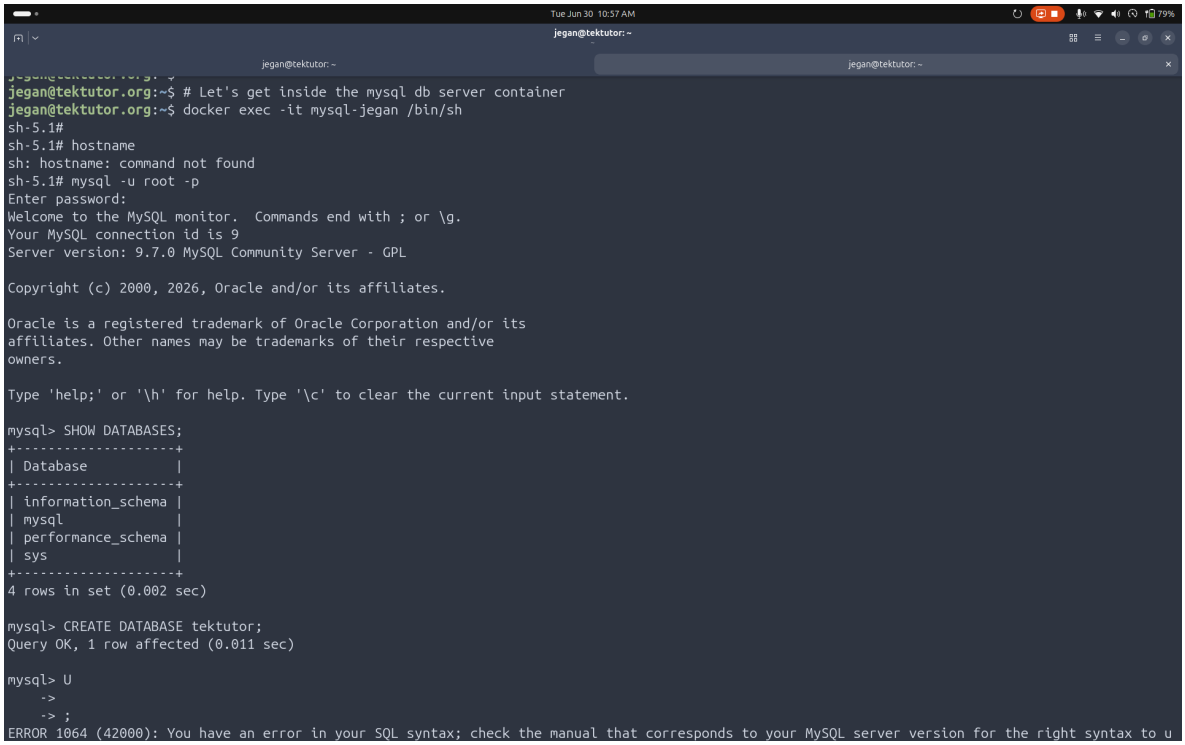
```
docker run -d --name mysql-jegan --hostname mysql-jegan -e
MYSQL_ROOT_PASSWORD=root@123 mysql:latest
docker ps
docker logs mysql-jegan
```

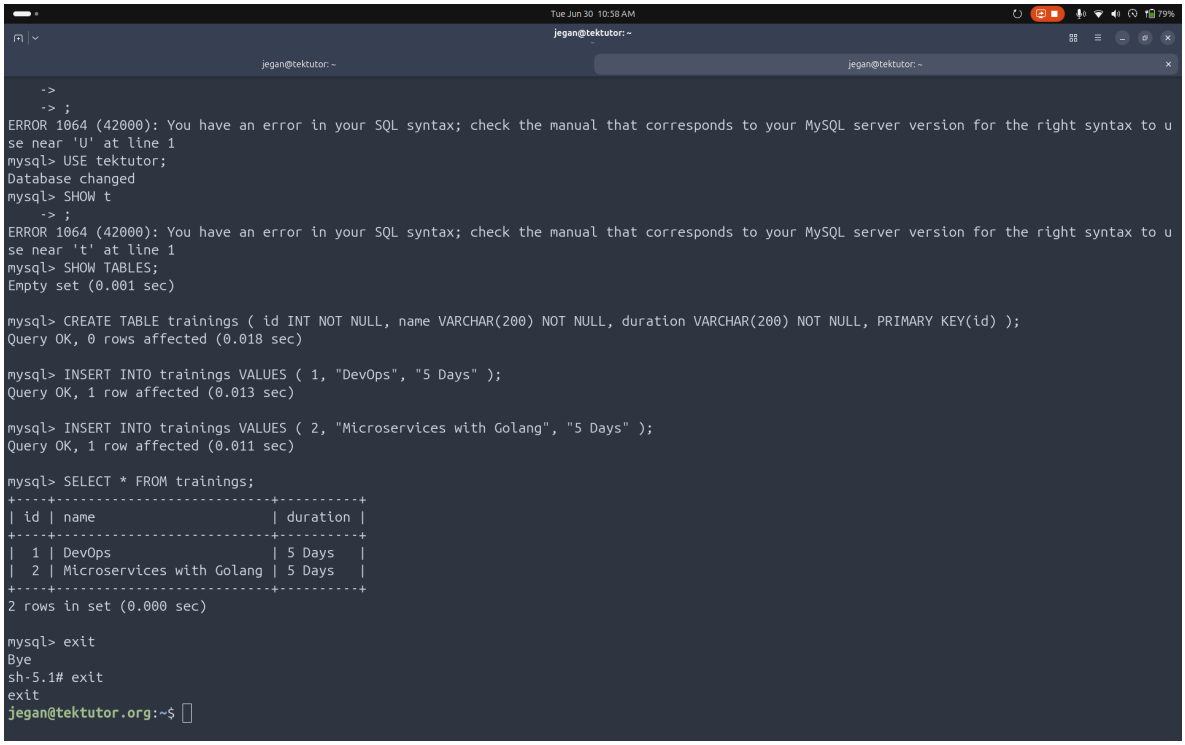




Let's get inside the mysql db server container

```
docker exec -it mysql-jegan /bin/sh
mysql -u root -p
SHOW DATABASES;
CREATE DATABASE tektutor;
USE tektutor;
CREATE TABLE trainings ( id INT NOT NULL, name VARCHAR(200) NOT NULL, duration
VARCHAR(200) NOT NULL, PRIMARY KEY(id) );
INSERT INTO trainings VALUES ( 1, "DevOps", "5 Days" );
INSERT INTO trainings VALUES ( 2, "Microservices with Golang", "5 Days" );
SELECT * FROM trainings;
exit
exit
docker ps
```

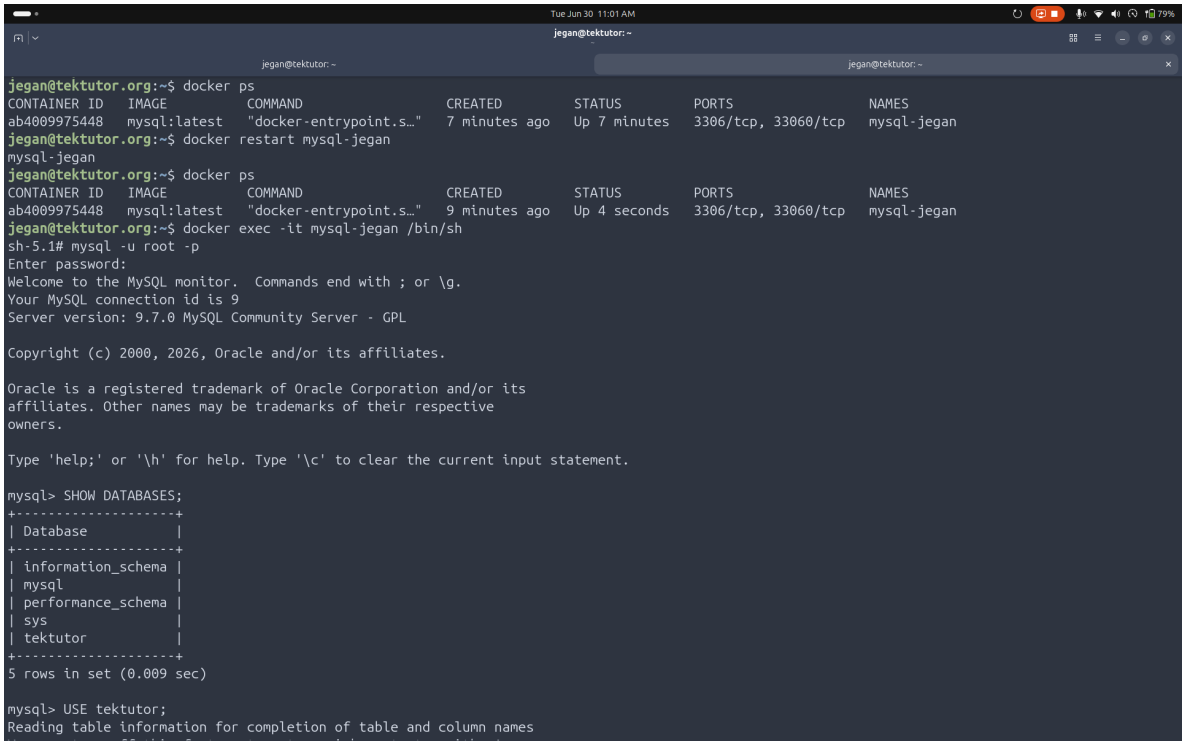


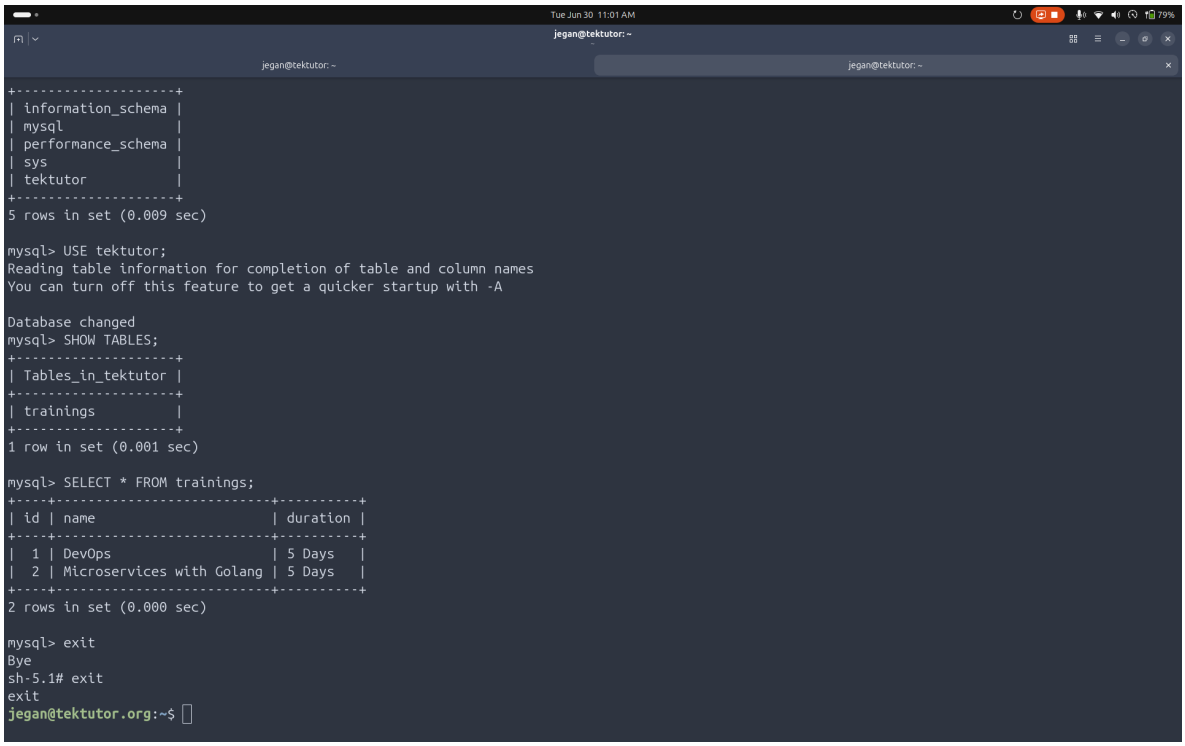


Let's try to restart the mysql container

```
docker restart mysql-jegan
docker ps
docker exec -it mysql-jegan /bin/sh
mysql -u root
SHOW DATABASES;
```

```
USE tektutor;
SHOW TABLES;
SELECT * FROM trainings;
exit
exit
```





Let's delete the mysql container. At this point, we not only lost the container, we also lost the data stored inside the container.

```
docker rm -f mysql-jegan
```

Hence, we must always store application data in an external storage.

```
mkdir -p /tmp/jegan/mysql
docker run -d --name mysql-jegan --hostname mysql-jegan -e
MYSQL_ROOT_PASSWORD=root@123 -v /tmp/jegan/mysql:/var/lib/mysql mysql:latest
docker exec -it mysql-jegan /bin/sh
mysql -u root -p
SHOW DATABASES;
CREATE DATABASE tektutor;
USE tektutor;
CREATE TABLE trainings ( id INT NOT NULL, name VARCHAR(200) NOT NULL, duration
VARCHAR(200) NOT NULL, PRIMARY KEY(id) );
INSERT INTO trainings VALUES ( 1, "DevOps", "5 Days" );
INSERT INTO trainings VALUES ( 2, "Microservices with Golang", "5 Days" );
SELECT * FROM trainings;
exit
exit
docker ps
```

Let's delete the mysql-jegan container and recreate a new one

```
docker rm mysql-jegan
docker run -d --name mysql-jegan --hostname mysql-jegan -e
MYSQL_ROOT_PASSWORD=root@123 -v /tmp/jegan/mysql:/var/lib/mysql mysql:latest
docker exec -it mysql-jegan /bin/sh
mysql -u root -p
SHOW DATABASES;
USE tektutor;
SHOW TABLES;
SELECT * FROM trainings;
exit
exit
```

Though we delete the container and recreated a new one, we didn't lose the data because we stored the database in an external disk.
This is how, containers are used in real-world applications.

Day 2

Info - Container Orchestration Platform

```
- in order to deploy your application into Container Orchestration Platform, we must first containerize them
- In real world applications, no one manages containerized application workloads manually
- In real world, Container Orchestration Platforms manage our containerized application workloads
- They provide
  - in-built monitoring, health-check, readiness check, liveliness check for your application
  - self-healing to your applications
  - a way to scale up/down your application instances based on user traffic
  - rolling update
    - upgrading your application from one version to other without any downtime
    - rolling back to earlier stable versions are also possible
  - ways to collect performance metrics
- any container orchestration platform works as a cluster of many nodes
- nodes are nothing but Physical Servers or Virtual Machines in private/public cloud
- examples
  - Docker SWARM
  - Kubernetes
  - Rancher
  - Red Hat Openshift
  - AWS eks
  - AWS ROSA
  - Azure aks
  - Azure ARO
```

Info - Docker SWARM

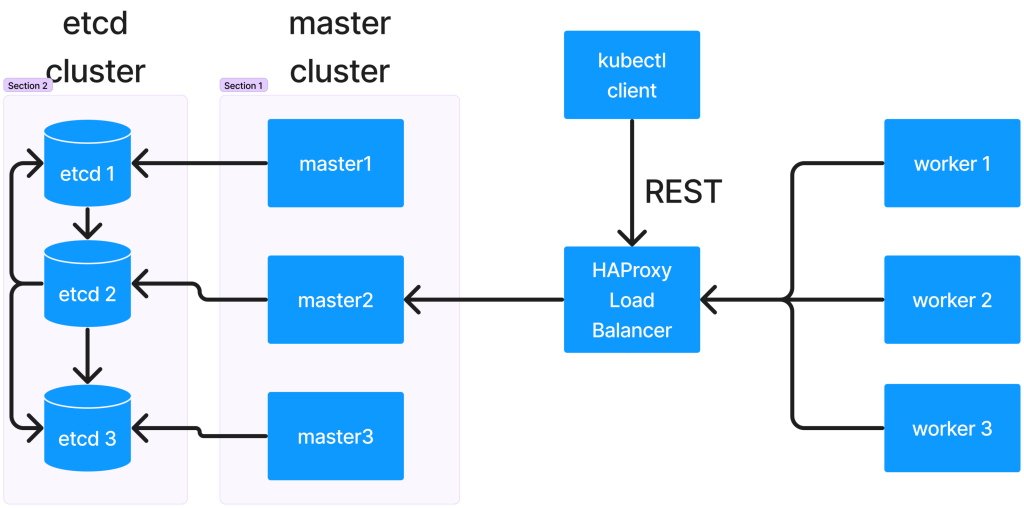
```
- is developed by Docker Inc as an open-source project
- it supported only Docker Containerized applications
- it is a very light-weight setup that works in laptop/dekstop with
```

```
normal configurations
- it is ideal setup for learning purpose
- it is ideal for Dev/QA environment
- it is not production-grade
```

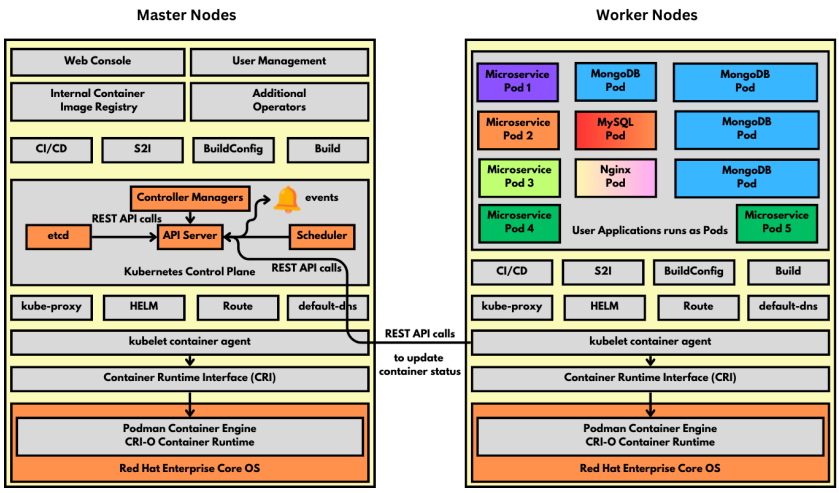
Info - Kubernetes

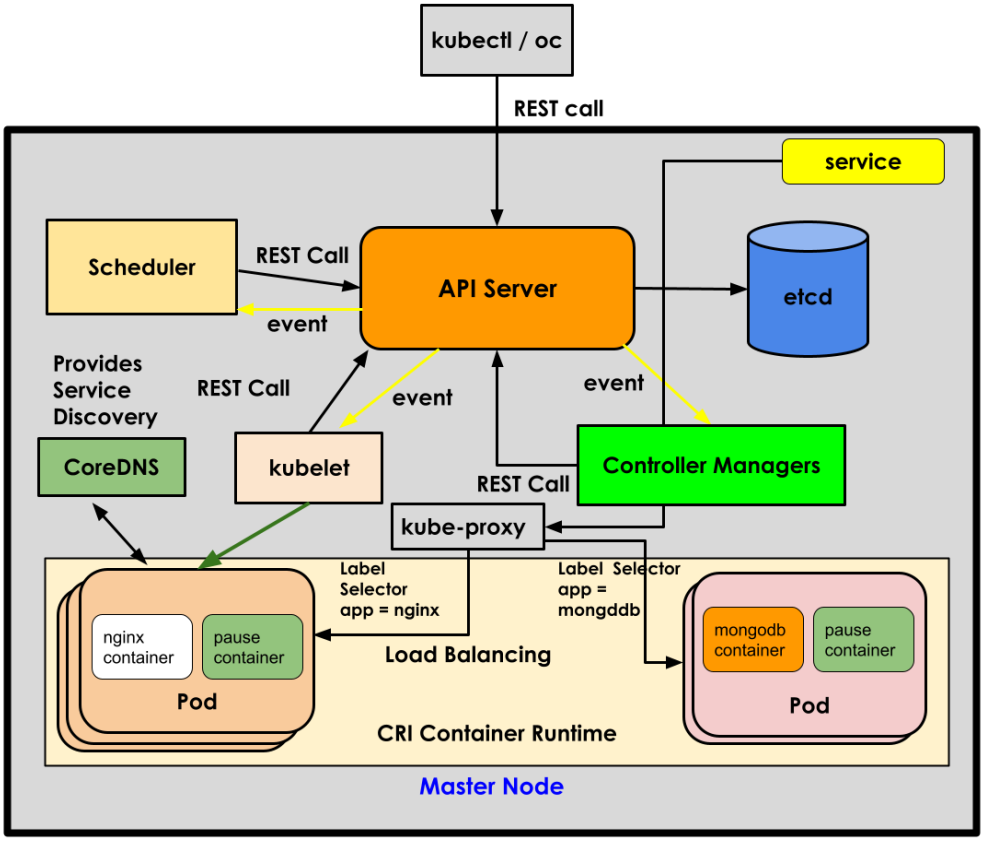
```
- is a Container Orchestration platform developed in Golang by Google
- intially it was called borg, Google has been using this container
orchestration platforms internally within google
  for several years
- it is production-grade because google used it on complex projects many
years
- it supports many different container runtimes/engines
- Kubernetes supports something called CRI(Container Runtime Interface)
- any container runtime/engine that must be supported by Kubernetes they
have to implement the CRI
- Kubernetes can/will interact with any type of Container Runtime/Engine
via CRI
- Kubernetes supports 2 types of nodes
  - Master Node
    - this is where Control Plane components will be running
    - Control Plane Components
      1. API Server
      2. etcd key/value database
      3. scheduler
      4. Controller Managers( collection of many controllers )
  - Worker Node
    - this is where user applications will be running
- Kubernetes is primarily a CLI application
- Kubernetes doesn't support
  - a production-grade webconsole
  - User Management
  - Internal Container Registry ( out of box )
  - Prometheus/Grafana performance metrics collection ( out of box )
- Kubernetes supports basic building blocks to extend the Kubernetes
features by adding
  - your own custom resources
  - your own custom controllers
- Kubernetes allows one to deploy their application only from a
Container Image
- In Kubernetes Master/Worker nodes, we can choose to install any Linux
OS distribution (ubuntu, rocky, fedora, RHEL, etc., )
```

Info - Kubernetes High-Level Architecture



Info - Red Hat Openshift High-Level Architecture





Info - Red Hat Openshift

```
- is a Red Hat's distribution of Kubernetes
- Openshift is developed on top of opensource Kubernetes
- Hence, Openshift is a superset of Kubernetes
- Openshift is Kubernetes with batteries included
- Openshift comes with these additional features on top of what is
already supported by Kubernetes
  1. User Management
  2. Production Web-console (GUI)
  3. Many additional features
     - Route
     - S2I ( Source to Image )
       - one can deploy their application from plain source code coming
from GitHub or any other version control
       - supports many different strategies
         - docker
         - source
         - pipeline
  4. Internal Container Registry
- Red Hat acqurired a company named CoreOS
  - CoreOS organization had several interesting products
    - rkt(pronounced as Rocket) - container runtime
```

```
    - CoreOS - Linux based minimal & secure operating system good enough
for Container Orchestration Platform
    - Network addons - Flannel
- Red Hat Openshift starting from v4.x,
  - supports only Red Hat Enterprise CoreOS operating (RHCOS) in Master
Nodes
  - supports either Red Hat Enterprise Linux(RHEL) or RHCOS in Worker
Node
  - Red Hat Openshift generally recommends to install RHCOS in all nodes
  - supports only CRI-O Container Runtime
  - RHCOS Operating System● Edit
● Create
● Describe
Finding IP Address of a Pod
    - is an Container Orchestration Platform optimized minimal and
secured Operating System
      based on RHEL
    - this OS comes with CRI-O & Podman pre-installed
    - it is an immutable OS many restrictions
    - some of the folders like /var, /bin, /usr/bin are all treated as
read-only folder
    - only through Machine Config Operators we can update certain
reserved folders, meaning
      application won't be able to modify those restricted folders
    - Ports 0-1024 are reserved for Openshift's internal use
```

Info - Pod

```
- Pod is a logical grouping of related containers
- Pod is record stored in etcd database
  - is a configuration object stored in etcd database
  - what really runs on nodes is containers
- Unlike Docker container where each container get a dedicated IP
address, in case of Pod, all containers
  that are part of a Pod they all share the same Private IP address
- applications runs with a Pod container
- Every Pod get its own port range i.e 0 to 65535
- if there are two container c1 and c2 within Pod P1, if c1 is using
port 8080 then c2 won't be able to use port 8080
- the containers within the same Pod, they can communicate with each
with regular IPC mechanisms
```

Info - Types of applications that can be deployed into Openshift

```
- Stateless applications
  - k8s/Openshift supports a resource called Deployment
- Stateful applications
  - k8s/Openshift supports a resource called StatefulSet
- Application that performs a one-time activity and stops in some time
  - Job
- Application that must be invoked periodically on a particular day and
time
  - CronJob (this internally used Job)
- Applications that run one instance per Node
  - DaemonSet
```

Info - API Server

```
- is the heart of kubernetes/openshift
- anything and everything in Kubernetes/Openshift is co-ordinated by API
server, hence this is
  one of the most critical components
- API server only performs database activities like
  - it creates a new record in etcd datastore
  - it updates an existing record in the etcd datastore
  - it retrieves an existing record from the etcd datastore
  - it deletes an existing record from the etcd datastore
  - everytime some new record are added, existing records updated,
existing records deleted it will
    send events to registered controllers
```

Info - etcd

```
- it is distributed database which stores data in key/value format
- it is an idendepent opensource project used by Kubernetes & openshift
- this can be used outside the scope of K8s/Openshift as well
```

Info - Scheduler

```
- is a Pod
- is one of the Control Plane components that runs in the master node
```

```
- it is responsible to identify a healthy node where a new Pod can be
deployed
- scheduler by itself can't deploy a pod into any node, it is allowed
only to share it scheduling
  recommendations to API Server via REST call
```

Info - Controller Managers

```
- the real doers are controllers
- is a collection of many in-built controllers
- for each type of resource supported in Openshift/Kubernetes there is
one Controller
- For example
  - Deployment Controller manages Deployment resource ( stateless
application )
  - ReplicaSet Controller manages ReplicaSet resource
  - Job Controller manages Job resource
  - StatefulSet Controller manages Statefult resource ( stateful
application )
  - DaemonSet Controller manages DaemonSet resource
- Controller is a application that runs within a Pod container
- they have unrestricted access within the Kubernetes/Openshift cluster
- they can monitor specific type of resources created/updated/deleted
under any project within the cluster
- they get notified by API Server via events
- Controllers if it needs to communicate something to API Server, it
make a REST API call
```

Lab - Login to openshift from command-line

```
cat ~/openshift.txt
oc login --username=kubeadmin --password=VVnz6-ezZ9W-RVbw4-NPK3R https://
api.ocp4.palmeto.org:6443 --insecure-skip-tls-verify
```

Lab - Listing Nodes in Red Hat Openshift

```
oc version
kubectl version

oc get nodes
kubectl get nodes
```

```
oc get nodes -o wide
kubectl get nodes -o wide
```

Lab - Finding additional meta-data about a node

```
oc describe node/master01.ocp4.palmeto.org
oc describe node/worker01.ocp4.palmeto.org
```

Lab - List and find all control plane pods

```
oc get pods --all-namespaces -o wide | grep api
oc get pods --all-namespaces -o wide | grep scheduler
```

Lab - Managing projects in Openshift

```
- each team in an organization setup, will create atleast 1 project to
isolate their application
  deployment from the other teams in the same organization
- generally, a single Openshift cluster will shared by all the team in
an orgnanization
- openshift, supports features to restrict access to only those team
members working a project
```

Create a new-project

```
oc new-project jegan
```

List all projects

```
oc get projects
oc get project
```

Switching between projects

```
oc project default
oc project jegan
```

Finding the currently active project

```
oc project
```

Deleting a project(please delete only your project)

```
oc delete project jegan
```

Lab - Deploying your first application into Openshift using imperative approach

```
oc new-project jegan-project
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3

# Listing deployments
oc get deployments
oc get deployment
oc get deploy

# Listing replicasets
oc get replicasets
oc get replicaset
oc get rs

# Listing pods
oc get pods
oc get pod
oc get po

# Listing mutliple resource with a single command
oc get deploy,rs,po
oc get all

# Finding more details about your deployment
oc describe deploy/nginx

# Finding more details about replicaset
oc describe rs/nginx-dbfb56c96

# Finding more details about the pod
oc describe pod/nginx-dbfb56c96-l296j

# Editing a deployment and update its replicas from 3 to 5, save and exit
oc edit deploy/nginx
```

```
oc get pods

oc get deploy/nginx -o yaml
oc get deploy/nginx -o json

oc get rs/nginx-dbfb56c96 -o yaml
oc get pod/nginx-dbfb56c96-jfx2m -o yaml
```

Lab - Scale up/down your deployments

```
oc project jegan-project
# Scale up
oc scale deploy/nginx --replicas=5

# Scale down
oc scale deploy/nginx --replicas=3
```

Lab - Port forward for quick testing/debugging (Not for production environment)

```
oc project jegan-project
oc get pods
# Terminal 1
oc port-forward pod/hello-7cf46f6476-fwzrn 9090:8080

# Terminal 2
curl http://localhost:9090

# To stop the port forward after your testing
# Press Ctrl+C in Terminal 1
# From Terminal 2, curl will no more work
```

Lab - Getting inside a node shell (only a cluster/kube admin do can do this at work place)

```
oc get nodes
oc debug node/master01.ocp4.palmeto.org
chroot /host
podman --version
crictl --version
crictl images
```

```
crictl ps
exit
exit
```

Lab - Getting inside a pod shell (this doesn't require admin access)

```
oc project jegan-project
oc get pods
oc rsh pod/nginx-dbfb56c96-wtbp7
exit

oc rsh deploy/nginx
exit
```

Lab - Creating an internal clusterip service from your nginx deployment

```
oc project jegan-project
oc get deploy -l app=nginx
oc expose deploy/nginx --type=ClusterIP --port=8080

oc get services
oc get service
oc get svc

oc get endpoints

oc describe svc/nginx

oc scale deploy/nginx --replicas=8

oc describe svc/nginx
oc get endpoints

oc scale deploy/nginx --replicas=3
```

```
oc describe svc/nginx
oc get endpoints
```

Day 3

Info - You can find Red Hat Base images here

```
https://catalog.redhat.com/en/software/containers/explore
```

Info - You can find other Red Hat Openshift compatible images here

```
https://catalog.redhat.com/en/search?searchType=Containers
```

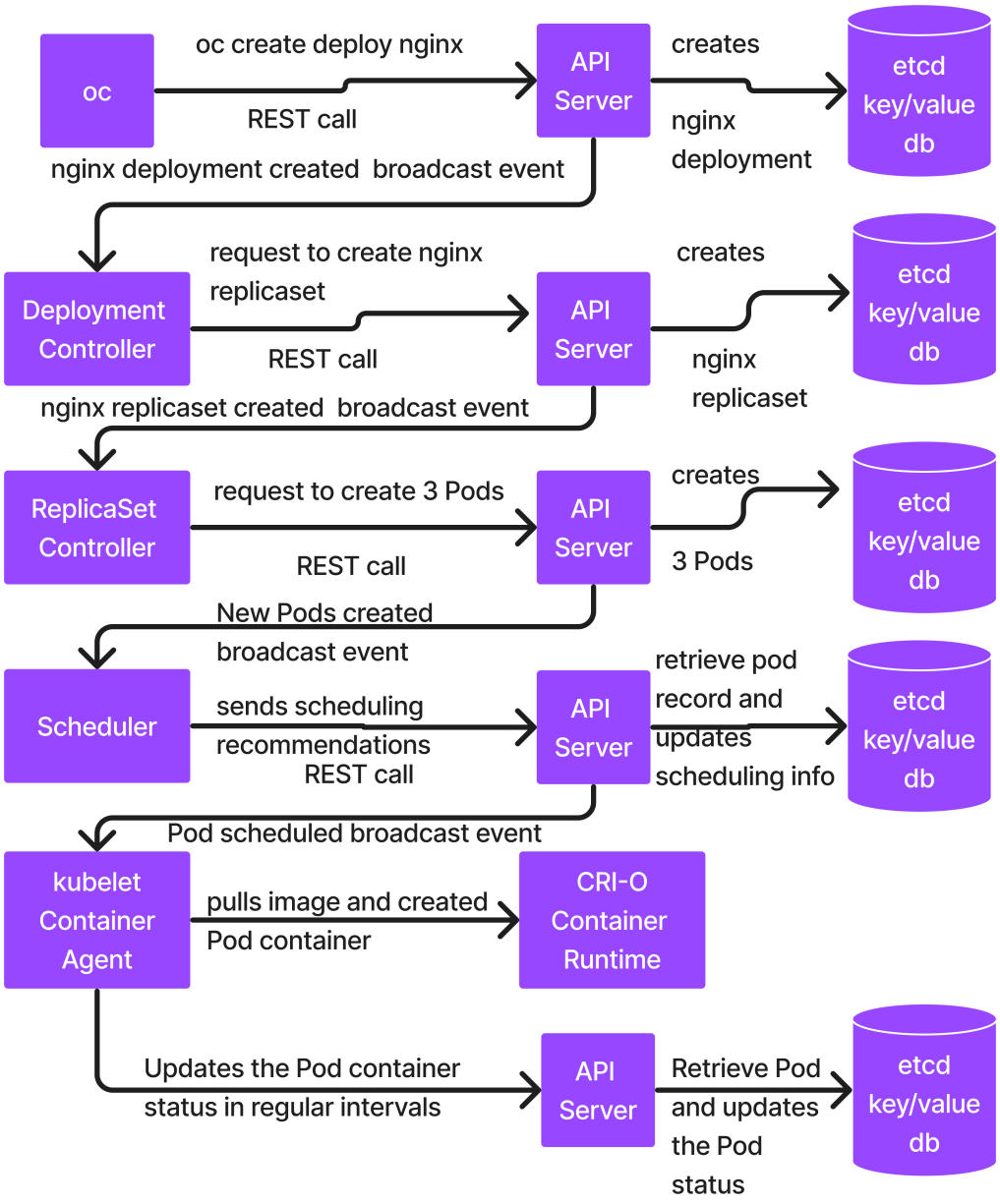
Info - What happens internally in Openshift when we deploy an application

```
oc project jegan-project
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3
```

Note

```
- oc client tool makes a REST call to API Server requesting the API
Server to create a deployment
- Once API Server receives the request from oc client, it creates a
Deployment database entry in etcd database
- API Server then sends broadcasting event saying new Deployment created
along with deployment details
- Deployment Controller receives this event, it then makes a REST call
to API Server requesting it to create a ReplicaSet for the nginx
deployment
- API Server creates a ReplicaSet db entry(new record) in the etcd
database
- API Server sends a broadcast event saying new ReplicaSet created
- ReplicaSet Controller receives the event, it then makes a REST call to
API Server requesting it to create 3 Pods
- API Server create 3 Pod records in the etcd database
```

- API Server sends broadcast event for each new Pod created in the etcd database
- Scheduler receives the event, it then identifies a healthy node where the new Pod can be deployed
- Scheduler makes a REST call to API Server to send it scheduling recommendataion. This will be done for each Pod.
- API Server receives the scheduling recommendations from Scheduler, it then retrieves the Pod record from etcd and updates it status as Scheduled to so and so node
- API Server sends a broadcasting event saying Pod1 scheduled to Worker01 node, this happens for each Pod.
- Kubelet Container Agent that runs on Worker01 node receives the event, it then pull the container image, creates and starts the container on Worker01
- Kubelet monitors the status of the Container created for Pod1, and it periodically updates the status back to API Server in a heart-beat fashion
- API Server receives these updates, retrieves the Pod database entry from etcd and updates the Pod status



Lab - Deploying a stateless application in declarative style

```
# Delete your existing project
oc delete project jegan

# Create a new project
oc new-project jegan

# Generate the declarative manifest file to deploy nginx
```

```
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3 --dry-run=client -o yaml

# Redirect the output shown to a yaml file
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3 --dry-run=client -o yaml > nginx-
deploy.yml

# Deploy the nginx in declarative fashion
oc create -f nginx-deploy.yml --save-config

# In the template section of the nginx-deploy.yml add additional labels and apply
oc apply -f nginx-deploy.yml

# List the deploy,replicaset and pods
oc get deploy,rs,po

## Delete the deployment declaratively
oc delete -f nginx-deploy.yml
oc get deploy,rs,po
```

Lab - Creating a ClusterIP Internal Service in declarative style

Create a pod.yml with code below

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: test
  name: test-pod
spec:
  containers:
  - image: image-registry.openshift-image-registry.svc:5000/openshift/
hello:latest
    imagePullPolicy: IfNotPresent
    name: test
```

Create the test pod

```
oc project jegan-project
oc apply -f pod.yml
oc get pods
```

Let's create the nginx deployment in declarative style

```
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3 --dry-run=client -o yaml
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3 --dry-run=client -o yaml > nginx-
deploy.yml
oc apply -f nginx-deploy.yml
```

Let's create the ClusterIP Internal service declaratively

```
oc expose deploy/nginx --type=ClusterIP --port=8080 --dry-run=client -o yaml
oc expose deploy/nginx --type=ClusterIP --port=8080 --dry-run=client -o yaml >
nginx-clusterip-svc.yml
oc apply -f nginx-clusterip-svc.yml
oc get svc
oc describe svc/nginx
```

Let's test the clusterip internal service using the test pod we created

```
oc rsh pod/test-pod
curl http://nginx:8080 # Service discovery - accessing service by its name
curl http://nginx.jegan-project.svc.cluster.local:8080 # Recommended compared to
previous command
curl http://172.30.240.164:8080
```

Lab - Creating an external NodePort service in declarative style

```
oc project jegan-project

# Delete existing clusterip service before creating nodeport service
oc delete -f nginx-clusterip-svc.yml

oc expose deploy/nginx --type=NodePort --port=8080 --dry-run=client -o yaml
oc expose deploy/nginx --type=NodePort --port=8080 --dry-run=client -o yaml >
nginx-nodeport-svc.yml
oc apply -f nginx-nodeport-svc.yml
oc get svc
oc describe svc/nginx # Get the NodePort from this command and substitute below

oc get nodes -o wide
```

Let's test the nodeport external service (don't need to run this inside pod shell)

```
curl http://192.168.100.11:31728 # Master1 IP
curl http://192.168.100.12:31728 # Master2 IP
curl http://192.168.100.13:31728 # Master3 IP
curl http://192.168.100.21:31728 # Worker1 IP
curl http://192.168.100.22:31728 # Worker2 IP
curl http://192.168.100.23:31728 # Worker3 IP

curl http://master01.ocp4.palmeto.org:31728
curl http://master02.ocp4.palmeto.org:31728
curl http://master03.ocp4.palmeto.org:31728
curl http://worker01.ocp4.palmeto.org:31728
curl http://worker02.ocp4.palmeto.org:31728
curl http://worker03.ocp4.palmeto.org:31728
```

Lab - LoadBalancer service in declarative style

```
oc project jegan-project

# Delete exisitng node port service
oc delete -f nginx-nodeport-svc.yml

# Create the LoadBalancer service in declarative style
oc expose deploy/nginx --type=LoadBalancer --port=8080 --dry-run=client -o yaml >
nginx-lb-svc.yml
oc apply -f nginx-lb-svc.yml

oc get svc
oc describe svc/nginx # You need to find your loadbalancer service ip and use it
below in the curl

# Testing the loadbalancer service
curl http://192.168.100.50:8080
```

Lab - Rolling update (upgrading nginx from version 1.26 to 1.27 and later to 1.28)

Let's delete your exisitng project

```
oc delete project jegan-project
```

Let's create new project

```
oc new-project jegan-project
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3 --dry-run=client -o yaml > nginx-
deploy.yml
oc apply -f nginx-deploy.yml

oc get pods -o yaml | grep image
```

Lab - Create an external route for nginx service

```
# Delete existing project
oc delete project jegan-project

# Deploy nginx
oc new-project jegan-project
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.26 --replicas=3 --dry-run=client -o yaml > nginx-
deploy.yml
oc apply -f nginx-deploy.yml

# Create clusterip internal service for nginx deployment
oc expose deploy/nginx --port=8080 --dry-run=client -o yaml > nginx-internal-
svc.yml
oc apply -f nginx-internal-svc.yml

# Create an external route for internal service
oc expose svc/nginx
oc get route
curl http://nginx-jegan-project.apps.ocp4.palmeto.org
```

Info - ConfigMap

```
- is a map that stores data in key/value format
- instead of hard-coding, we store certain details like NFS Server IP,
NFS Share Path, etc in the config map
- generally used for storing this like
  - JDK_HOME, MAVEN_HOME, LOG_PATH, etc.,
- deployment can retrieve the details from the ConfigMap and use it
- this is good for non-sensitive data
```

Info - Secrets

```
- is also map that stores data in key/value format
- it is used to store any sensitive confidential data like login
credentials, certificates, etc.,
```

Info - Persistent Volume (PV)

```
- is an external disk/storage
- this type of storage can come from NFS,NAS,AWS S3, etc.,
- this is always created on the cluster-wide and accessible to all
projects in the cluster
- this can be provisioned manually by Administrators or can be
provisioned on demand dynamically if there is a StorageClass
- Generally in case of NFS
  - we need to mention NFS Server IP/Hostname
  - Disk size required in MiB/GiB/TiB
  - StorageClass ( Optional )
  - Access
    - ReadWriteOnce
    - ReadWriteMany
```

Info - Persistent Volume Claim (PVC)

```
- a application that runs in a Pod will have to request for external
storage in Openshift/Kubernetes by defining
  its requirement in a Persistent Volume Claim
- generally created by non-admin, usually the dev team and it is created
in the project scope
- If the Storage Controller is able to locate a matching Peristent
Volume, then it let the PVC go and claim and use the PV
- the applicaiton Pod that needs the store will refer the PVC name in
the deployment, and it can request the external store
  to be mounted in its preferred mount point within the Pod
```

Lab - Deploy wordpress and mariadb multi-pod application

```
# Clone the TekTutor Training repository if you haven't done it already on the
lab machine
```

```
cd ~
git clone https://github.com/tektutor/openshift-july-2026.git
cd openshift-july-2026
cd Day3/wordpress-with-configmaps-and-secrets
# Ensure name 'jegan' is replaced with yours in all the yml file
# Ensure /var/nfs/jegan/wordpress, /var/nfs/jegan/mysql is replaced with your
linux user ( update mysql-pv.yml mysql-pvc.yml wordpress-pv.yml wordpress-
pvc.yml )
# Ensure NFS IP is updated to 192.168.10.201 in case you are working in server 2
# Ensure mysql-deploy.yml update the pvc name with the one that you created
# Ensure wordpress-deploy.yml update the pvc name with the one that you created

oc delete project jegan-project
oc new-project jegan-project
./deploy.sh

oc get pods

# Then switch to Openshift webconsole Topolgy and click on the wordpress route(up
arrow) to see the blog
```

Demo - Installing Helm package manager

```
sudo su -
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/
scripts/get-helm-4
chmod 700 get_helm.sh
./get_helm.sh
```

Info - Helm Overview

```
- helm is a package manager for Kubernetes & Openshift
- helm is a Kubernetes/Openshift aware tool, hence it knows how to find
the dependency and figure out the sequence in which
  K8s/Openshift resources must be created
- the helm packaged application is called Helm chart
- Openshift comes with pre-integrated Helm support
- However, we can deploy application from Helm Charts from command-line
as well as from the Openshift webconsole
```

Lab - Packaging wordpress multi-pod application as Helm chart and deploying into Openshift

```
cd ~
mkdir -p wordpress-helm-chart
cd wordpress-helm-chart

helm version
helm create wordpress
tree
cd wordpress/templates
rm *
rm -rf tests
cp ~/openshift-july-2026/Day3/wordpress-helm-chart/manifest-scripts/*.yml .
cd ..
cp ~/openshift-july-2026/Day3/wordpress-helm-chart/values.yaml .
cd ..
tree wordpress

helm package wordpress
helm install worpress wordpress-0.1.0.tgz
oc get pods

# Switch to your openshift webconsole topology in your project

# Once you are done you may undeploy
helm list
helm uninstall wordpress
```

Day 4

Demo - Statefulset - mysql cluster

```
cd ~/openshift-july-2026
git pull
cd Day4/statefult
oc apply -f sfs.yml
oc get pods -w
```

Lab - Node Affinity

Note

```
- There are two types of Node Affinity
  1. Preferred
     - In this case, the scheduler will try to locate nodes that meets
the criteria mentioned under preferred
       node affinity section, if scheduler finds such nodes, your
application gets deployed there
     - In case the scheduler is not able to find nodes that meets your
criteria, the pods gets deployed in any node
  2. Required
     - In this case, the scheduler will only deploy your pods on nodes
that meets the criteria
     - Until scheduler finds such nodes that meets your criteria, your
pods will be in the Pending state
```

Let's try the preferred node affinity

```
cd ~/openshift-july-2026
git pull
cd Day4/node-affinity
cat preferred-node-affinity.yml
oc project jegan-project

# List all nodes that has label disk=ssd
oc get nodes -l disk=ssd

# Assuming node nodes meets that criteria, let's see what scheduler does in this
scenario
oc apply -f preferred-node-affinity.yml
oc get pods -o wide # As the scheduler couldn't find nodes that has label
disk=ssd, it would have deployed your pods as usual

# Let's label a node with lab disk=ssd
oc label node/worker03.ocp4.palmeto.org disk=ssd
oc get nodes -l disk=ssd

# List and see to note the pods deployed already will not be redeployed to
worker03 as node affinity is not applied on already
# deployed pods
oc get pods -o wide

# Let's delete the deployment and redeploy
oc delete -f preferred-node-affinity.yml
oc apply -f preferred-node-affinity.yml
```

```
oc get pods -o wide # Now all your nginx pods must be deployed to worker03

# Let's delete the preferred-node-affinity.yml deployment to proceed with
required-node-affinity
oc delete -f preferred-node-affinity.yml

# Let's remove the label from worker03 node
oc label node/worker03.ocp4.palmeto.org disk-
oc get nodes -l disk=ssd

# Let's deploy the required-node-affinity nginx deployment
oc apply -f required-node-affinity.yml
oc get pods -o wide # all pods are supposed to be in Pending as scheduler
couldn't find a node that meets your app required criteria

# Label one of the worker node with disk=ssd
oc label node/worker01.ocp4.palmeto.org disk=ssd
oc get pods -o wide # now all your nginx pods should be running in worker01
```

Lab - Ingress

Note

```
- Ingress is a way to make our application accessible outside the
cluster with public url
- Ingress is not Service, it is routing(forwarding) set of rules
- In your openshift cluster, depending on which Load Balancer is setup
by your Openshift Cluster Administrator, we
  need to use either
  1. HAProxy Ingress Controller or
  2. Nginx Ingress Controller or
  3. F5 Ingress Controller, etc
- Whenever we create an Ingress resource, the Ingress Controller that is
active in the cluster will keep monitoring
  for the Ingress resource created in any project within the cluster
- Whenever the Controller detects such a Route, it picks the rules
mentioned in the Ingress resource and configures
  a Load Balancer
- HAProxy Ingress Controller only know how to configure a HAProxy Load
Balancer
- Nginx Ingress Controller knows only how to configure a Nginx Load
Balancer
- It is also important that our Ingress annotation mentions the Ingress
Controller that should manage our Ingress
- Ingress is not a service
  - as Ingress will forward the call to one of the service based on rule
defined in the Ingress
```

```
  - in other words, the backend of Ingress is set of services
( CluterIP, NodePort or LoadBalancer Services )
  - Ingress represents a group of Services
- Service ( ClusterIP, NodePort or LoadBalancer )
  - Service represents a group of load-balanced pods
  - the backend of these services are Pods
```

Let's delete and recreate a new project

```
oc delete project jegan-project
oc new-project jegan-project

cd ~/openshift-july-2026
git pull
cd Day4/ingress
oc apply -f nginx-deploy.yml
oc apply -f hello-deploy.yml
oc apply -f nginx-svc.yml
oc apply -f hello-svc.yml
oc describe svc/nginx
oc describe svc/hello
oc apply -f ingress.yml
oc get ingress

curl http://tektutor.apps.ocp4.palmeto.org/hello
curl http://tektutor.apps.ocp4.palmeto.org/nginx
```

Lab - How two microserives exchange data via Queue in a asynchronous fashion

```
oc delete project jegan
oc new-project jegan

oc new-app --name=jms-producer https://github.com/tektutor/openshift-
july-2026.git --context-dir=Day4/jms-demo/producer --strategy=docker
oc new-app --name=jms-consumer https://github.com/tektutor/openshift-
july-2026.git --context-dir=Day4/jms-demo/consumer --strategy=docker

oc logs -f buildconfig/jms-producer
oc logs -f buildconfig/jms-consumer

oc get pods

# Terminal 1
oc logs -f deploy/jms-producer
```

```
# Terminal 2
oc logs -f deploy/jms-consumer
```

Lab - Securing your application using https url (edge route)

```
oc delete project jegan-project
oc new-project jegan-project
oc create deploy nginx --image=image-registry.openshift-image-registry.svc:5000/
openshift/bitnami-nginx:1.28 --replicas=3
oc get deploy,pods

# The command below creates a clusterip internal service for nginx deployment
oc expose deploy/nginx --port=8080

openssl version
# Let's generate a private key
openssl genrsa -out key.key

# Let's create a public key using the private key for your organization domain
openssl req -new -key key.key -out csr.csr -subj="/CN=nginx-
jegan.apps.ocp4.palmeto.org"

# Sign the public key using the private key and generate certificate(.crt)
openssl x509 -req -in csr.csr -signkey key.key -out crt.crt

# The command below creates an edge(https) route
oc create route edge --service nginx --hostname nginx-jegan.apps.ocp4.palmeto.org
--key key.key --cert crt.crt

oc get route
curl --insecure https://nginx-jegan.apps.ocp4.palmeto.org
```

Day 5

Info - How many users one Pod can handle?

```
- there is no fixed number
- it depends on your application's per-request cost and its concurrency
model
- how to compute the concurrent cacacity
  concurrent requests = ( throughput in req/sec ) x ( average request
duration in sec )
```

```
- if each request takes 200ms and one pod can support upto 250~500
request/second
- a lightweight SpringBoot application with no database dependency with
500m CPU and 512Mi RAM can approximately
  handle 1000+ requests/second
- the same springboot pod with 40ms database query per request drops to
roughly 200~400 concurrent users
- Tomcat can handle 10 concurrent database-bound requests
- Benchmarking is the right way to find the number as concurrency
depends on your application's ability
  and how long it takes to respond a single request
```

Info - Openshift S2I

```
- In Kubernetes, we can only deploy application using container images
- In Openshift, we can deploy applicaiton using container images and
from github(or any other version control) url i.e from source code
- This feature in Openshift is called S2I ( Source to Image )
- Openshift supports many different types of strategies
  1. Docker
  2. Source
  3. Custom
  4. Pipeline(Jenkins/TekTon)
  5. Binary(S2I Binary)
```

Info - Openshift S2I Docker Strategy

```
- You will have to provide your version control ur, for example -
https://github.com/tektutor/spring-ms.git
- The GitHub url should have your application source code and Dockerfile
- BuildConfig will be autogenerated by Openshift
- Application will be build to create application binary, using your
application binary it will create a custom image
- the custom image built will be pushed into Openshift's Internal Image
Registry
- It will then deploy your application using the image it pushed into
Openshift Internal Image Registry
- It will automatically create BuildConfig, ImageStream, Deployment and
Service for your application
- oc expose svc/hello - you need to create your own route
- you can then access the application using the route url from command-
line or from your openshift web console
```

Info - OpenShift S2I Source Strategy

```
- You will have to provide your version control ur, for example - https://github.com/tektutor/spring-ms.git
- The GitHub url should have your application source code, Dockerfile is not required even if it is there it will be ignored
- BuildConfig will be autogenerated by Openshift
- Application will be build to create application binary, using your application binary it will create a custom image
- the custom image built will be pushed into Openshift's Internal Image Registry
- It will then deploy your application using the image it pushed into Openshift Internal Image Registry
- It will automatically create Dockerfile, BuildConfig, ImageStream, Deployment and Service for your application
- oc expose svc/hello - you need to create your own route
- you can then access the application using the route url from command-line or from your openshift web console
```

Info - Microservice Overview

```
- Microservice is an architectural style where application is built as a collection of small,
  independent and loosely coupled services
- each microservice will be responsible for specific business logic
- microservice to microservice communications can be achieved with HTTP/REST, gRPC or message queues
- Key properties of Microservices
  - Single Responsibility
  - Can be deployed independently
  - Technology agnostic - can be developed using any language stack
  - Decentralized Database
  - Resilience - Failures in one service will not affect others or will not bring down the entire application
  - Scalability - every microservice can be scaled up/down independent of other microservices
```

Info - Monolithic Application Overview

```
- is a traditional application that is built as a single application binary
- all components will be bundled in the single application binary
  - Frontend
```

```
  - Business Layer
  - Data Access Layer
- Key properties of Monolithic applications
  - Single codebase
  - Tightly coupled components
  - Entire application is deployed as a single unit
  - Centralized Database
  - Scaling Limitations
```

Info - Secure your Openshift Cluster and applications deployed in Openshift

Authentication & Authorization

```
- Authentication
  - Could setup Identity providers (IDPs) to authenticate openshift
login
  - LDAP, GitHub, GitLab, Google, HTPasswd, Keycloak (OIDC)
- ServiceAccounts vs User accounts
- Token and OAuth flows (oc whoami -t)

- Authorization
  - RBAC (Role-Based Access Control)
  - Roles, ClusterRoles, RoleBindings, ClusterRoleBindings
  - Custom Roles vs. default roles (admin, view, edit, etc.)
  - Least privilege principle
```

Network Security

```

- Pod-to-Pod & Pod-to-Service
- NetworkPolicy (namespaced) and ClusterNetworkPolicy (cluster-wide)
- Default deny-all and zero-trust networking
- Controlling ingress/egress with egress firewall rules
- Ingress Security
- Routes and TLS termination types
- Securing routes with certificates
```

Cluster Security & Hardening

```

- Security Context Constraints (SCC) – OpenShift equivalent of
PodSecurityPolicies
```

```
- restricted, anyuid, privileged SCCs
- Creating custom SCCs
- Admission Controllers and Validating/Mutating Webhooks
- Pod Security Standards (Baseline, Restricted)
- Running containers as non-root
- Disabling privileged escalation
```

Image & Supply Chain Security

```
- ImageStreams and trusted registries
- image signature verification (oc adm policy add-scc-to-user, oc adm
signature)
- Image scanning (e.g., Red Hat Quay, Clair, Trivy)
- Content Trust and ImagePolicy admission
- Immutable images and reproducible builds
- Security scanning in CI/CD pipelines
```

Secrets & Config Security

```
- Kubernetes Secrets vs ConfigMaps
- Encryption at rest with KMS providers (e.g., AWS KMS, Vault)
- Sealed Secrets (Bitnami SealedSecrets)
- External secret managers (HashiCorp Vault, CyberArk, AWS Secrets
Manager)
- Securing environment variables in deployments
```

Platform Access & API Security

```
- API server authentication and RBAC for oc and kubectl users
- OpenShift Console access restrictions
- API audit logging
- Certificate management for API server and etcd
- OAuth token lifetimes and rotation
```

Compliance & Monitoring

```
- Compliance Operator (CIS Benchmark for OpenShift)
- OpenSCAP scans and remediation
- Audit logging and forwarding (e.g., to ELK or SIEM)
```

```
- Security events in oc get events and cluster logs
- Cluster-wide alerts for security breaches
```

Security Tools & Operators

```
- Compliance Operator — CIS, NIST profiles
- Gatekeeper / OPA — policy as code
- ACS (Red Hat Advanced Cluster Security) — runtime threat detection,
network graph
- Falco — behavioral threat detection
- Trivy — image scanning in CI/CD
```

Secure Multitenancy

```
- Namespace isolation best practices
- Project quotas and limits
- Network segmentation per tenant
- RBAC delegation to team leads
- Resource constraints to prevent noisy neighbor issues
```

Security Updates & Patch Management

```
- OpenShift upgrade process & channels (fast, stable)
- Node OS updates (RHCOS)
- Operator lifecycle management (OLM) security
- CVE tracking and remediation using Red Hat Insights
```

Runtime & Workload Security

```
- Pod security context:
  - runAsUser, fsGroup, readOnlyRootFilesystem
- Dropping Linux capabilities
  - Seccomp profiles and AppArmor (on supported platforms)
- Container resource limits to prevent DoS
- Detecting crypto-mining and lateral movement
```

Ingress/Egress & Perimeter Security

```
- Securing ingress with TLS, WAF integration (e.g., F5, HAProxy)
- Rate limiting & DoS protection
- Egress control with EgressFirewall and proxies
- Bastion hosts & jump boxes for controlled access
```

Backup & Disaster Recovery (Security Angle)

```
- etcd encryption and backup security
- Secrets encryption during backups
- Immutable backup storage
- RBAC restrictions for restore operations
```

Zero Trust Architecture

```
- Network segmentation
- Identity-aware access to workloads
- Mutual TLS between services
- Policy enforcement using OPA/Gatekeeper
- Just-in-time access control
```

Best Practices & Standards

```
- CIS Benchmark for OpenShift
- NIST 800-53 alignment
- ISO 27001 security practices
- Regular pen-testing and red teaming
- Shift-left security in CI/CD
```

Info - Jenkins Overview

```
- Kohsuke Kawaguchi was a former employee of Sun Microsystems
- In he developed a Continuous Integration Build Server to automate his
testing at Sun Microsystems as a hobby
- This product originally called Hudson and was an opensource project
- Meanwhile, Oracle acquired Sun Microsystems
```

```
- Koshuke Kawaguchi founded a company called Cloudbees
- When he quit Oracle, he had created a fork of Hubson branch and new
branch was named as Jenkins
- Cloudbees is the Enterprise variant of Jenkins
- Jenkins has over 10000+ active opensource contributors around the
globe
```

Info - Openshift Network Model

```
- Openshift's network operates in layers
- It is important to understand how traffic flows through each layers
for debugging & troubleshooting
- Core Concepts
  1. Container Network Interface (CNI)
  2. Pod to Pod Communication
  3. Services and Load Balancing
```

Container Network Interace (CNI)

```
- Openshift uses a CNI plugin to handle pod networking
- the default is OVN-Kubernetes ( Openshift Virtual Network)
- OVN creates a virtual network overlay where every pod gets a real,
routable IP address
  from the cluster subnet
```

Pod to Pod Communication

```
- When a Pod A sends traffic to Pod B, OVN handles the routing without
needing kube-proxy
- Packets flow through the overlay network
- Both pods live on the same logical network namesapces, if even if they
are on different nodes
- The network plugin encapsulates packets and routes them to the correct
node
```

Services and Load Balancing

```
- Services abstract pods IP addresses as Pod IPs are ephemeral
- When you create a service, it gets a stable cluster IP
- The kube-proxy watches endpoints and programs iptable rules on every
node
- The rules use DNAT with random probability-weighted distribution for
```

```
new connections
- each new connection picks a backend pod randomly according to their
weights
- Pod to Pod communication via Internal Service
  - Pod A queries the service DNS name, gets resolved to cluster IP
  - Packtet hits a nodes's iptable rules
  - DNAT rewrites the destination to a real Pod IP
  - OVN routes to the target node and pod
- Pod to Pod communication via External Route
  - Uses Route or Ingress object
  - Route creates a HAProxy rule on the ingress controller pods
  - Traffic lands on the Ingress controller, which forwards to the
service
  - service load balancing takes over from there
- Egress ( pod to external traffic )
  - Pods use their Node's IP aas source
  - Outbound traffic appears to come from the cluster, not from
individual pods
```

Info - Subnet

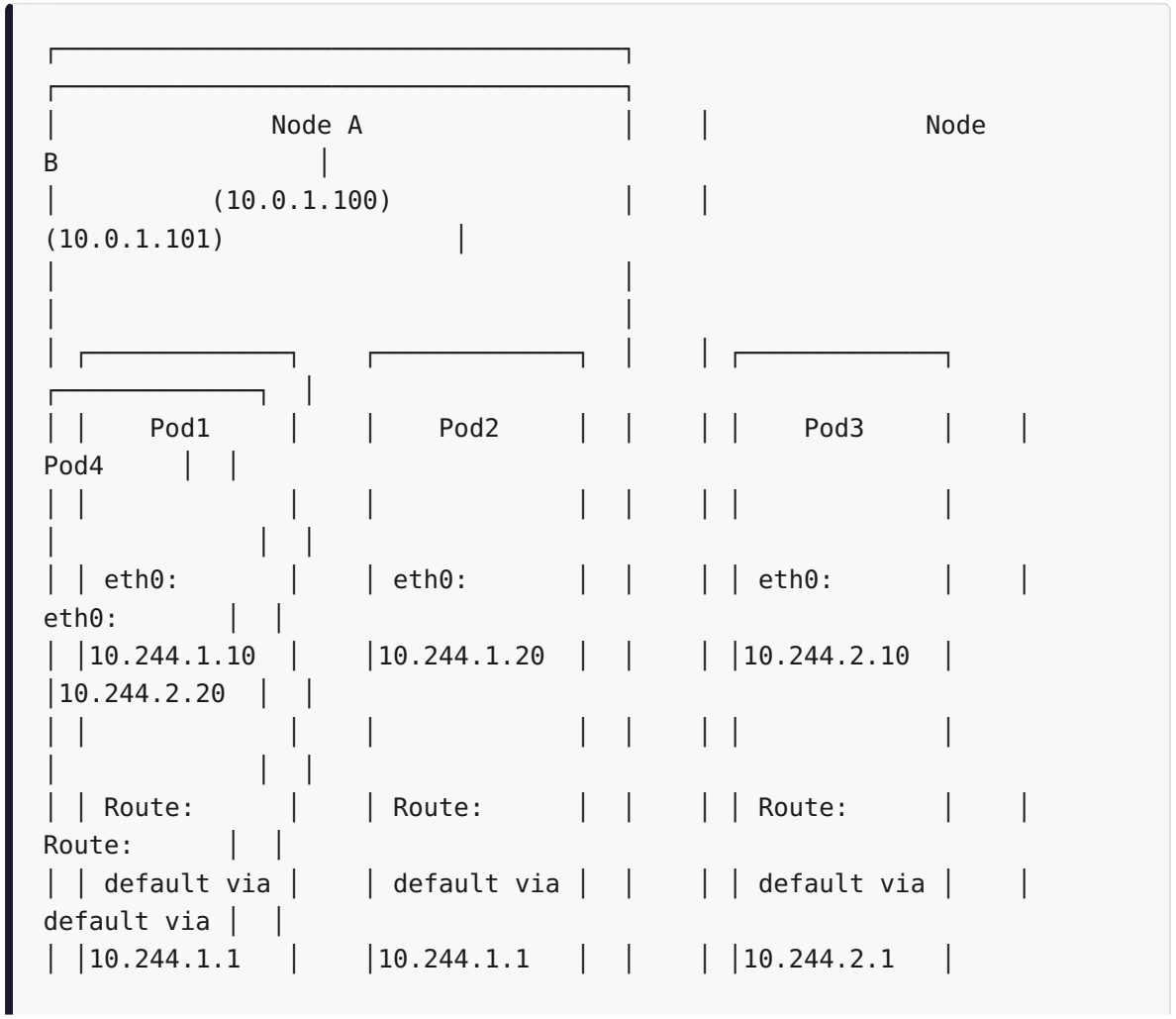
```
- Range of IP address
- a bigger network is broken down to small network called subnet to
apply different security policy or access
- IPV4 IP Address
  - 100.200.3.20 ( 4 bytes )
  - A.B.C.D
  - A - 1 byte ( 8 bits )
  - B - 1 byte ( 8 bits )
  - C - 1 byte ( 8 bits )
  - D - 1 byte ( 8 bits )
  - 1 byte we represents numbers in the range 0 to 255 (256 values)
- Let's say there is subnet 10.131.0.0/24
  - 10 is fixed ( 8 bits )
  - 131 is fixed ( 8 bits )
  - 0 is fixed ( 8 bits )
  - How many IP addresses are there in 10.131.0.0/24 - 256 IP addresses
are there
```

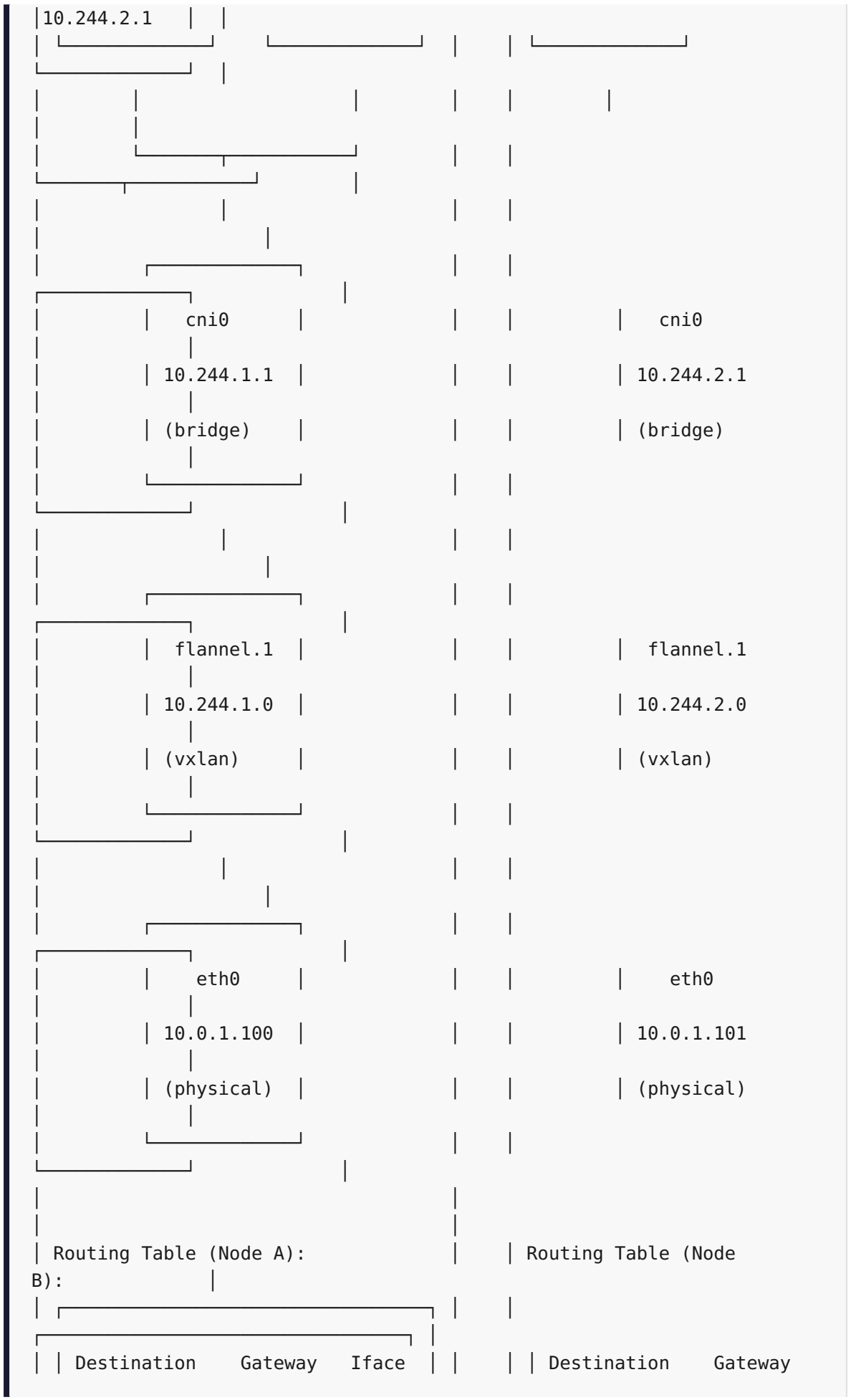
Flannel Network Model

```
Flannel is a simple and easy way to configure a layer 3 network fabric
designed for Kubernetes.
It provides networking for container clusters by creating a virtual
network that spans across all nodes.

Key Concepts
```

```
- Overlay Network
- Creates a virtual network layer on top of the existing host network
- Each node gets a subnet from a larger cluster network
- Pods can communicate across nodes using this overlay
- Network Backends
- VXLAN (Default)

Encapsulates packets in UDP
- Works across most network infrastructures
- Good performance with modern kernels

Host-Gateway
- Uses host routing tables
- Better performance but requires Layer 2 connectivity
- No packet encapsulation overhead

UDP
- Legacy backend
- User-space packet forwarding
- Lower performance but maximum compatibility
```

Architecture




10.244.2.1
cni0
10.244.1.1
(bridge)
cni0
10.244.2.1
(bridge)
flannel.1
10.244.1.0
(vxlan)
flannel.1
10.244.2.0
(vxlan)
eth0
10.0.1.100
(physical)
eth0
10.0.1.101
(physical)
Routing Table (Node A):
Destination Gateway Iface
Routing Table (Node B):
Destination Gateway

```
Iface  | |
| | 10.244.1.0/24  0.0.0.0   cni0   | |    | | 10.244.2.0/24  0.0.0.0
cni0   | |
| | 10.244.2.0/24  10.244.2.0 fl.1  | |    | | 10.244.1.0/24  10.244.1.0
fl.1  | |
| | 10.244.0.0/16  0.0.0.0   fl.1   | |    | | 10.244.0.0/16  0.0.0.0
fl.1   | |
| | 10.0.1.0/24    0.0.0.0   eth0   | |    | | 10.0.1.0/24    0.0.0.0
eth0   | |
| | default        10.0.1.1  eth0   | |    | | default        10.0.1.1
eth0   | |
| └──────────────────────────────────┘ |    |
└──────────────────────────────────────┘ |
|                                        |
|                                        |
| VXLAN FDB (Forwarding Database):    |    | VXLAN FDB (Forwarding
Database):    |
| ┌──────────────────────────────────┐ |    |
┌──────────────────────────────────────┐ |
| | MAC Address       IP Address    | |    | | MAC Address       IP
Address    | |
| | aa:bb:cc:dd:ee:02 10.0.1.101    | |    | | aa:bb:cc:dd:ee:01
10.0.1.100    | |
| └──────────────────────────────────┘ |    |
└──────────────────────────────────────┘ |
└──────────────────────────────────────────┘
└──────────────────────────────────────────┘
                      |                                          |
                      └──────────────────┬───────────────────────┘
                                         |
                                  Physical Network
                                   (10.0.1.0/24)
```

Packet Flow: Pod1 to Pod3 with Flannel VXLAN

Detailed Packet Capture Analysis
Scenario: Pod1 (10.244.1.10) pings Pod3 (10.244.2.10)

```
# Inside Pod1 - Original ICMP packet
tcpdump -i eth0 -nn icmp
```

Packet Structure:

```
┌──────────────────────────────────────────────────────────────┐
│                    Ethernet Header                           │
├──────────────────────────────────────────────────────────────┤
| Dst MAC: 02:42:0a:f4:01:01 (cni0 bridge)                     |
```

```
| Src MAC: 02:42:0a:f4:01:0a (Pod1 eth0)                          |
| EtherType: 0x0800 (IPv4)                                        |
├─────────────────────────────────────────────────────────────────┤
|                       IP Header                                 |
├─────────────────────────────────────────────────────────────────┤
| Version: 4, Header Length: 20 bytes                             |
| Source IP: 10.244.1.10                                          |
| Destination IP: 10.244.2.10                                     |
| Protocol: 1 (ICMP)                                              |
| TTL: 64                                                         |
├─────────────────────────────────────────────────────────────────┤
|                      ICMP Header                                |
├─────────────────────────────────────────────────────────────────┤
| Type: 8 (Echo Request)                                          |
| Code: 0                                                         |
| Identifier: 1234                                                |
| Sequence: 1                                                     |
| Data: "Hello from Pod1"                                         |
└─────────────────────────────────────────────────────────────────┘
```

Packet on Node A (before VXLAN encapsulation)

```
# On Node A - Capture on cni0 bridge
sudo tcpdump -i cni0 -nn icmp
# 15:30:45.123456 IP 10.244.1.10 > 10.244.2.10: ICMP echo request, id 1234, seq 1
```

VXLAN Encapsulated Packet on Physical Network

```
# On Node A - Capture on eth0 (physical interface)
sudo tcpdump -i eth0 -nn 'port 8472'
```

Encapsulated Packet Structure:

```
┌─────────────────────────────────────────────────────────────────┐
|                 Outer Ethernet Header                           |
├─────────────────────────────────────────────────────────────────┤
| Dst MAC: aa:bb:cc:dd:ee:02 (Node B eth0)                        |
| Src MAC: aa:bb:cc:dd:ee:01 (Node A eth0)                        |
| EtherType: 0x0800 (IPv4)                                        |
├─────────────────────────────────────────────────────────────────┤
|                   Outer IP Header                               |
├─────────────────────────────────────────────────────────────────┤
| Version: 4, Header Length: 20 bytes                             |
| Source IP: 10.0.1.100 (Node A)                                  |
| Destination IP: 10.0.1.101 (Node B)                             |
| Protocol: 17 (UDP)                                              |
```

```
| TTL: 64                                                                 |
├─────────────────────────────────────────────────────────────────────────┤
|                        UDP Header                                       |
├─────────────────────────────────────────────────────────────────────────┤
| Source Port: 45678 (random)                                             |
| Destination Port: 8472 (VXLAN)                                          |
| Length: 78 bytes                                                        |
├─────────────────────────────────────────────────────────────────────────┤
|                      VXLAN Header                                       |
├─────────────────────────────────────────────────────────────────────────┤
| Flags: 0x08 (Valid VNI)                                                 |
| VNI: 1 (Virtual Network Identifier)                                     |
| Reserved: 0x000000                                                      |
├─────────────────────────────────────────────────────────────────────────┤
|                  Inner Ethernet Header                                  |
├─────────────────────────────────────────────────────────────────────────┤
| Dst MAC: 02:42:0a:f4:02:01 (Node B cni0)                                |
| Src MAC: 02:42:0a:f4:01:01 (Node A cni0)                                |
| EtherType: 0x0800 (IPv4)                                                |
├─────────────────────────────────────────────────────────────────────────┤
|                    Inner IP Header                                      |
├─────────────────────────────────────────────────────────────────────────┤
| Version: 4, Header Length: 20 bytes                                     |
| Source IP: 10.244.1.10 (Pod1)                                           |
| Destination IP: 10.244.2.10 (Pod3)                                      |
| Protocol: 1 (ICMP)                                                      |
| TTL: 63 (decremented by 1)                                              |
├─────────────────────────────────────────────────────────────────────────┤
|                      ICMP Header                                        |
├─────────────────────────────────────────────────────────────────────────┤
| Type: 8 (Echo Request)                                                  |
| Code: 0                                                                 |
| Identifier: 1234                                                        |
| Sequence: 1                                                             |
| Data: "Hello from Pod1"                                                 |
└─────────────────────────────────────────────────────────────────────────┘
```

Packet Capture Commands and Output

On Node A:

```
# Capture original packet leaving Pod1
sudo tcpdump -i veth12345678 -nn icmp -v
# 15:30:45.123456 IP (tos 0x0, ttl 64, len 84) 10.244.1.10 > 10.244.2.10:
#   ICMP echo request, id 1234, seq 1, length 64

# Capture VXLAN encapsulated packet on physical interface
sudo tcpdump -i eth0 -nn 'udp port 8472' -v
# 15:30:45.123460 IP (tos 0x0, ttl 64, len 134) 10.0.1.100.45678 >
10.0.1.101.8472:
```

```
#   VXLAN, flags [I] (0x08), vni 1
#   IP (tos 0x0, ttl 63, len 84) 10.244.1.10 > 10.244.2.10:
#     ICMP echo request, id 1234, seq 1, length 64
```

On Node B:

```
# Capture incoming VXLAN packet
sudo tcpdump -i eth0 -nn 'udp port 8472' -v
# 15:30:45.123465 IP (tos 0x0, ttl 64, len 134) 10.0.1.100.45678 >
10.0.1.101.8472:
#   VXLAN, flags [I] (0x08), vni 1
#   IP (tos 0x0, ttl 63, len 84) 10.244.1.10 > 10.244.2.10:
#     ICMP echo request, id 1234, seq 1, length 64

# Capture decapsulated packet on cni0
sudo tcpdump -i cni0 -nn icmp -v
# 15:30:45.123470 IP (tos 0x0, ttl 63, len 84) 10.244.1.10 > 10.244.2.10:
# ICMP echo request, id 1234, seq 1, length 64
```

Indide Pod3

```
# Final packet received by Pod3
tcpdump -i eth0 -nn icmp -v
# 15:30:45.123475 IP (tos 0x0, ttl 63, len 84) 10.244.1.10 > 10.244.2.10:
#   ICMP echo request, id 1234, seq 1, length 64
```

Return Path (Pod3 → Pod1)

The ICMP reply follows the reverse path:

```
# Pod3 sends reply
# 15:30:45.123480 IP 10.244.2.10 > 10.244.1.10: ICMP echo reply, id 1234, seq 1

# Node B encapsulates and sends via VXLAN
# 15:30:45.123485 IP 10.0.1.101.54321 > 10.0.1.100.8472:
#   VXLAN, flags [I] (0x08), vni 1
#   IP 10.244.2.10 > 10.244.1.10: ICMP echo reply, id 1234, seq 1

# Pod1 receives the reply
# 15:30:45.123490 IP 10.244.2.10 > 10.244.1.10: ICMP echo reply, id 1234, seq 1
```

Info - Calico Openshift Network Plugin

```
- Calico Network in OpenShift
- Calico is a popular Container Network Interface (CNI) plugin that
```

```
provides networking and
  network security for containerized applications in OpenShift clusters.
  What is Calico?
  - Calico is an open-source networking solution that delivers:

Pod-to-pod networking across nodes
- Network policy enforcement for security
- IP address management (IPAM)
- Service mesh integration
- Key Features in OpenShift
  1. Pure Layer 3 Networking
     - Uses standard IP routing instead of overlay networks
     - Better performance with lower latency
     - Simplified troubleshooting using standard networking tools
  2. Network Policies
     - Kubernetes-native network policies
     - Advanced Calico network policies with additional features

- Ingress and egress traffic control
  - Application-layer policy enforcement
- Scalability
  - Supports thousands of nodes
  - Efficient BGP routing protocols
  - Distributed architecture without single points of failure
- Security
  - Workload isolation at the network level
  - Encryption in transit
  - Integration with service mesh security

- Architecture Components
  - Felix Agent
    - Runs on each node as a DaemonSet
    - Programs routing tables and iptables rules
    - Handles network policy enforcement

  - BGP Client (BIRD)
    - Distributes routing information between nodes
    - Maintains network topology
    - Enables cross-node pod communication

  - Calico Controller
    - Watches Kubernetes API for network policy changes
    - Translates policies into Felix-readable format
    - Manages IP address allocation

- Common Use Cases
  - Multi-tenant environments requiring strong network isolation
  - High-performance applications needing low network latency
  - Hybrid cloud deployments with consistent networking policies
  - Compliance requirements demanding network traffic control
```

Comparison

```
+------------------+----------+---------------+----------------+
| Feature          | Calico   | OpenShift SDN | OVN-Kubernetes |
+------------------+----------+---------------+----------------+
| Performance      | High     | Medium        | Medium-High    |
| Network Policies | Advanced | Basic         | Basic          |
| Complexity       | Medium   | Low           | Medium         |
| BGP Support      | Yes      | No            | No             |
| Overlay Network  | Optional | Yes           | Yes            |
| eBPF Support     | Yes      | No            | Limited        |
| IPv6 Support     | Yes      | Limited       | Yes            |
| Windows Support  | Yes      | No            | Yes            |
| Encryption       | WireGuard| IPSec         | IPSec          |
| IPAM             | Custom   | Built-in      | Built-in       |
| Troubleshooting  | Standard | OpenShift     | OVN Tools      |
|                  | Tools    | Tools         |                |
+------------------+----------+---------------+----------------+
```

Info - Openshift Weave Network Plugin

```
- Weave Network Plugin in OpenShift
  - Weave Network (Weave Net) is a Container Network Interface (CNI)
plugin that
    can be used with OpenShift to provide networking between containers
and pods across a cluster.
- What is Weave Network?
  - Weave Network creates a virtual network that connects Docker
containers across multiple hosts
    and enables their automatic discovery. It provides:
- Overlay Network: Creates a Layer 2 network overlay that spans multiple
hosts
- Automatic IP Management: Assigns IP addresses to containers
automatically
- Service Discovery: Built-in DNS-based service discovery
- Network Segmentation: Support for network policies and
microsegmentation
- Key Features in OpenShift Context
  1. Multi-Host Networking
     - Connects pods across different OpenShift nodes
     - Handles pod-to-pod communication seamlessly
     - Supports both IPv4 and IPv6
  2. Network Policies
     - Implements Kubernetes NetworkPolicy resources
     - Provides microsegmentation capabilities
     - Allows traffic filtering between pods/namespaces
  3. Encryption
    - Optional encryption of inter-host traffic
```

```
    - Uses NaCl crypto library for performance
    - Helps secure pod communication across untrusted networks
  4. Integration with OpenShift
     - Works as a CNI plugin
     - Integrates with OpenShift's Software Defined Networking (SDN)
     - Compatible with OpenShift service mesh

Architecture Components
- Weave Router
- Runs as a DaemonSet on each node
- Handles packet forwarding and routing
- Maintains network topology information
- Weave Net Plugin
  - CNI plugin that interfaces with container runtime
  - Allocates IP addresses to pods
  - Configures network interfaces
  - IPAM (IP Address Management)
  - Distributes IP address ranges across nodes
  - Prevents IP conflicts
  - Supports custom IP ranges
```

Usecases

```
- Multi-cloud deployments where encryption is required
- Strict network segmentation requirements
- Legacy applications that need specific networking features
- Development environments requiring flexible networking
- Comparison with OpenShift SDN
```

Limitations

```
- Performance overhead especially with encryption enabled
- Additional complexity compared to default OpenShift SDN
- Resource consumption from running additional networking components
- Troubleshooting complexity due to overlay networking
- Weave Network provides a robust networking solution for OpenShift,
  particularly useful when advanced networking features like -
encryption and comprehensive
  network policies are required.
```

Flannel vs Calico vs Weave

```
+------------------------+---------------------------
+---------------------------+---------------------------+
| Feature                | Calico                    |
```

```
Flannel                    | Weave                       |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Overlay Technology     | BGP (L3), VXLAN, IPIP    | VXLAN, UDP, host-
gw       | VXLAN, UDP                |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Network Policies       | Full NetworkPolicy +      | No native
support         | Full NetworkPolicy        |
|                          | Calico policies
|                            | support                   |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Encryption             | WireGuard, IPSec          | No native
encryption      | Built-in NaCl encryption  |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Performance            | Excellent (native BGP)    | Good (simple
overlay)     | Good (with encryption     |
|                          |
|                            | overhead)                 |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Setup Complexity       | Complex (BGP knowledge    |
Simple                     | Moderate                  |
|                          | required)
|                            |                           |
+------------------------+---------------------------
+---------------------------+---------------------------+
| IP Address Management  | IPAM with IP pools        | Simple host-
subnet        | Distributed IPAM          |
|                          |                           |
allocation                 |                           |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Service Discovery      | Standard Kubernetes DNS   | Standard
Kubernetes DNS   | Built-in DNS + K8s DNS    |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Multi-tenancy          | Advanced policy engine    | Basic
namespace           | Network segmentation      |
|                          |                           |
isolation                  |                           |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Scalability            | Excellent (BGP routing)   | Good (limited by
etcd)    | Good (mesh topology)      |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Cross-subnet Support   | Native (BGP)              | Limited
```

```
(requires         | Good (automatic           |
|                        |                           | host-gw
mode)             | discovery)                |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Windows Support        | Yes                       |
Limited                    | No                         |
+------------------------+---------------------------
+---------------------------+---------------------------+
| eBPF Support           | Yes (advanced features)   |
No                         | No                         |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Resource Usage         | Low to moderate           | Very
low                  | Moderate to high          |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Troubleshooting        | Complex (BGP knowledge)   |
Simple                     | Moderate                   |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Enterprise Features    | Calico Enterprise         |
Limited                    | Weave Cloud (deprecated)  |
|                        | (advanced security)
|                           |                             |
+------------------------+---------------------------
+---------------------------+---------------------------+
| IPv6 Support           | Full dual-stack           |
Basic                      | Yes                        |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Network Observability  | Flow logs, metrics        | Basic
metrics             | Network map, metrics      |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Maturity               | Very mature               |
Mature                     | Mature                     |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Best Use Cases         | - Large scale clusters    | - Simple
deployments     | - Multi-cloud setups      |
|                        | - Advanced security       | - Learning/
development    | - Encryption required     |
|                        | - Compliance requirements | - Resource
constrained    | - Easy troubleshooting    |
|                        | - Multi-cloud             |
environments            | - Legacy app support      |
+------------------------+---------------------------
+---------------------------+---------------------------+
| Vendor/Maintainer      | Tigera (formerly          | CoreOS/Red
```

```
Hat            | Weaveworks                |
|                        | Project Calico)
|                           |                            |
+-------------------------+--------------------------
+---------------------------+---------------------------+
| OpenShift Support      | Yes (OVN-Kubernetes       |
Limited                   | Third-party plugin        |
|                        | uses Calico for policies)
|                           |                            |
+-------------------------+--------------------------
+---------------------------+---------------------------+
```

Lab - Deploying our custom spring-boot appliction into Openshift using S2I docker strategy

```
oc new-app --name=hello-microservice https://github.com/tektutor/openshift-
july-2026.git --context-dir=Day5/simple-springboot-microservice --strategy=docker

oc expose svc/hello-microservice

oc logs -f bc/hello-microservice

oc get svc,route

curl --insecure http://hello-microservice-jegan.apps.ocp4.palmeto.org
```

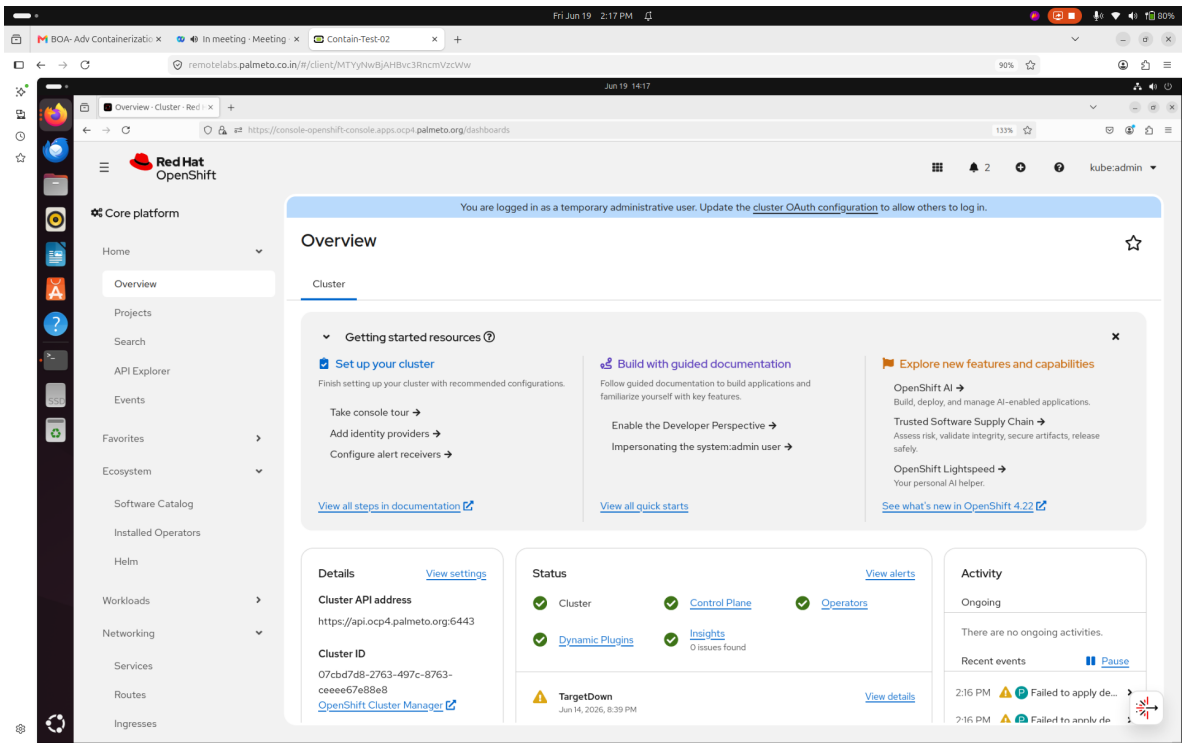
Lab - Horizontal Pod Auto-scaling based on CPU Utilization

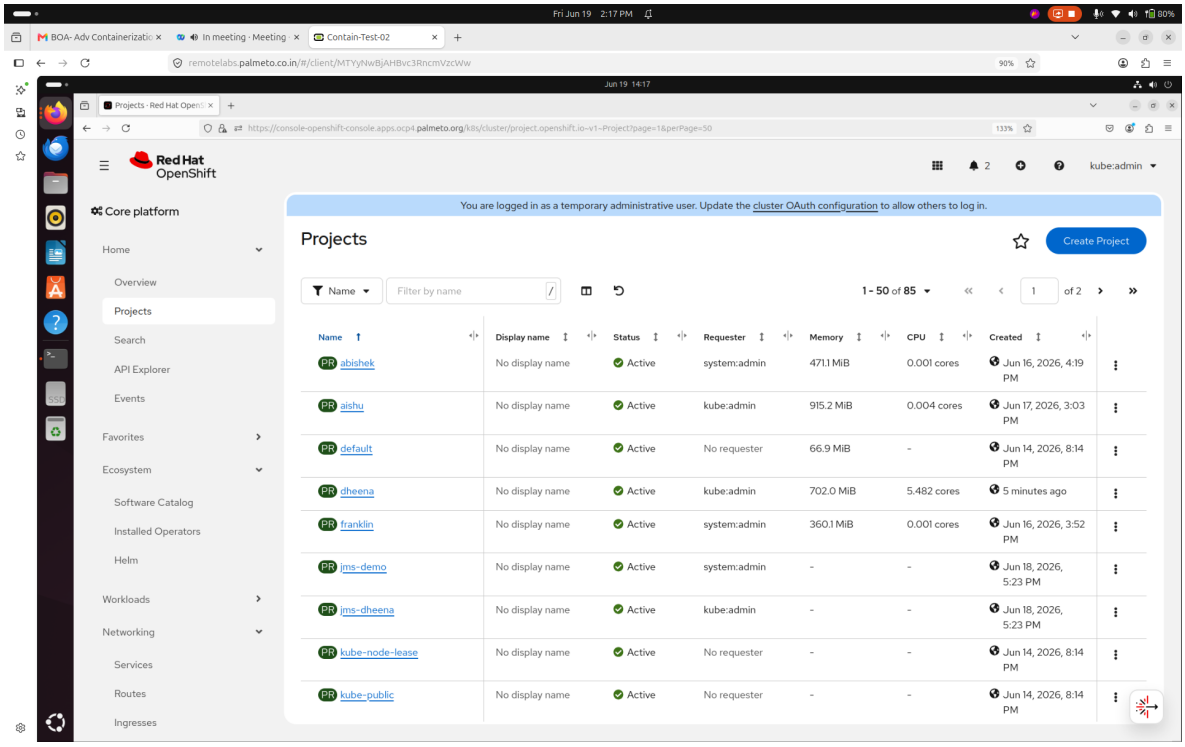
```
oc delete project jegan
oc new-project jegan

cd ~
git clone https://github.com/tektutor/openshift-july-2026.git
cd openshift-july-2026
cd Day5/auto-scaling
oc create -f hello-deploy.yml --save-config=true
oc get pods
oc create -f hello-hpa.yml --save-config

oc expose deploy/nginx --port=8080
oc expose svc/nginx
oc get route
```

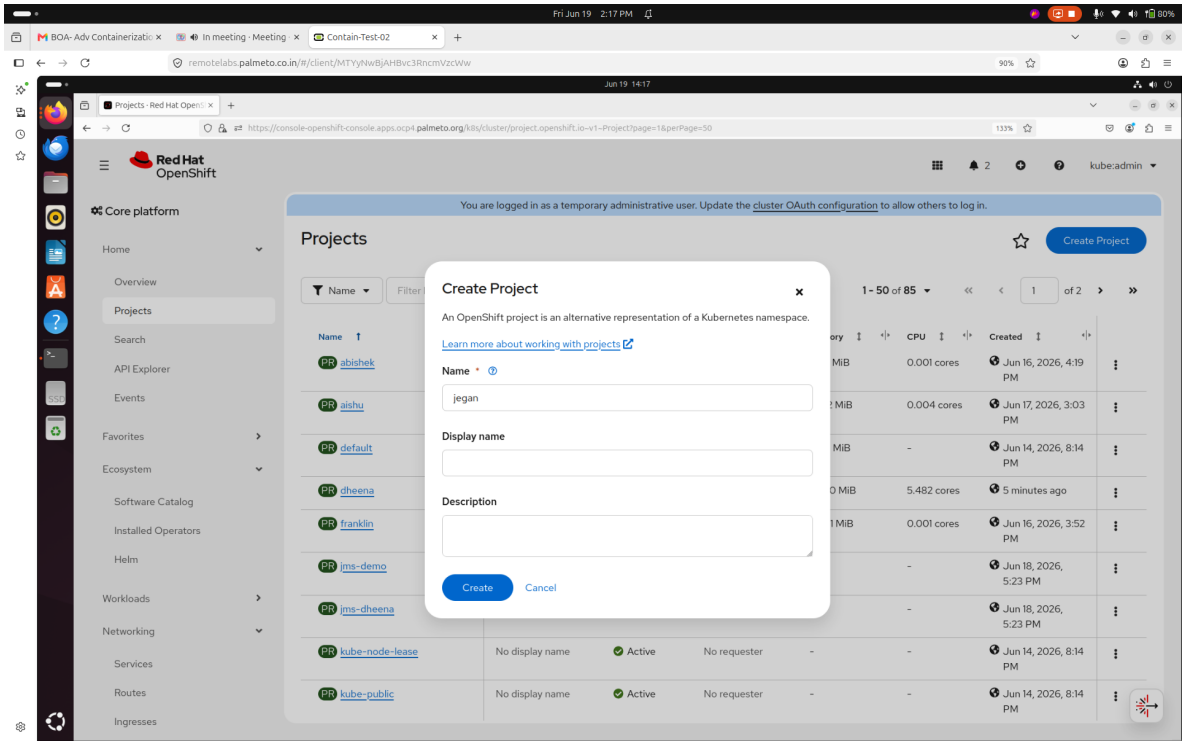
We need to stree the pod with more traffic

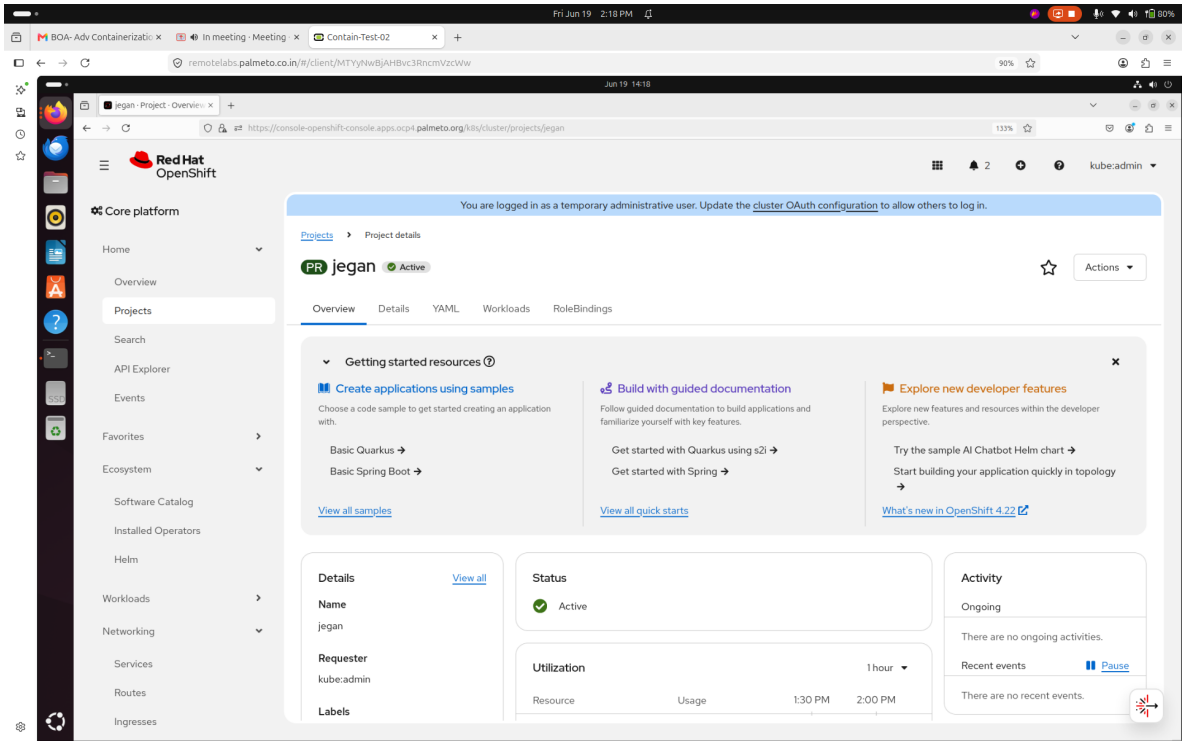
```
ab -k -n 200000 -c 1000 https://nginx-jegan.apps.ocp4.palmeto.org/
```

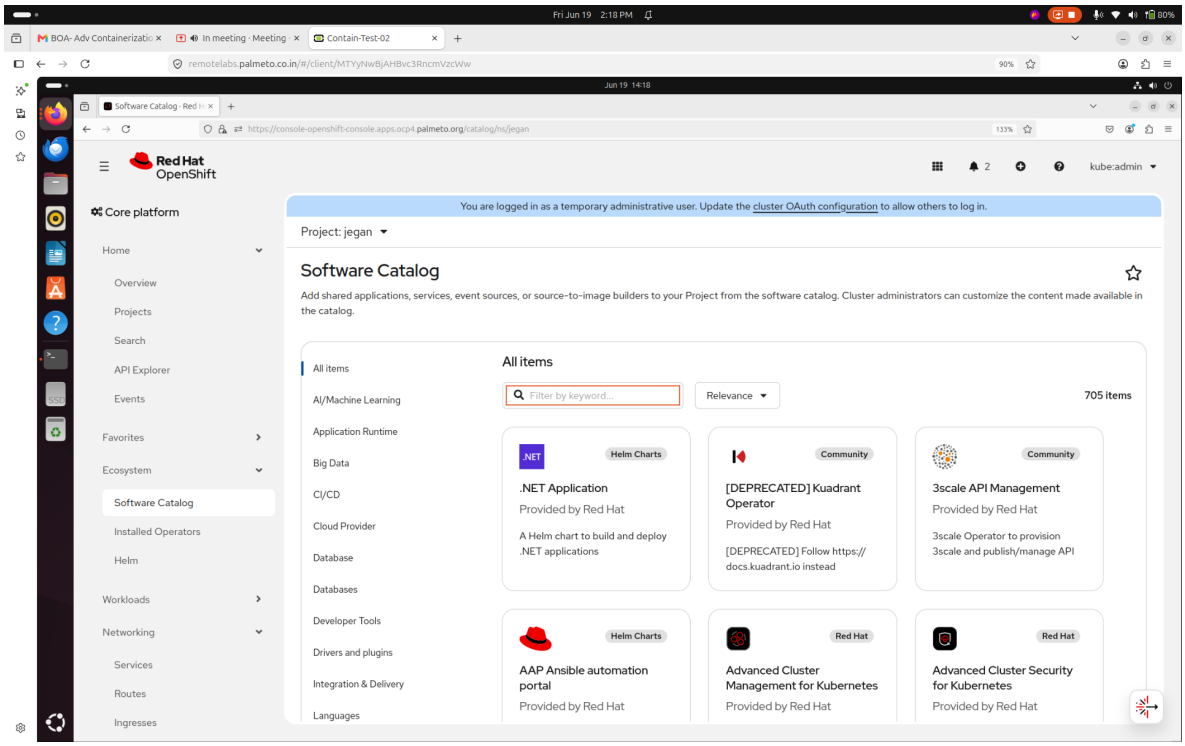
Lab - CICD with Jenkins & Red Hat Openshift

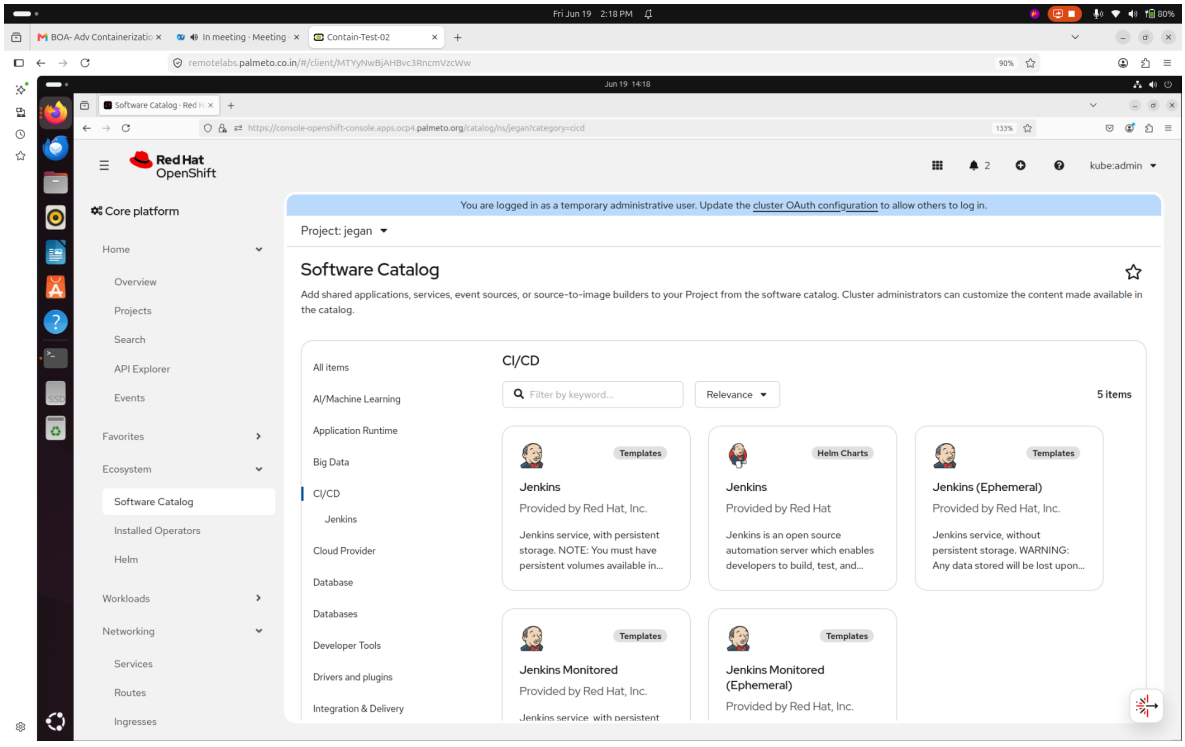


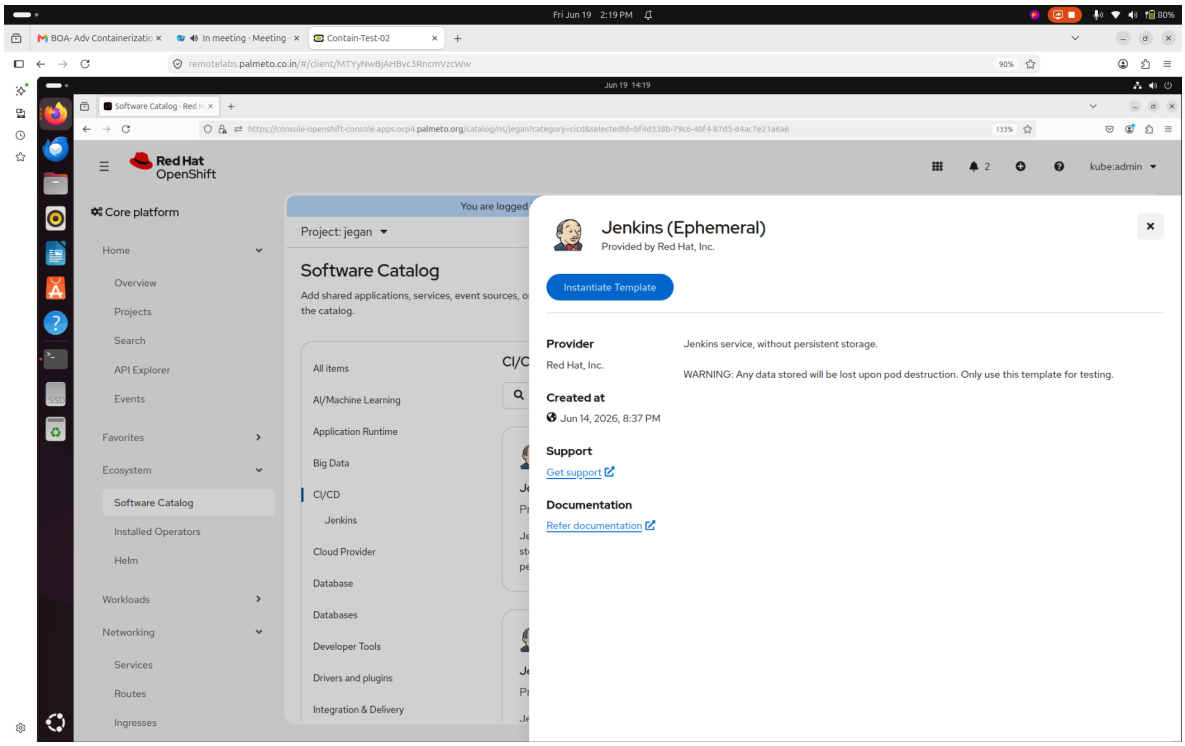


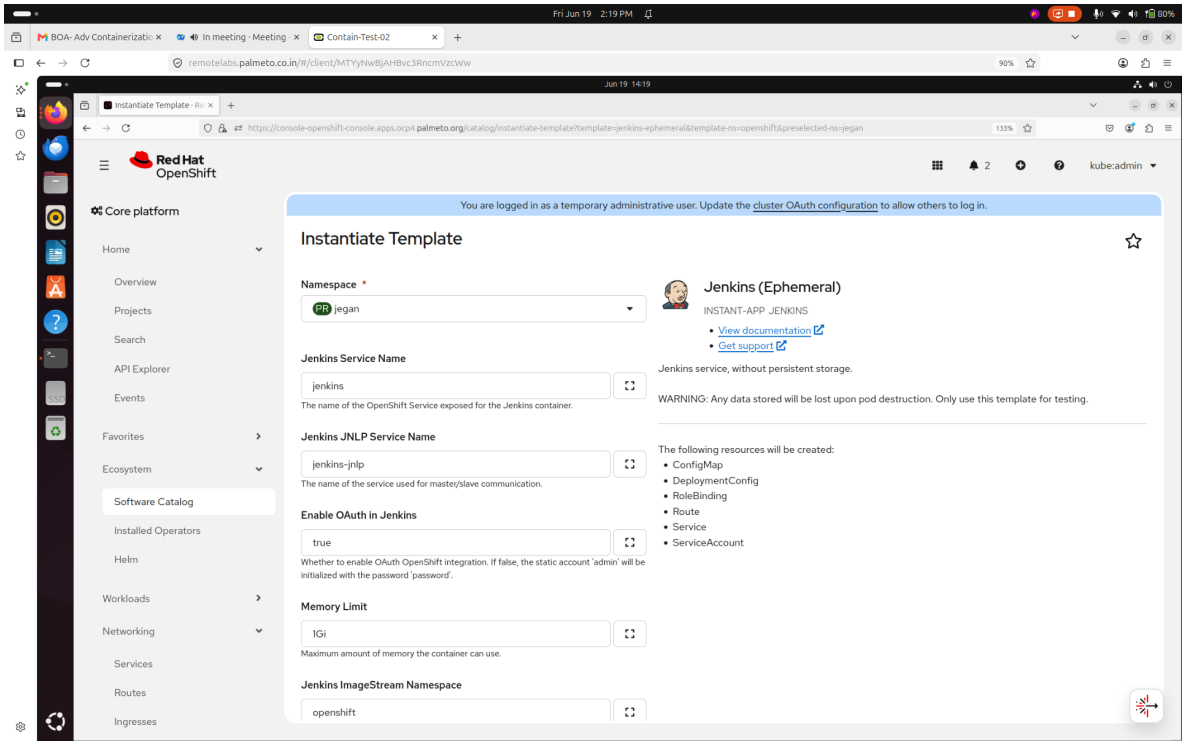

Red Hat OpenShift
Core platform
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses
kube:admin
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Projects
Create Project
Create Project
An OpenShift project is an alternative representation of a Kubernetes namespace.
Learn more about working with projects
Name *
jegan
Display name
Description
Create
Cancel
abishek
aishu
default
dheena
franklin
jms-demo
jms-dheena
kube-node-lease
kube-public
No display name
Active
No requester

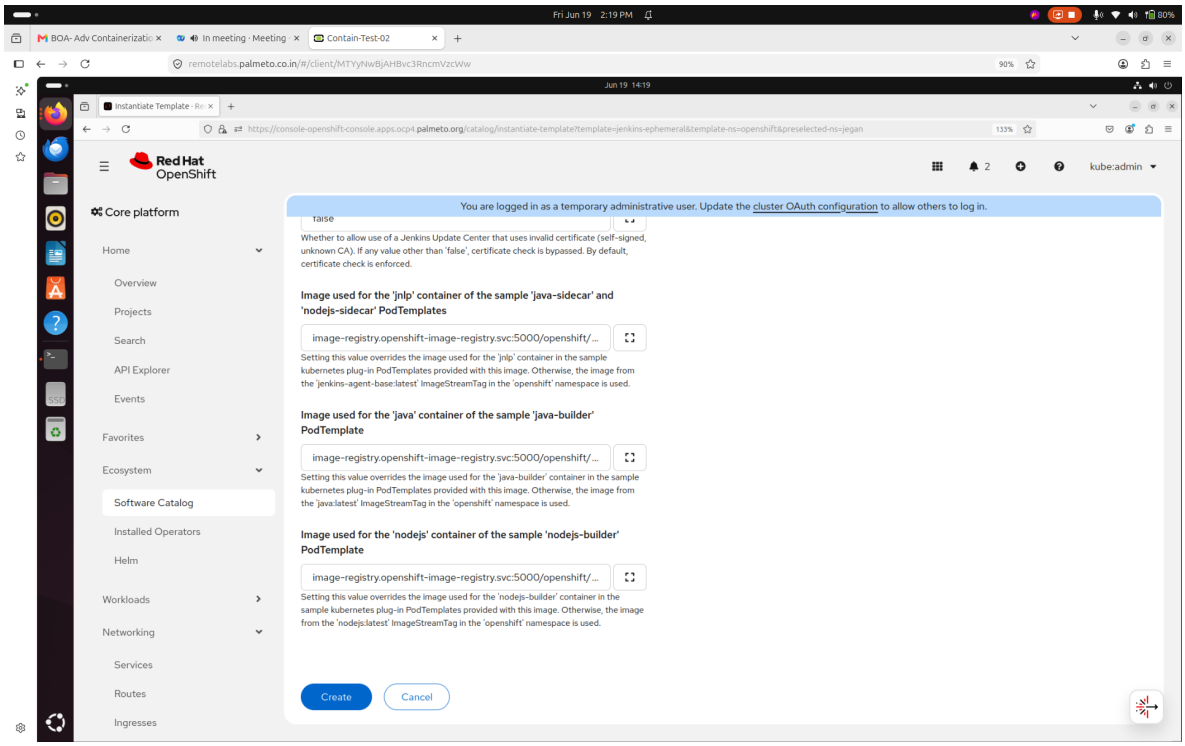

Red Hat OpenShift
Core platform
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses
kube:admin
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Projects
Project details
jegan
Active
Actions
Overview
Details
YAML
Workloads
RoleBindings
Getting started resources
Create applications using samples
Choose a code sample to get started creating an application with.
Basic Quarkus
Basic Spring Boot
View all samples
Build with guided documentation
Follow guided documentation to build applications and familiarize yourself with key features.
Get started with Quarkus using s2i
Get started with Spring
View all quick starts
Explore new developer features
Explore new features and resources within the developer perspective.
Try the sample AI Chatbot Helm chart
Start building your application quickly in topology
What's new in OpenShift 4.22
Details
View all
Name
jegan
Requester
kube:admin
Labels
Status
Active
Utilization
1 hour
Resource
Usage
1:30 PM
2:00 PM
Activity
Ongoing
There are no ongoing activities.
Recent events
Pause
There are no recent events.

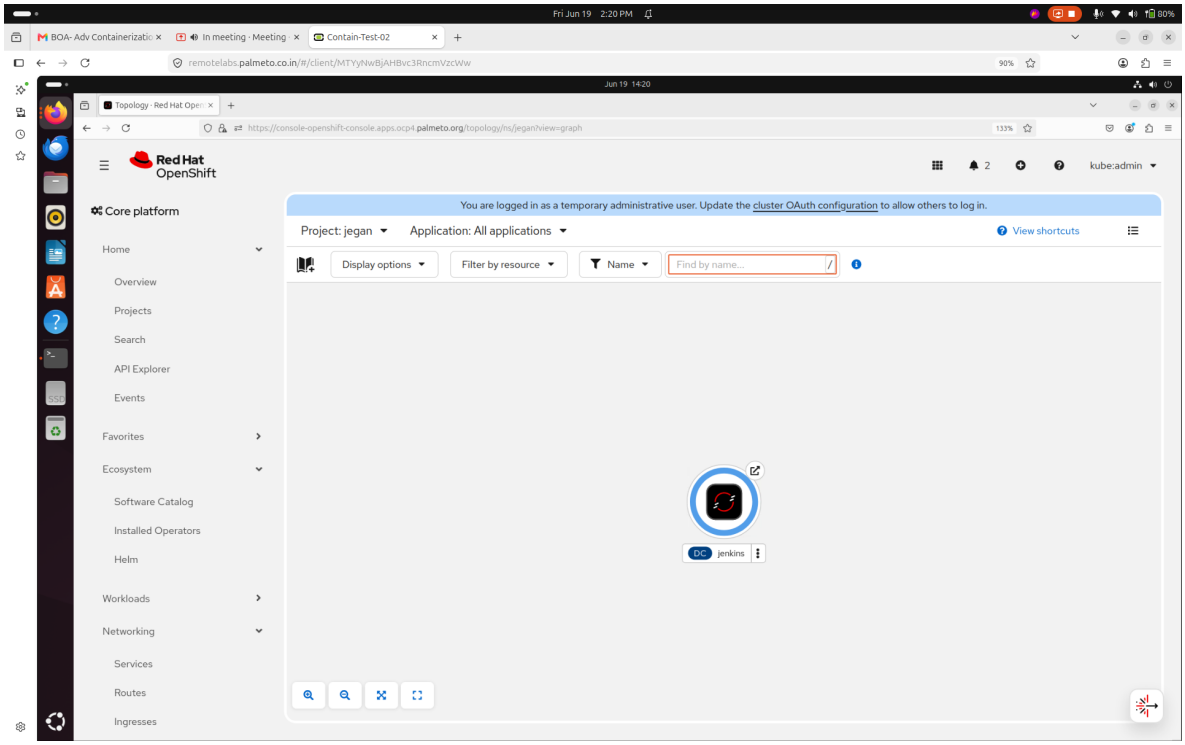

Red Hat OpenShift
Core platform
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses
kube:admin
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Software Catalog
Add shared applications, services, event sources, or source-to-image builders to your Project from the software catalog. Cluster administrators can customize the content made available in the catalog.
All items
AI/Machine Learning
Application Runtime
Big Data
CI/CD
Cloud Provider
Database
Databases
Developer Tools
Drivers and plugins
Integration & Delivery
Languages
All items
Filter by keyword...
Relevance
705 items
Helm Charts
.NET Application
Provided by Red Hat
A Helm chart to build and deploy .NET applications
Community
[DEPRECATED] Kuadrant Operator
Provided by Red Hat
[DEPRECATED] Follow https://docs.kuadrant.io instead
Community
3scale API Management
Provided by Red Hat
3scale Operator to provision 3scale and publish/manage API
Helm Charts
AAP Ansible automation portal
Provided by Red Hat
Red Hat
Advanced Cluster Management for Kubernetes
Provided by Red Hat
Red Hat
Advanced Cluster Security for Kubernetes
Provided by Red Hat

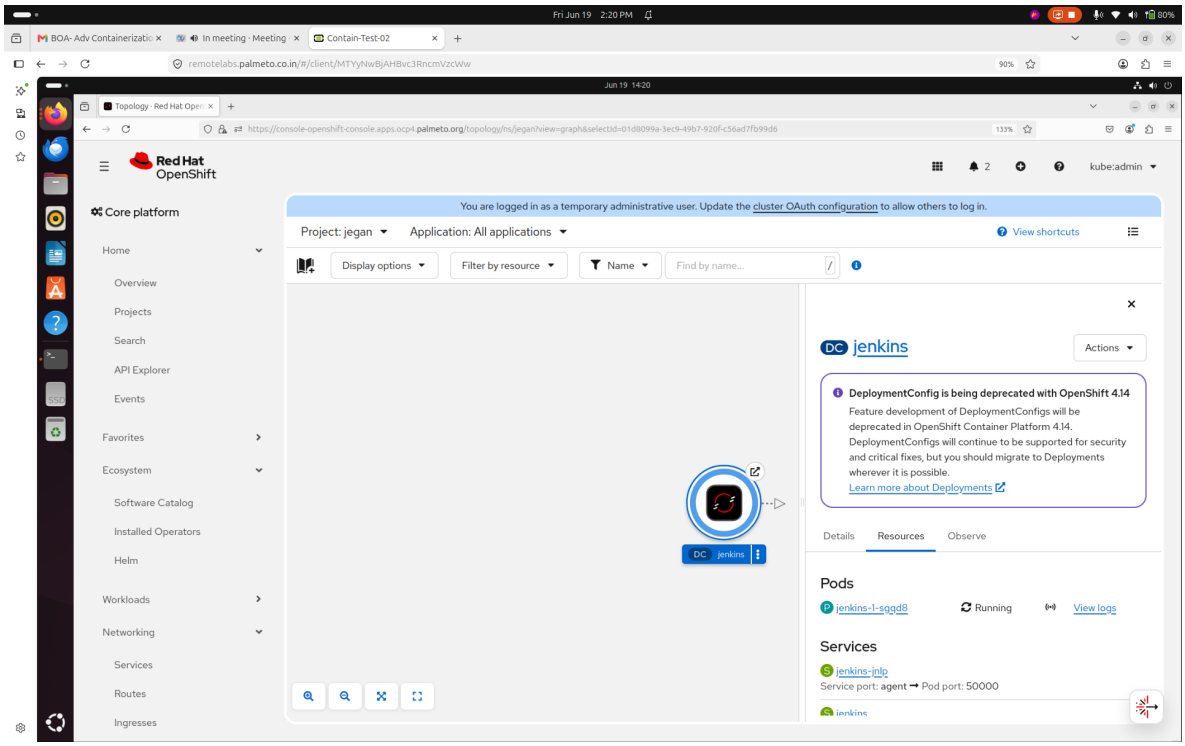

Red Hat OpenShift
Core platform
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses
kube:admin
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Software Catalog
Add shared applications, services, event sources, or source-to-image builders to your Project from the software catalog. Cluster administrators can customize the content made available in the catalog.
All items
AI/Machine Learning
Application Runtime
Big Data
CI/CD
Jenkins
Cloud Provider
Database
Databases
Developer Tools
Drivers and plugins
Integration & Delivery
CI/CD
Filter by keyword...
Relevance
5 items
Templates
Jenkins
Provided by Red Hat, Inc.
Jenkins service, with persistent storage. NOTE: You must have persistent volumes available in...
Helm Charts
Jenkins
Provided by Red Hat
Jenkins is an open source automation server which enables developers to build, test, and...
Templates
Jenkins (Ephemeral)
Provided by Red Hat, Inc.
Jenkins service, without persistent storage. WARNING: Any data stored will be lost upon...
Templates
Jenkins Monitored
Provided by Red Hat, Inc.
Templates
Jenkins Monitored (Ephemeral)
Provided by Red Hat, Inc.

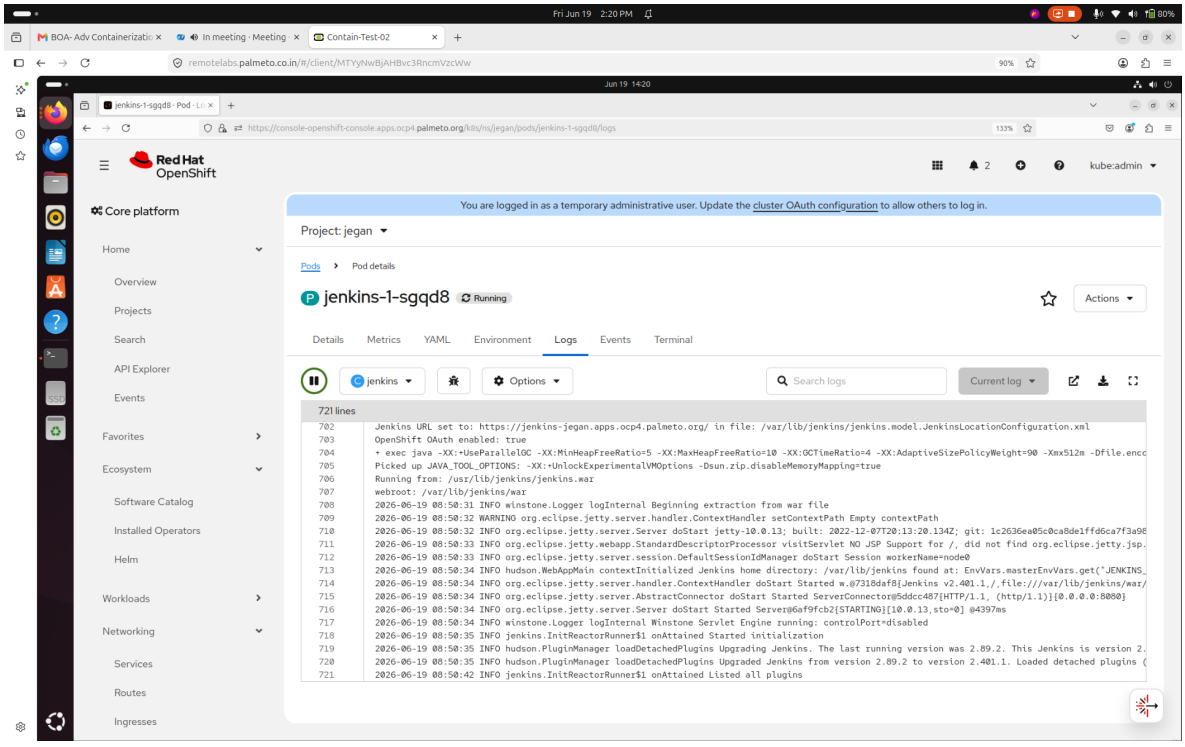

Red Hat OpenShift
Core platform
Project: jegan
Software Catalog
Jenkins (Ephemeral)
Provided by Red Hat, Inc.
Instantiate Template
Provider
Red Hat, Inc.
Created at
Jun 14, 2026, 8:37 PM
Support
Get support
Documentation
Refer documentation
Jenkins service, without persistent storage.
WARNING: Any data stored will be lost upon pod destruction. Only use this template for testing.

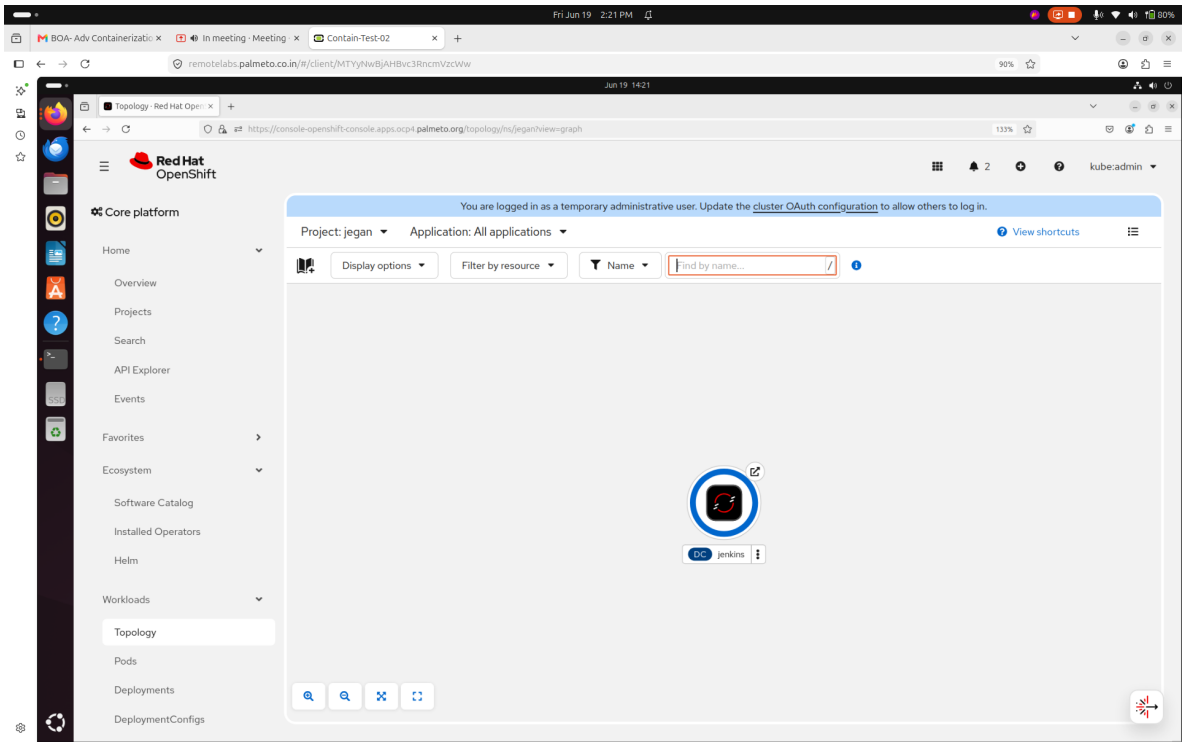

Red Hat OpenShift
Core platform
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Instantiate Template
Namespace *
jegan
Jenkins Service Name
jenkins
The name of the OpenShift Service exposed for the Jenkins container.
Jenkins JNLP Service Name
jenkins-jnlp
The name of the service used for master/slave communication.
Enable OAuth in Jenkins
true
Whether to enable OAuth OpenShift integration. If false, the static account 'admin' will be initialized with the password 'password'.
Memory Limit
1Gi
Maximum amount of memory the container can use.
Jenkins ImageStream Namespace
openshift
Jenkins (Ephemeral)
INSTANT-APP JENKINS
View documentation
Get support
Jenkins service, without persistent storage.
WARNING: Any data stored will be lost upon pod destruction. Only use this template for testing.
The following resources will be created:
ConfigMap
DeploymentConfig
RoleBinding
Route
Service
ServiceAccount

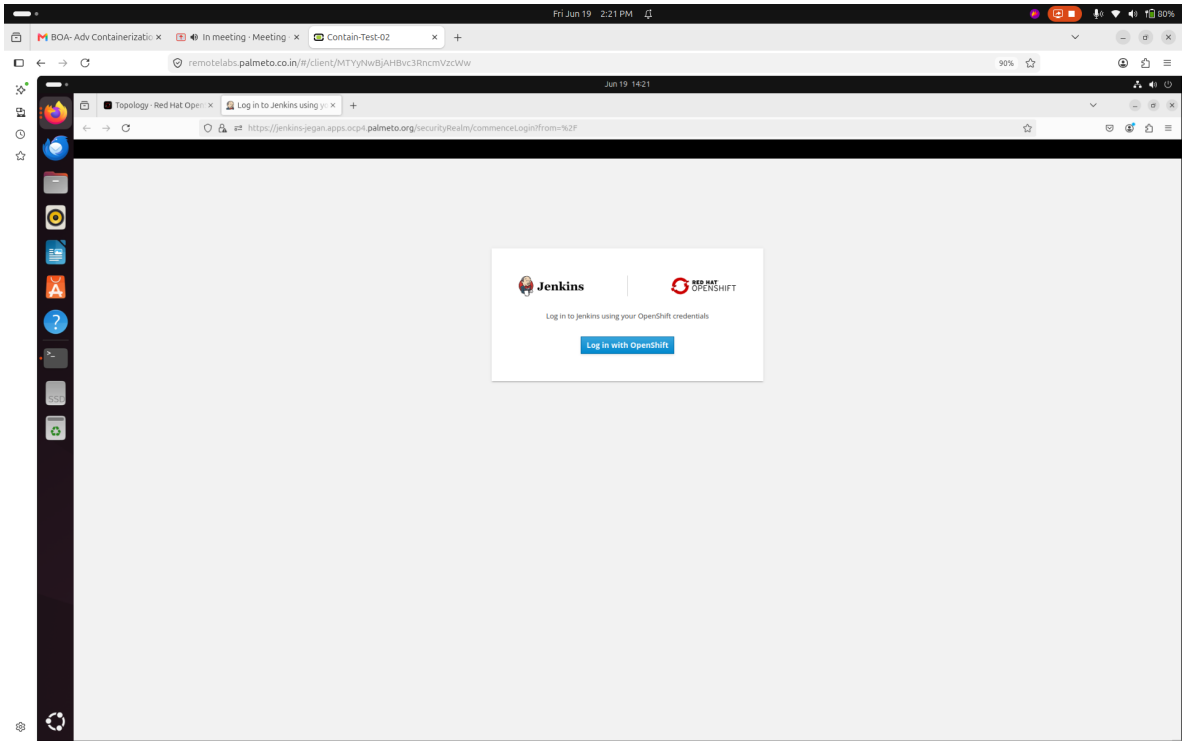

Red Hat OpenShift
kube:admin
Core platform
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses
false
Whether to allow use of a Jenkins Update Center that uses invalid certificate (self-signed, unknown CA). If any value other than 'false', certificate check is bypassed. By default, certificate check is enforced.
Image used for the 'jnlp' container of the sample 'java-sidecar' and 'nodejs-sidecar' PodTemplates
image-registry.openshift-image-registry.svc:5000/openshift/...
Setting this value overrides the image used for the 'jnlp' container in the sample kubernetes plug-in PodTemplates provided with this image. Otherwise, the image from the 'jenkins-agent-base:latest' imageStreamTag in the 'openshift' namespace is used.
Image used for the 'java' container of the sample 'java-builder' PodTemplate
image-registry.openshift-image-registry.svc:5000/openshift/...
Setting this value overrides the image used for the 'java-builder' container in the sample kubernetes plug-in PodTemplates provided with this image. Otherwise, the image from the 'java:latest' ImageStreamTag in the 'openshift' namespace is used.
Image used for the 'nodejs' container of the sample 'nodejs-builder' PodTemplate
image-registry.openshift-image-registry.svc:5000/openshift/...
Setting this value overrides the image used for the 'nodejs-builder' container in the sample kubernetes plug-in PodTemplates provided with this image. Otherwise, the image from the 'nodejs:latest' ImageStreamTag in the 'openshift' namespace is used.
Create
Cancel

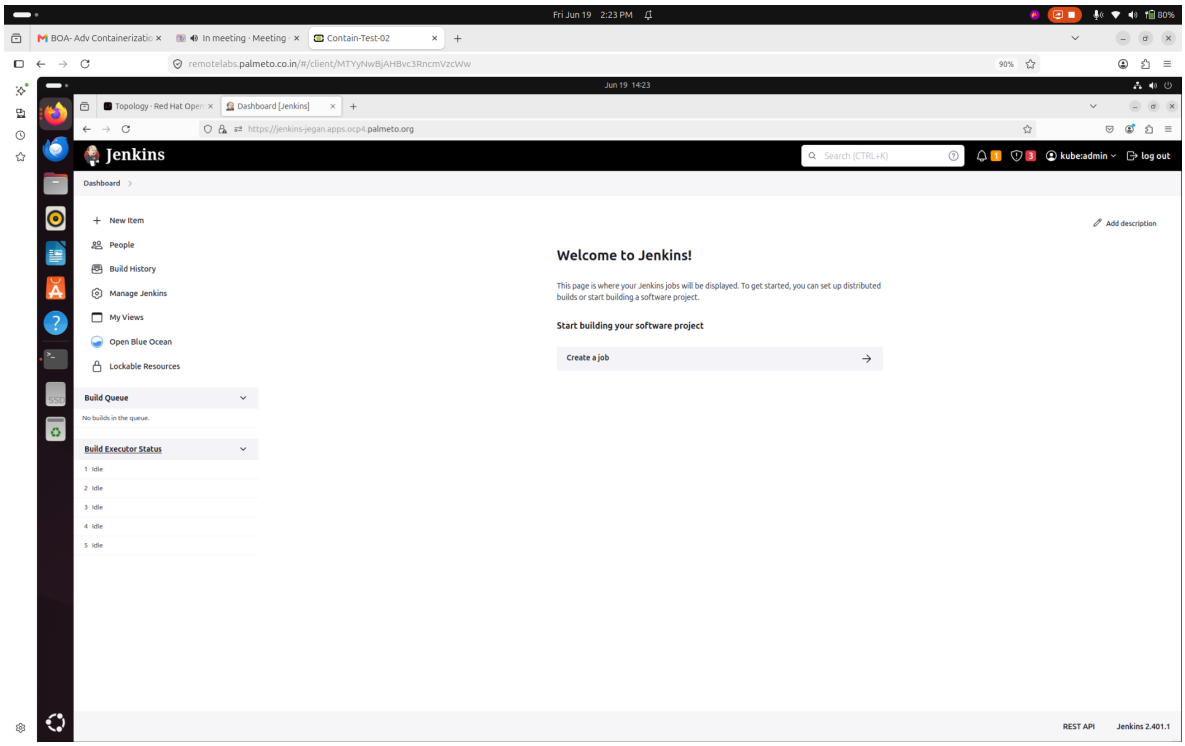

Red Hat OpenShift
kube:admin
Core platform
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Application: All applications
View shortcuts
Display options
Filter by resource
Name
Find by name...
DC jenkins
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses

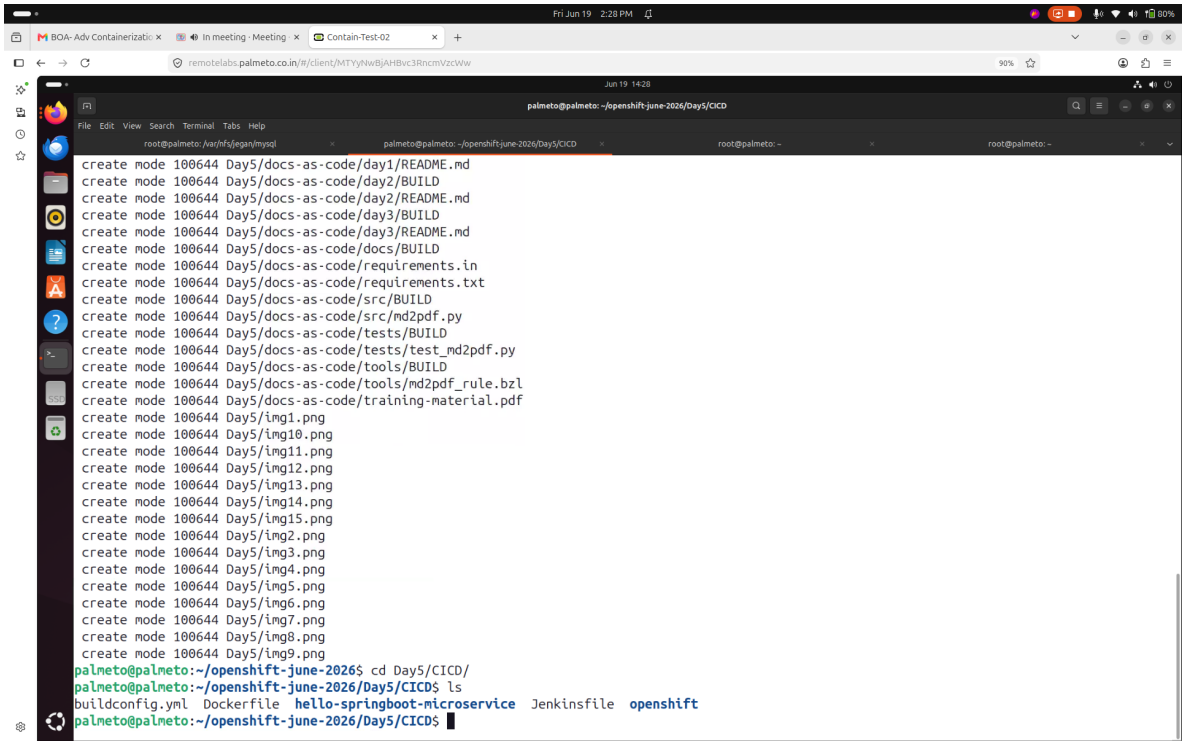

Red Hat OpenShift
Core platform
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Application: All applications
View shortcuts
Display options
Filter by resource
Name
Find by name...
DC jenkins
Actions
DeploymentConfig is being deprecated with OpenShift 4.14
Feature development of DeploymentConfigs will be deprecated in OpenShift Container Platform 4.14. DeploymentConfigs will continue to be supported for security and critical fixes, but you should migrate to Deployments wherever it is possible.
Learn more about Deployments
Details
Resources
Observe
Pods
jenkins-1-sgqd8
Running
View logs
Services
jenkins-jnlp
Service port: agent → Pod port: 50000
jenkins

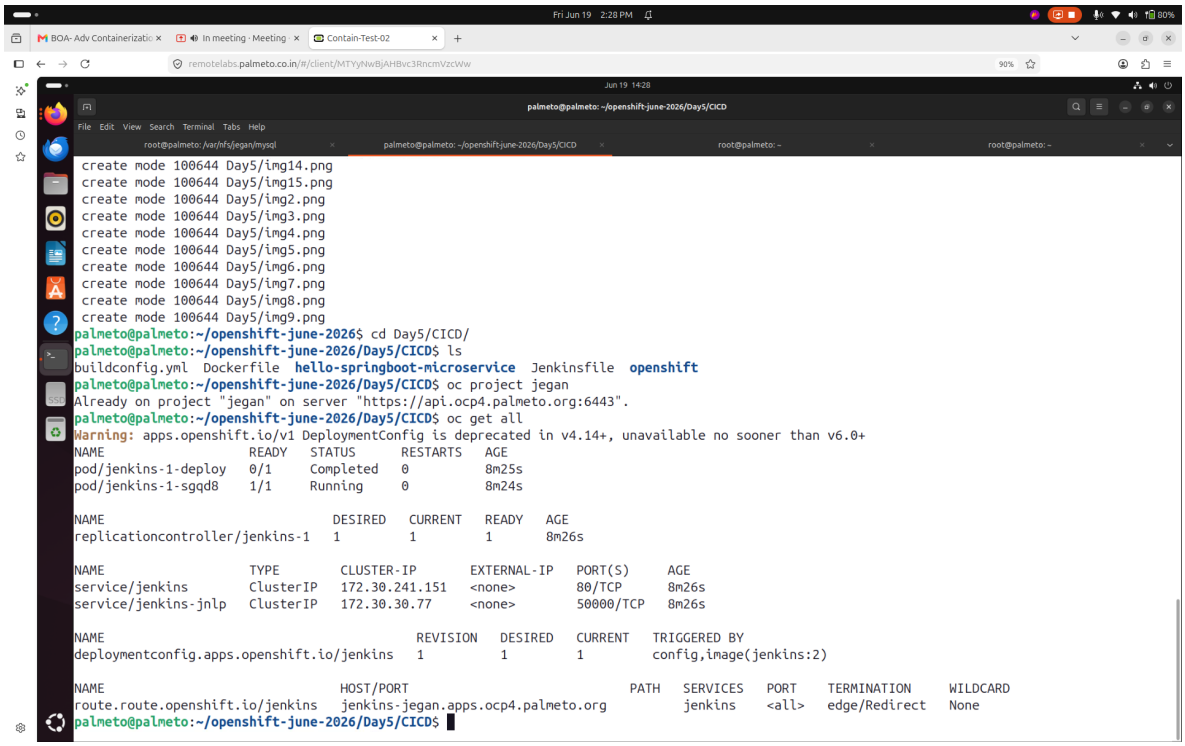

Red Hat OpenShift
Core platform
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Networking
Services
Routes
Ingresses
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Pods
Pod details
jenkins-1-sgqd8
Running
Actions
Details
Metrics
YAML
Environment
Logs
Events
Terminal
jenkins
Options
Search logs
Current log
721 lines
Jenkins URL set to: https://jenkins-jegan.apps.ocp4.palmeto.org/ in file: /var/lib/jenkins/jenkins.model.JenkinsLocationConfiguration.xml
OpenShift OAuth enabled: true
+ exec java -XX:+UseParallelGC -XX:MinHeapFreeRatio=5 -XX:MaxHeapFreeRatio=10 -XX:GCTimeRatio=4 -XX:AdaptiveSizePolicyWeight=90 -Xmx512m -Dfile.enc
Picked up JAVA_TOOL_OPTIONS: -XX:+UnlockExperimentalVMOptions -Dsun.zip.disableMemoryMapping=true
Running from: /usr/lib/jenkins/jenkins.war
webroot: /var/lib/jenkins/war
2026-06-19 08:50:31 INFO winstone.Logger logInternal Beginning extraction from war file
2026-06-19 08:50:32 WARNING org.eclipse.jetty.server.handler.ContextHandler setContextPath Empty contextPath
2026-06-19 08:50:32 INFO org.eclipse.jetty.server.Server doStart jetty-10.0.13; built: 2022-12-07T20:13:20.134Z; git: 1c2636ea05c0ca8de1ffd6ca7f3a9
2026-06-19 08:50:33 INFO org.eclipse.jetty.webapp.StandardDescriptorProcessor visitServlet NO JSP Support for /, did not find org.eclipse.jetty.jsp.
2026-06-19 08:50:33 INFO org.eclipse.jetty.server.session.DefaultSessionIdManager doStart Session workerName=node0
2026-06-19 08:50:34 INFO hudson.WebAppMain contextInitialized Jenkins home directory: /var/lib/jenkins found at: EnvVars.masterEnvVars.get("JENKINS_
2026-06-19 08:50:34 INFO org.eclipse.jetty.server.handler.ContextHandler doStart Started w.@7318daf8{Jenkins v2.401.1,/,file:///var/lib/jenkins/war/
2026-06-19 08:50:34 INFO org.eclipse.jetty.server.AbstractConnector doStart Started ServerConnector@5ddcc487{HTTP/1.1, (http/1.1)}{0.0.0.0:8080}
2026-06-19 08:50:34 INFO org.eclipse.jetty.server.Server doStart Started Server@6af9fcb2{STARTING}[10.0.13,sto=0] @4397ms
2026-06-19 08:50:34 INFO winstone.Logger logInternal Winstone Servlet Engine running: controlPort=disabled
2026-06-19 08:50:35 INFO jenkins.InitReactorRunner$1 onAttained Started initialization
2026-06-19 08:50:35 INFO hudson.PluginManager loadDetachedPlugins Upgrading Jenkins. The last running version was 2.89.2. This Jenkins is version 2.
2026-06-19 08:50:35 INFO hudson.PluginManager loadDetachedPlugins Upgraded Jenkins from version 2.89.2 to version 2.401.1. Loaded detached plugins (
2026-06-19 08:50:42 INFO jenkins.InitReactorRunner$1 onAttained Listed all plugins

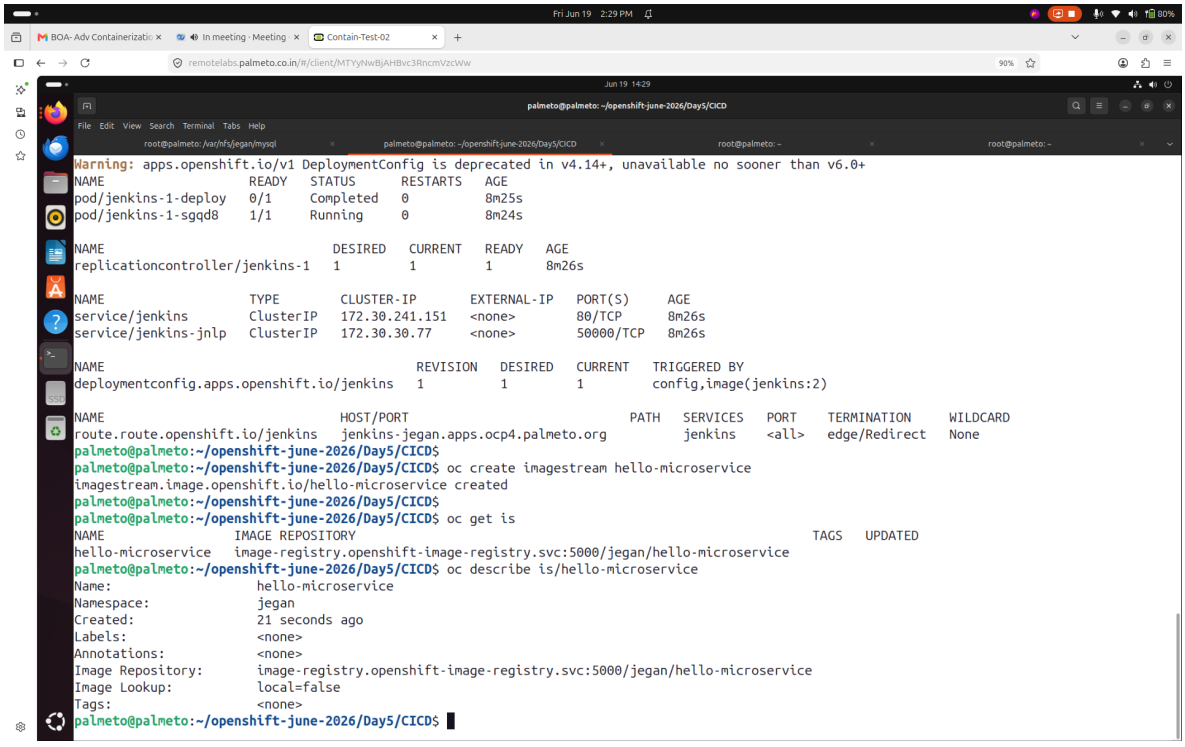

Red Hat OpenShift
Core platform
Home
Overview
Projects
Search
API Explorer
Events
Favorites
Ecosystem
Software Catalog
Installed Operators
Helm
Workloads
Topology
Pods
Deployments
DeploymentConfigs
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Application: All applications
View shortcuts
Display options
Filter by resource
Name
Find by name...
DC jenkins

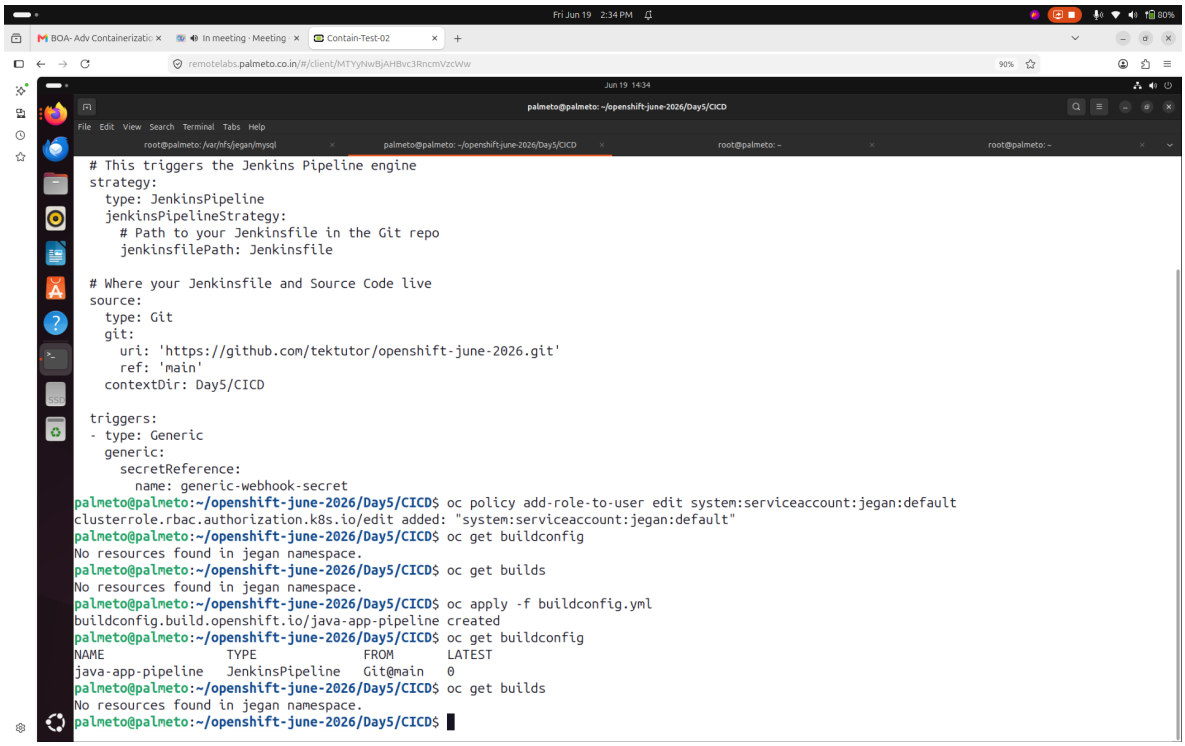

Jenkins
Log in to Jenkins using your OpenShift credentials
Log in with OpenShift

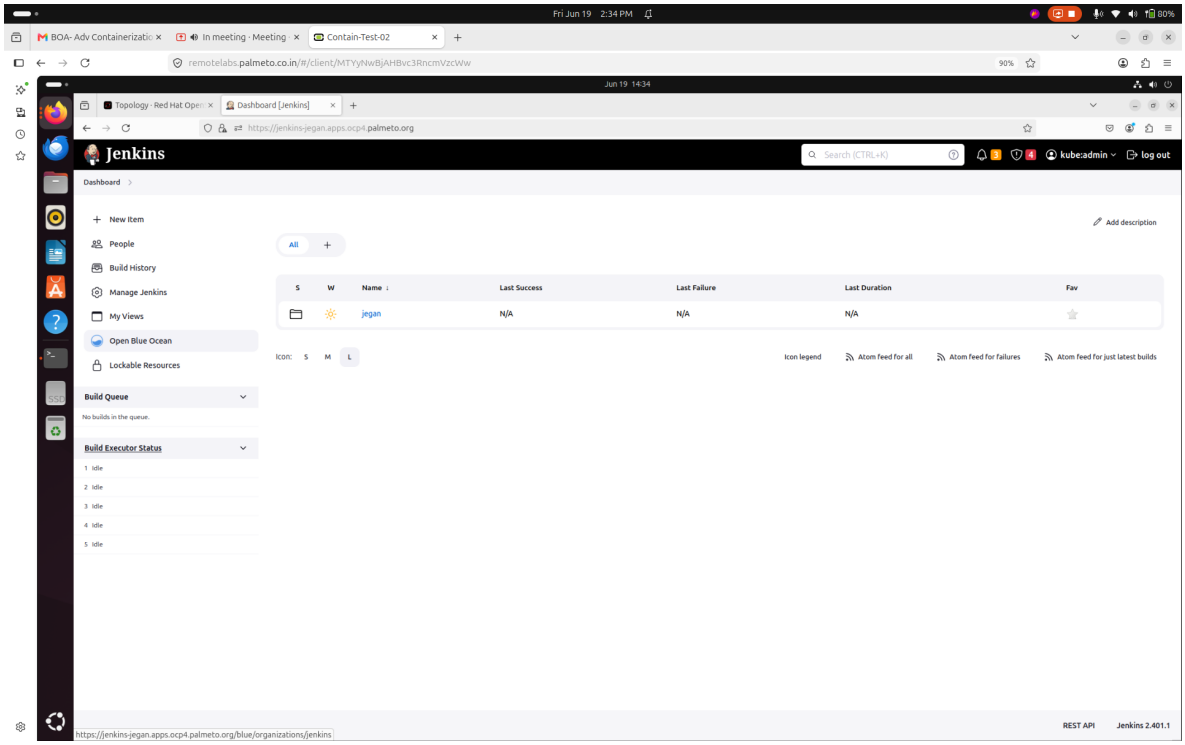

Jenkins
Dashboard
New Item
People
Build History
Manage Jenkins
My Views
Open Blue Ocean
Lockable Resources
Build Queue
No builds in the queue.
Build Executor Status
Welcome to Jenkins!
This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.
Start building your software project
Create a job
Add description
REST API
Jenkins 2.401.1

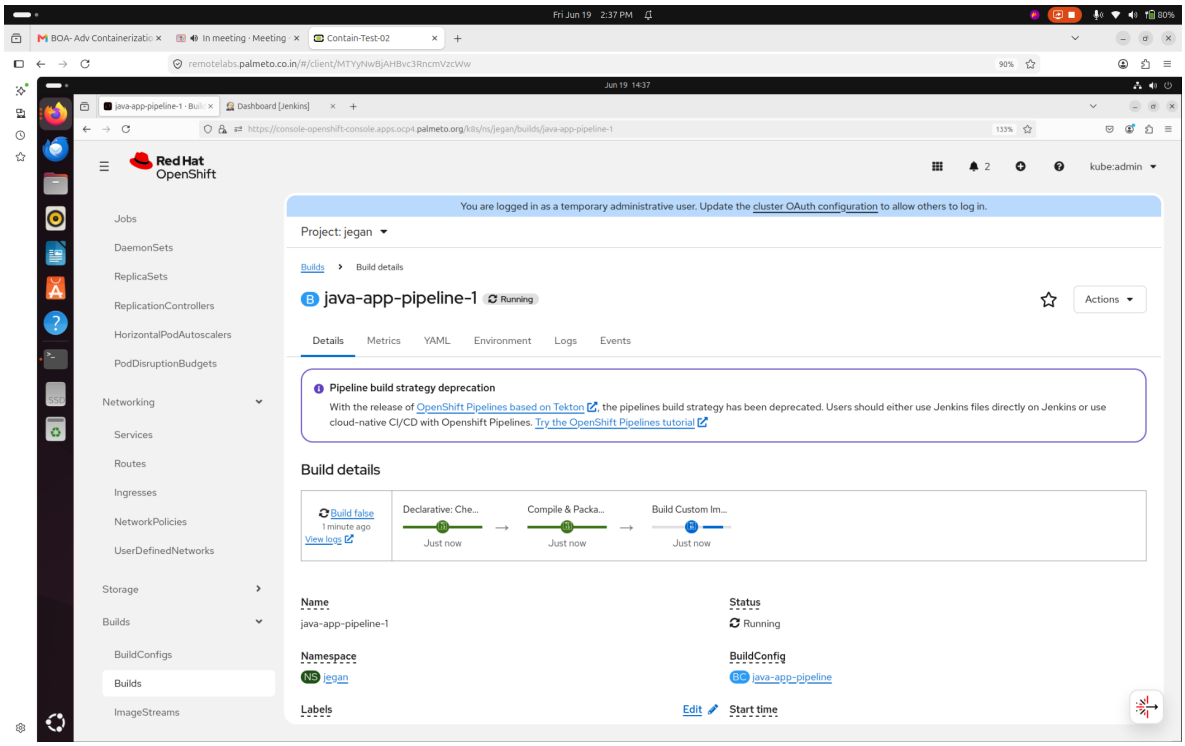

create mode 100644 Day5/docs-as-code/day1/README.md
create mode 100644 Day5/docs-as-code/day2/BUILD
create mode 100644 Day5/docs-as-code/day2/README.md
create mode 100644 Day5/docs-as-code/day3/BUILD
create mode 100644 Day5/docs-as-code/day3/README.md
create mode 100644 Day5/docs-as-code/docs/BUILD
create mode 100644 Day5/docs-as-code/requirements.in
create mode 100644 Day5/docs-as-code/requirements.txt
create mode 100644 Day5/docs-as-code/src/BUILD
create mode 100644 Day5/docs-as-code/src/md2pdf.py
create mode 100644 Day5/docs-as-code/tests/BUILD
create mode 100644 Day5/docs-as-code/tests/test_md2pdf.py
create mode 100644 Day5/docs-as-code/tools/BUILD
create mode 100644 Day5/docs-as-code/tools/md2pdf_rule.bzl
create mode 100644 Day5/docs-as-code/training-material.pdf
create mode 100644 Day5/img1.png
create mode 100644 Day5/img10.png
create mode 100644 Day5/img11.png
create mode 100644 Day5/img12.png
create mode 100644 Day5/img13.png
create mode 100644 Day5/img14.png
create mode 100644 Day5/img15.png
create mode 100644 Day5/img2.png
create mode 100644 Day5/img3.png
create mode 100644 Day5/img4.png
create mode 100644 Day5/img5.png
create mode 100644 Day5/img6.png
create mode 100644 Day5/img7.png
create mode 100644 Day5/img8.png
create mode 100644 Day5/img9.png
palmeto@palmeto:~/openshift-june-2026$ cd Day5/CICD/
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ ls
buildconfig.yml Dockerfile hello-springboot-microservice Jenkinsfile openshift
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$

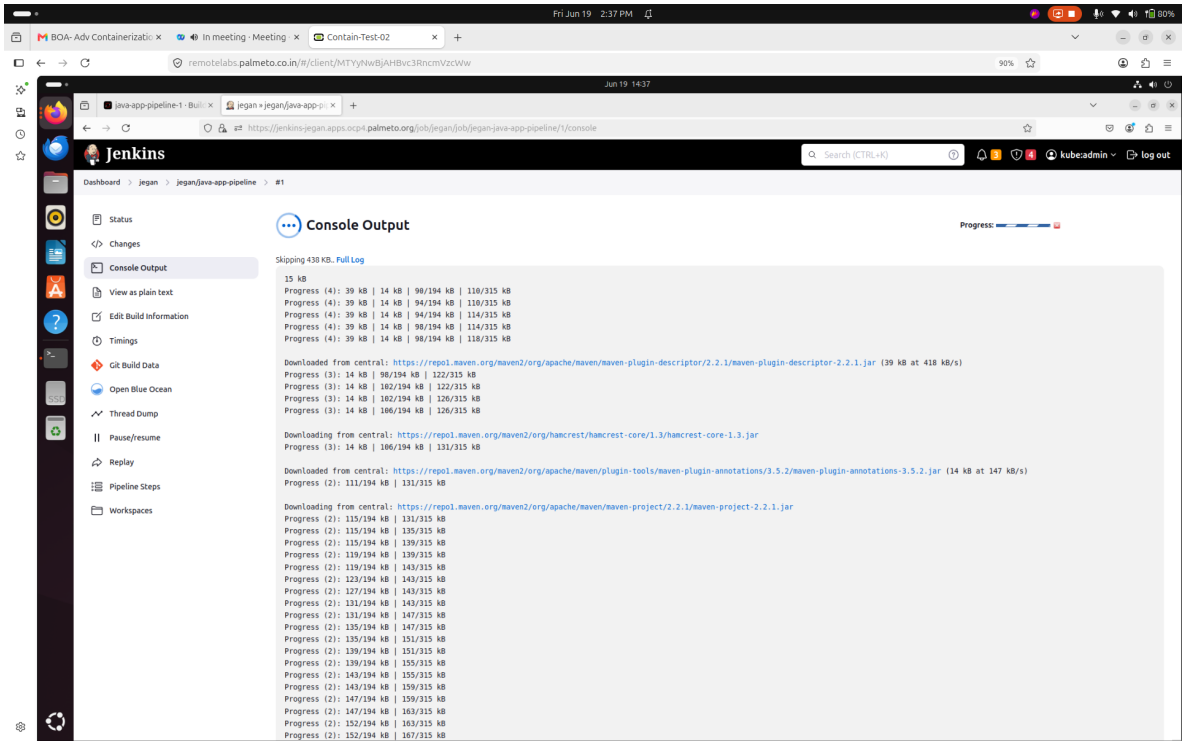

create mode 100644 Day5/img14.png
create mode 100644 Day5/img15.png
create mode 100644 Day5/img2.png
create mode 100644 Day5/img3.png
create mode 100644 Day5/img4.png
create mode 100644 Day5/img5.png
create mode 100644 Day5/img6.png
create mode 100644 Day5/img7.png
create mode 100644 Day5/img8.png
create mode 100644 Day5/img9.png
palmeto@palmeto:~/openshift-june-2026$ cd Day5/CICD/
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ ls
buildconfig.yml Dockerfile hello-springboot-microservice Jenkinsfile openshift
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc project jegan
Already on project "jegan" on server "https://api.ocp4.palmeto.org:6443".
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc get all
Warning: apps.openshift.io/v1 DeploymentConfig is deprecated in v4.14+, unavailable no sooner than v6.0+
NAME READY STATUS RESTARTS AGE
pod/jenkins-1-deploy 0/1 Completed 0 8m25s
pod/jenkins-1-sgqd8 1/1 Running 0 8m24s
NAME DESIRED CURRENT READY AGE
replicationcontroller/jenkins-1 1 1 1 8m26s
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/jenkins ClusterIP 172.30.241.151 <none> 80/TCP 8m26s
service/jenkins-jnlp ClusterIP 172.30.30.77 <none> 50000/TCP 8m26s
NAME REVISION DESIRED CURRENT TRIGGERED BY
deploymentconfig.apps.openshift.io/jenkins 1 1 1 config,image(jenkins:2)
NAME HOST/PORT PATH SERVICES PORT TERMINATION WILDCARD
route.route.openshift.io/jenkins jenkins-jegan.apps.ocp4.palmeto.org jenkins <all> edge/Redirect None
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$

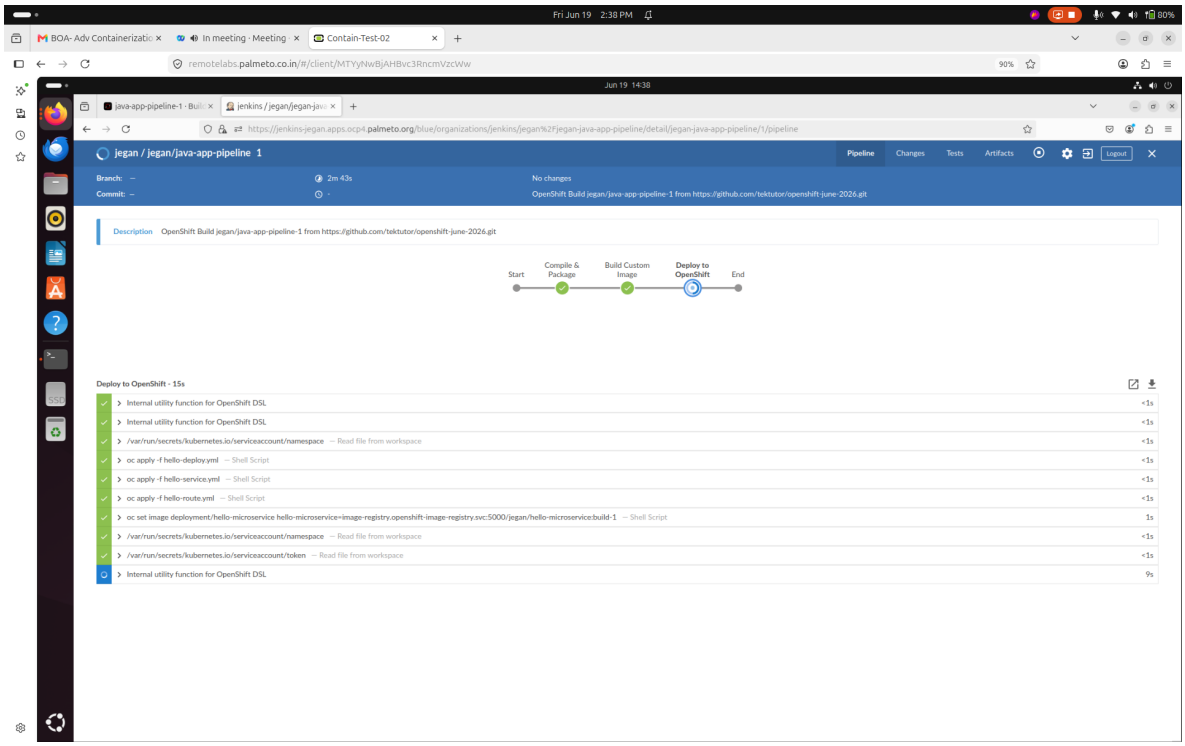

Warning: apps.openshift.io/v1 DeploymentConfig is deprecated in v4.14+, unavailable no sooner than v6.0+
NAME READY STATUS RESTARTS AGE
pod/jenkins-1-deploy 0/1 Completed 0 8m25s
pod/jenkins-1-sgqd8 1/1 Running 0 8m24s
NAME DESIRED CURRENT READY AGE
replicationcontroller/jenkins-1 1 1 1 8m26s
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/jenkins ClusterIP 172.30.241.151 <none> 80/TCP 8m26s
service/jenkins-jnlp ClusterIP 172.30.30.77 <none> 50000/TCP 8m26s
NAME REVISION DESIRED CURRENT TRIGGERED BY
deploymentconfig.apps.openshift.io/jenkins 1 1 1 config,image(jenkins:2)
NAME HOST/PORT PATH SERVICES PORT TERMINATION WILDCARD
route.route.openshift.io/jenkins jenkins-jegan.apps.ocp4.palmeto.org jenkins <all> edge/Redirect None
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc create imagestream hello-microservice
imagestream.image.openshift.io/hello-microservice created
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc get is
NAME IMAGE REPOSITORY TAGS UPDATED
hello-microservice image-registry.openshift-image-registry.svc:5000/jegan/hello-microservice
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc describe is/hello-microservice
Name: hello-microservice
Namespace: jegan
Created: 21 seconds ago
Labels: <none>
Annotations: <none>
Image Repository: image-registry.openshift-image-registry.svc:5000/jegan/hello-microservice
Image Lookup: local=false
Tags: <none>
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$

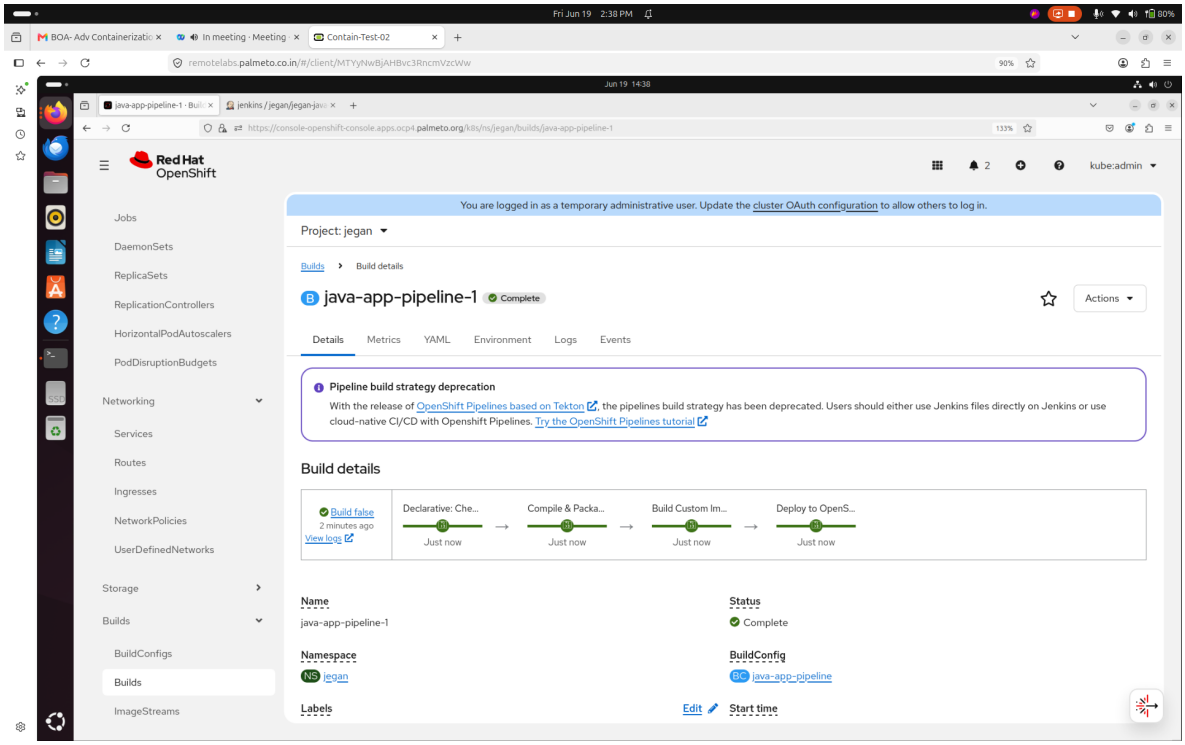

Fri Jun 19 2:34 PM
palmeto@palmeto: ~/openshift-june-2026/Day5/CICD
File Edit View Search Terminal Tabs Help
root@palmeto: /var/nfs/jegan/mysql
palmeto@palmeto: ~/openshift-june-2026/Day5/CICD
root@palmeto: ~
root@palmeto: ~
This triggers the Jenkins Pipeline engine
strategy:
type: JenkinsPipeline
jenkinsPipelineStrategy:
Path to your Jenkinsfile in the Git repo
jenkinsfilePath: Jenkinsfile
Where your Jenkinsfile and Source Code live
source:
type: Git
git:
uri: 'https://github.com/tektutor/openshift-june-2026.git'
ref: 'main'
contextDir: Day5/CICD
triggers:
- type: Generic
generic:
secretReference:
name: generic-webhook-secret
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc policy add-role-to-user edit system:serviceaccount:jegan:default
clusterrole.rbac.authorization.k8s.io/edit added: "system:serviceaccount:jegan:default"
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc get buildconfig
No resources found in jegan namespace.
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc get builds
No resources found in jegan namespace.
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc apply -f buildconfig.yml
buildconfig.build.openshift.io/java-app-pipeline created
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc get buildconfig
NAME TYPE FROM LATEST
java-app-pipeline JenkinsPipeline Git@main 0
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$ oc get builds
No resources found in jegan namespace.
palmeto@palmeto:~/openshift-june-2026/Day5/CICD$

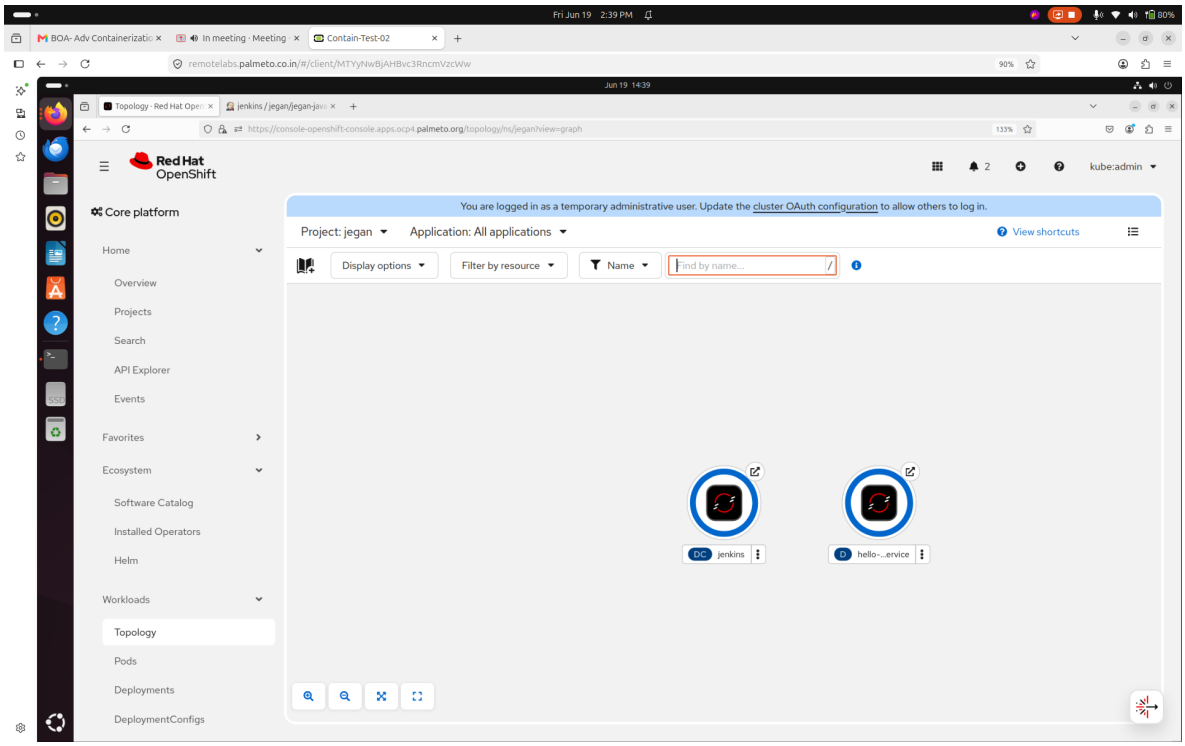

Fri Jun 19 2:34 PM
Jenkins
kube:admin
log out
Dashboard
New Item
People
Build History
Manage Jenkins
My Views
Open Blue Ocean
Lockable Resources
Build Queue
No builds in the queue.
Build Executor Status
1 Idle
2 Idle
3 Idle
4 Idle
5 Idle
Add description
All
S W Name Last Success Last Failure Last Duration Fav
jegan N/A N/A N/A
Icon: S M L
Icon legend
Atom feed for all
Atom feed for failures
Atom feed for just latest builds
REST API
Jenkins 2.401.1

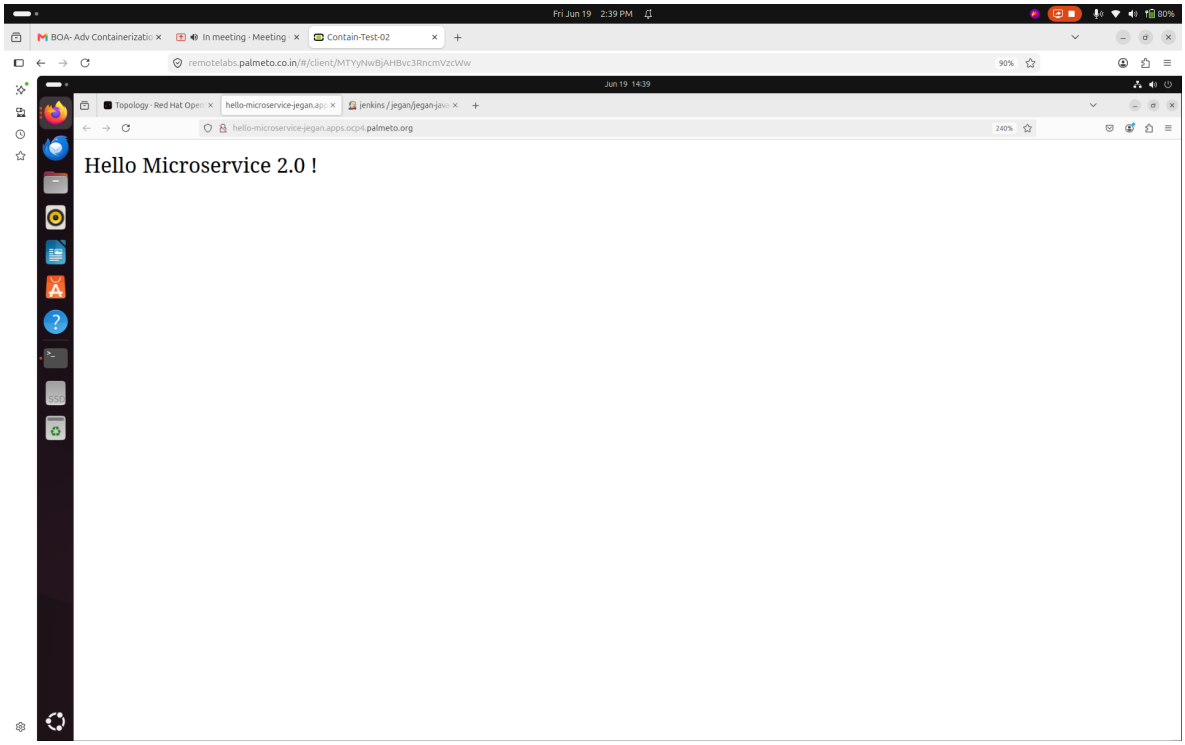

Red Hat OpenShift
kube:admin
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Builds
Build details
java-app-pipeline-1
Running
Actions
Details
Metrics
YAML
Environment
Logs
Events
Pipeline build strategy deprecation
With the release of OpenShift Pipelines based on Tekton, the pipelines build strategy has been deprecated. Users should either use Jenkins files directly on Jenkins or use cloud-native CI/CD with Openshift Pipelines. Try the OpenShift Pipelines tutorial
Build details
Build false
1 minute ago
View logs
Declarative: Che...
Compile & Packa...
Build Custom Im...
Just now
Just now
Just now
Name
java-app-pipeline-1
Status
Running
Namespace
jegan
BuildConfig
java-app-pipeline
Labels
Edit
Start time
Jobs
DaemonSets
ReplicaSets
ReplicationControllers
HorizontalPodAutoscalers
PodDisruptionBudgets
Networking
Services
Routes
Ingresses
NetworkPolicies
UserDefinedNetworks
Storage
Builds
BuildConfigs
Builds
ImageStreams


Jenkins
kube:admin
log out
Dashboard
jegan
jegan/java-app-pipeline
#1
Console Output
Progress:
Skipping 430 KB.. Full Log
15 kB
Progress (4): 39 kB | 14 kB | 90/194 kB | 110/315 kB
Progress (4): 39 kB | 14 kB | 94/194 kB | 110/315 kB
Progress (4): 39 kB | 14 kB | 94/194 kB | 114/315 kB
Progress (4): 39 kB | 14 kB | 98/194 kB | 114/315 kB
Progress (4): 39 kB | 14 kB | 98/194 kB | 118/315 kB
Downloaded from central: https://repo1.maven.org/maven2/org/apache/maven/maven-plugin-descriptor/2.2.1/maven-plugin-descriptor-2.2.1.jar (39 kB at 418 kB/s)
Progress (3): 14 kB | 98/194 kB | 122/315 kB
Progress (3): 14 kB | 102/194 kB | 122/315 kB
Progress (3): 14 kB | 102/194 kB | 126/315 kB
Progress (3): 14 kB | 106/194 kB | 126/315 kB
Downloading from central: https://repo1.maven.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar
Progress (3): 14 kB | 106/194 kB | 131/315 kB
Downloaded from central: https://repo1.maven.org/maven2/org/apache/maven/plugin-tools/maven-plugin-annotations/3.5.2/maven-plugin-annotations-3.5.2.jar (14 kB at 147 kB/s)
Progress (2): 111/194 kB | 131/315 kB
Downloading from central: https://repo1.maven.org/maven2/org/apache/maven/maven-project/2.2.1/maven-project-2.2.1.jar
Progress (2): 115/194 kB | 131/315 kB
Progress (2): 115/194 kB | 135/315 kB
Progress (2): 115/194 kB | 139/315 kB
Progress (2): 119/194 kB | 139/315 kB
Progress (2): 119/194 kB | 143/315 kB
Progress (2): 123/194 kB | 143/315 kB
Progress (2): 127/194 kB | 143/315 kB
Progress (2): 131/194 kB | 143/315 kB
Progress (2): 131/194 kB | 147/315 kB
Progress (2): 135/194 kB | 147/315 kB
Progress (2): 135/194 kB | 151/315 kB
Progress (2): 139/194 kB | 151/315 kB
Progress (2): 139/194 kB | 155/315 kB
Progress (2): 143/194 kB | 155/315 kB
Progress (2): 143/194 kB | 159/315 kB
Progress (2): 147/194 kB | 159/315 kB
Progress (2): 147/194 kB | 163/315 kB
Progress (2): 152/194 kB | 163/315 kB
Progress (2): 152/194 kB | 167/315 kB
Status
Changes
Console Output
View as plain text
Edit Build Information
Timings
Git Build Data
Open Blue Ocean
Thread Dump
Pause/resume
Replay
Pipeline Steps
Workspaces


jegan / jegan/java-app-pipeline 1
Pipeline
Changes
Tests
Artifacts
Logout
Branch: –
Commit: –
2m 43s
No changes
OpenShift Build jegan/java-app-pipeline-1 from https://github.com/tektutor/openshift-june-2026.git
Description OpenShift Build jegan/java-app-pipeline-1 from https://github.com/tektutor/openshift-june-2026.git
Start
Compile & Package
Build Custom Image
Deploy to OpenShift
End
Deploy to OpenShift - 15s
Internal utility function for OpenShift DSL
Internal utility function for OpenShift DSL
/var/run/secrets/kubernetes.io/serviceaccount/namespace — Read file from workspace
oc apply -f hello-deploy.yml — Shell Script
oc apply -f hello-service.yml — Shell Script
oc apply -f hello-route.yml — Shell Script
oc set image deployment/hello-microservice hello-microservice=image-registry.openshift-image-registry.svc:5000/jegan/hello-microservice:build-1 — Shell Script
/var/run/secrets/kubernetes.io/serviceaccount/namespace — Read file from workspace
/var/run/secrets/kubernetes.io/serviceaccount/token — Read file from workspace
Internal utility function for OpenShift DSL


Red Hat OpenShift
kube:admin
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: jegan
Jobs
DaemonSets
ReplicaSets
ReplicationControllers
HorizontalPodAutoscalers
PodDisruptionBudgets
Networking
Services
Routes
Ingresses
NetworkPolicies
UserDefinedNetworks
Storage
Builds
BuildConfigs
Builds
ImageStreams
Builds
Build details
java-app-pipeline-1
Complete
Actions
Details
Metrics
YAML
Environment
Logs
Events
Pipeline build strategy deprecation
With the release of OpenShift Pipelines based on Tekton, the pipelines build strategy has been deprecated. Users should either use Jenkins files directly on Jenkins or use cloud-native CI/CD with Openshift Pipelines. Try the OpenShift Pipelines tutorial
Build details
Build false
2 minutes ago
View logs
Declarative: Che...
Just now
Compile & Packa...
Just now
Build Custom Im...
Just now
Deploy to OpenS...
Just now
Name
java-app-pipeline-1
Status
Complete
Namespace
jegan
BuildConfig
java-app-pipeline
Labels
Edit
Start time






```
cd ~/openshift-july-2026
git pull
cd Day5/CICD
oc project jegan-project
oc apply -f buildconfig.yml
oc get buildconfigs
```

```
oc create imagestream hello-microservice
oc policy add-role-to-user edit system:serviceaccount:jegan-project:default
oc start-build bc/java-app-pipeline

oc logs -f bc/java-app-pipeline
```

Certifications Recommended

Recommended for Developers

```
- To prepare for EX288 Certification, you may attend the training that
covers topics listed in
  Containers & Kubernetes Fundamentals (DO188) and Red Hat OpenShift
Development II: Building Kubernetes Applications (DO288).
- Red Hat doesn't mandate attending training before taking EX288
Certification, those who are technically hands-on will be able
  to clear the Certification with consistent preparation
```

```
- Red Hat Certified Specialist in Containers (EX188)
- Red Hat Certified Specialist in OpenShift Application Development
(EX288)
- Red Hat Certified Cloud-Native Developer (EX378) - Quarkus/Java
Focused
- Red Hat Certified Specialist in OpenShift AI (EX267)
- Event-Driven Development (EX453 ) - Kafka/AMQ Streams
- Building Resilient Microservices EX328 - Service Mesh/Istio
```

Recommended for DevOps Engineers

```
- Red Hat Certified Specialist in OpenShift Automation and Integration
(EX380)
- Red Hat Certified Cloud-Native Developer (RHCCD)
- Red Hat Certified OpenShift Architect
- The Foundation: Red Hat Certified Engineer (EX294 - RHCE)
- The Platform: EX288 (Developer) OR EX280 (Admin)
- The Pipeline: EX288 (OpenShift Pipelines)
- The Automation Pinnacle: Red Hat Certified Specialist in MultiCluster
Management (EX480)
```

Recommended for Administrators

> - Prerequisite: Red Hat Certified System Administrator (EX200 - RHCSA)
> - Core: Red Hat Certified Specialist in OpenShift Administration (EX280)
> - Advanced: Red Hat Certified Specialist in OpenShift Automation & Integration (EX380)
> - OpenShift Virtualization (EX316) - Running VMs alongside containers
> - OpenShift Data Foundation (EX370) - Managing cluster storage/ODF